Bachelorthesis
Attribute Stream-Based Access Control im Linux-Kernel

Christian Koch

25. Oktober 2026

Inhaltsverzeichnis

Tabellenverzeichnis **iv**

Abbildungsverzeichnis **v**

1 Einleitung **1**
1.1 Motivation . . . 1
1.2 Wissenschaftliche Fragestellung . . . 1
1.3 Zielsetzung . . . 1
1.4 Aufbau der Arbeit . . . 2

2 Grundlagen **3**
2.1 Zugriffskontrolle und Reference-Monitor-Konzept . . . 3
2.2 Attribute Based Access Control . . . 4
2.3 Attribute Stream-Based Access Control . . . 5
2.4 Policy-Beschreibungen für ASBAC . . . 6
2.5 Userspace und Kernelspace in Linux . . . 7
2.6 LSM und Linux LSM-Hooks . . . 7
2.7 eBPF und eBPF-LSM . . . 8
2.8 eBPF-Maps und gepinnte Maps . . . 9
2.9 eBPF-Verifier, Stack und Programmkomplexität . . . 9
2.10 Aya als Rust-Framework für eBPF . . . 10
2.11 Dynamische Attribute und Policy-Generationen . . . 11
2.12 Überwachung bestehender Zugriffe und File Descriptors . . . 11
2.13 NixOS . . . 12
2.14 Verwandte Arbeiten und Abgrenzung des eigenen Ansatzes . . . 13

3 Anforderungsanalyse **15**
3.1 Funktionale Anforderungen . . . 15
3.2 Operative Anforderungen . . . 17
3.3 Entwicklungsbezogene Anforderungen . . . 18

4 Konzeption **20**
4.1 Entwurfsfolgen der eBPF-Einschränkungen . . . 20
4.2 Architektur . . . 20
4.3 Datenfluss . . . 21
4.3.1 Policy-Datenfluss . . . 21
4.3.2 Datenfluss dynamischer Attribute . . . 22
4.3.3 Zugriffsentscheidungen im Kernel . . . 22
4.3.4 Nachträgliche Kontrolle bestehender Zugriffe . . . 22
4.4 Fachliches Datenmodell . . . 22
4.5 Entwurfsentscheidungen . . . 23
4.5.1 Policy-Verarbeitung im Userspace und Auswertung im Kernelspace 23
4.5.2 Einsatz von eBPF-LSM statt eines klassischen LSM . . . 24
4.5.3 Auswahl des LSM-Hooks . . . 24
4.5.4 Kommunikation über eBPF-Maps . . . 25

4.5.5 Generationenmodell für Policy-Aktualisierungen 26
4.5.6 Getrennte Repräsentation statischer Policies und Streampolicies . 28
4.5.7 Tail-Call-basierte Aufteilung der eBPF-Programme 28
4.5.8 Combining-Strategie für Policy-Entscheidungen 29
4.5.9 Identifikation von Dateien über Device und Inode 30
4.5.10 Nachbewertung und selektiver FD-Entzug 31
4.5.11 Administration und Diagnose über ein separates Tool 32

5 Entwurf und Umsetzung **33**
5.1 Implementierungsstruktur 33
5.1.1 Aufteilung des Rust-Workspaces 33
5.1.2 Abhängigkeiten zwischen den Crates 33
5.2 Gemeinsame Datenstrukturen und Map-ABI 35
5.2.1 Kernelgeeignete Policy-Strukturen 35
5.2.2 Repräsentation dynamischer Attribute 36
5.2.3 Objektidentitäten im Map-Schlüssel 36
5.2.4 Aufbau und Pinning der eBPF-Maps 37
5.3 Initialisierung und Laufzeitsteuerung 38
5.3.1 Laden und Prüfen des eBPF-Programms 38
5.3.2 Initialisierung der Maps und Tail Calls 38
5.3.3 Start der Userspace-Komponenten 39
5.3.4 Gemeinsames Veröffentlichungsverfahren 39
5.4 Verarbeitung und Laden von Policies 40
5.4.1 Einlesen der Policydateien 40
5.4.2 Syntaktische Verarbeitung 40
5.4.3 Semantische Validierung 41
5.4.4 Übersetzung in Map-Einträge 42
5.4.5 Generationenwechsel und Rollback 42
5.4.6 Beispiel der Policy- und Attributsemantik 43
5.5 Bereitstellung dynamischer Attribute 43
5.5.1 Aufbau des Attributverzeichnisses 43
5.5.2 Parsen und Validieren der Attribute 44
5.5.3 Erzeugung der Attribut-Map-Einträge 44
5.5.4 Aktualisierung der Attributgeneration 45
5.5.5 Bereitstellung der Zeitinformation 45
5.6 Implementierung des Kernelspace-PEP 45
5.6.1 Verarbeitung des file_open-Hooks 46
5.6.2 Tail-Call-Kette 46
5.6.3 Auswertung statischer Policies 47
5.6.4 Auswertung von Streampolicies 47
5.6.5 Combining und Durchsetzung 48
5.7 Implementierung des Userspace-PEP 48
5.7.1 Ermittlung bestehender Dateizugriffe 48
5.7.2 Erneute Policy-Auswertung 50
5.7.3 Entzug des betroffenen File Descriptors 50
5.8 Implementierung des Administrationstools 51

5.8.1 Auswahl und Ausgabe des aktiven Zustands 51
5.8.2 Darstellung nicht umkehrbarer Hashwerte 51
5.8.3 Read-only-Schnittstelle und Fehlerverhalten 52
5.9 Fehlerbehandlung und Konsistenzsicherung 52
5.10 Build und Deployment auf dem Zielsystem 53
5.10.1 Build-Prozess und Build-Artefakte 53
5.10.2 Start auf dem Zielsystem 54
5.10.3 Implementierungsbedingte Grenzen 54

6 Evaluation **56**
6.1 Testumgebung 56
6.1.1 Hard- und Softwareumgebung 56
6.2 Teststrategie und Testszenarien 58
6.2.1 Teststufen und Bewertungskriterien 58
6.2.2 Unit- und Komponententests 60
6.2.3 Privilegierte End-to-End-Tests 61
6.3 Ergebnisse 65
6.3.1 Ergebnisse der funktionalen Prüfungen 65
6.3.2 Robustheit und Grenzen des Entzugsmechanismus 66
6.3.3 Quantitative Ergebnisse 67
6.3.4 Bewertung der Anforderungen 68

7 Zusammenfassung und Ausblick **72**
7.1 Zusammenfassung und Beantwortung der Forschungsfrage 72
7.2 Grenzen der Ergebnisse 73
7.3 Ausblick 74

Tabellenverzeichnis

1 Wirkung einer Deny-Policy mit der Bedingung `system.clearance <= 2` ohne weitere Deny-Policies 30
2 Aufteilung des Rust-Workspaces in Crates und Verantwortlichkeiten . . . 34
3 Gepinnte eBPF-Maps als Map-ABI des Prototyps 37
4 Hard- und Softwarekonfiguration der Testumgebung 58
5 Teststufen der Evaluation 59
6 Grundlegende End-to-End-Szenarien 63
7 End-to-End-Szenarien für Enforcement und Fehlerverhalten 64
8 Zusammenfassung der funktionalen Testergebnisse 66
9 Quantitative Messergebnisse auf dem Zielsystem 67
10 Einzelbewertung der Akzeptanzkriterien von OA-03 anhand des archivierten Laufs 70

Abbildungsverzeichnis

1 Konzeptionelle Architektur des Prototyps 21
2 Interne Abhängigkeiten der Rust-Crates 35

1 Einleitung

1.1 Motivation

Die Sicherheit von Linux-Systemen ist ein fundamentales Anliegen in der heutigen IT-Landschaft. Das Linux Security Module (LSM) Framework stellt Schnittstellen bereit, über die Sicherheitsmodule unterschiedliche Zugriffskontrollmodelle im Linux-Kernel durchsetzen können[Ker26p]. LSM-Hooks ermöglichen es, sicherheitsrelevante Kerneloperationen abzufangen und auf dieser Grundlage Zugriffsentscheidungen zu treffen. Ein LSM-Hook prüft einen Zugriff jedoch grundsätzlich zu dem Zeitpunkt, an dem die jeweilige Kerneloperation ausgeführt wird. Bei dauerhaft bestehenden Zugriffen wird eine spätere Änderung der Entscheidungsgrundlage dadurch nicht automatisch erneut bewertet.

1.2 Wissenschaftliche Fragestellung

Diese Arbeit untersucht, wie zentrale Prinzipien von Attribute Stream-Based Access Control (ASBAC) für Dateiöffnungen unter Linux prototypisch umgesetzt werden können. Im Mittelpunkt steht dabei die Frage, wie dynamische Attribute aus dem Userspace für Zugriffsentscheidungen eines extended Berkeley Packet Filter (eBPF)-LSM-Programms bereitgestellt und Änderungen bei bereits bestehenden Dateizugriffen nachträglich berücksichtigt werden können.

Daraus ergibt sich folgende wissenschaftliche Fragestellung:

> Wie kann ASBAC für Dateiöffnungen prototypisch mittels eBPF-LSM unter Linux umgesetzt werden, und welche technischen Einschränkungen ergeben sich insbesondere bei der nachträglichen Kontrolle bereits bestehender Dateizugriffe?

Zur Beantwortung dieser Fragestellung werden insbesondere die Überführung textueller Policy-Dateien in eine kernelgeeignete Repräsentation, die Bereitstellung dynamischer Attribute für Zugriffsentscheidungen im Kernel sowie die nachträgliche Überwachung bestehender Zugriffssituationen betrachtet.

1.3 Zielsetzung

Zielsetzung der Arbeit ist ein Prototyp, der zentrale Eigenschaften von ASBAC auf die Kontrolle von Dateiöffnungen unter Linux überträgt. Insbesondere sollen Stream-Attribute wie Zeitinformationen sowie externe, zur Laufzeit aktualisierte Attribute in die Zugriffsentscheidung einbezogen werden. Der Prototyp soll textuelle Policies einlesen, in eine kernelgeeignete Repräsentation übersetzen und neue Dateiöffnungen mittels eBPF-LSM im Kernel bewerten. Nach der erfolgreichen Aktivierung geänderter Policies oder Attribute sowie beim Erreichen entscheidungsrelevanter Zeitgrenzen soll er bereits bestehende

Dateizugriffe im Userspace ereignisbasiert erneut bewerten und bei einer festgestellten Verletzung einen Entzug versuchen.

Nicht Gegenstand der Arbeit ist die vollständige Implementierung der Publish-Subscribe-Architektur der Streaming Attribute Policy Language (SAPL) oder eines allgemeinen, eigenständigen ASBAC-Policy Decision Point (PDP). Der Prototyp ist auf Dateiöffnungen über den LSM-Hook `file_open` und auf die nachträgliche Untersuchung geöffneter File Descriptors (FD) beschränkt.

1.4 Aufbau der Arbeit

Kapitel 2 führt die für die Arbeit erforderlichen Grundlagen ein. Dazu gehören der Reference-Monitor-Ansatz, Attribute-Based Access Control und ASBAC sowie die für die Umsetzung relevanten Linux-Technologien LSM, eBPF und eBPF-Maps. Zudem wird der eigene Ansatz gegenüber der Publish-Subscribe-Architektur von SAPL und verwandten Konzepten fortdauernder Autorisierung abgegrenzt.

Kapitel 3 leitet aus der Zielsetzung die funktionalen, operativen und entwicklungsbezogenen Anforderungen an den Prototyp ab.

Kapitel 4 entwickelt darauf aufbauend die Systemarchitektur und begründet die wesentlichen Entwurfsentscheidungen. Im Mittelpunkt stehen die Aufgabenteilung zwischen Userspace und Kernelspace, die Repräsentation und konsistente Aktualisierung von Policies und Attributen sowie die ergänzende Nachbewertung bereits geöffneter FD.

Kapitel 5 beschreibt die konkrete Umsetzung des Prototyps. Es erläutert die Struktur des Rust-Workspace, die gemeinsame Map-Application Binary Interface (ABI), die Policy- und Attributloader, den Kernelspace-Polcy Enforcement Point (PEP) auf Grundlage des LSM-Hooks `file_open`, den Userspace-PEP und das Administrationstool. Darüber hinaus werden Fehlerbehandlung, Build, Deployment und implementierungsbedingte Grenzen dargestellt.

Kapitel 6 evaluiert den Prototyp auf einem dedizierten Linux-Zielsystem. Unit-, Komponenten- und End-to-End-Tests untersuchen das funktionale Verhalten und ausgewählte Fehlerfälle; ergänzende Messungen charakterisieren den Aufwand kontrollierter Dateiöffnungen sowie die Reaktions- und Entzugslatenz.

Kapitel 7 beantwortet abschließend die Forschungsfrage, ordnet die Aussagekraft der Ergebnisse ein und zeigt Ansatzpunkte für weiterführende Arbeiten auf.

2 Grundlagen

Dieses Kapitel beschreibt die fachlichen und technischen Grundlagen, auf denen der in dieser Arbeit entwickelte Prototyp aufbaut. Zunächst werden grundlegende Konzepte der Zugriffskontrolle eingeordnet, insbesondere das Reference-Monitor-Konzept sowie attributbasierte Zugriffskontrollmodelle. Darauf aufbauend wird erläutert, wie Attribute in dynamischen Systemen als veränderliche Entscheidungsgrundlage verwendet werden können und welche Rolle textuelle Policy-Beschreibungen dabei einnehmen.

Für die Umsetzung im Linux-Kernel sind außerdem die Trennung zwischen Userspace und Kernelspace, LSM sowie eBPF relevant. Diese Mechanismen bestimmen, an welchen Stellen Zugriffsentscheidungen technisch getroffen werden können und welche Einschränkungen bei der Implementierung im Kernel gelten. Ein besonderer Fokus liegt dabei auf eBPF-Maps als Schnittstelle zwischen Userspace und Kernelspace, gepinnten Maps für dauerhaft zugreifbare Zustände sowie den Vorgaben des eBPF-Verifiers.

Abschließend werden die für den Prototyp spezifischen Grundlagen betrachtet: die Verwendung von Aya als Rust-Framework, die Abbildung dynamischer Attribute und Policy-Generationen sowie die Überwachung bereits bestehender Zugriffe über FD. Damit schafft das Kapitel die Grundlage für die anschließende Anforderungsanalyse und die spätere Architektur des Systems.

2.1 Zugriffskontrolle und Reference-Monitor-Konzept

Zugriffskontrolle beschreibt Mechanismen, mit denen entschieden wird, ob ein Subjekt eine bestimmte Operation auf einer Ressource ausführen darf. Ein Subjekt kann dabei beispielsweise ein Prozess oder ein Benutzer sein, während Ressourcen Dateien, Netzwerkendpunkte oder andere geschützte Objekte des Systems sein können. Eine Zugriffsentscheidung basiert auf einer Menge von Policies, die festlegen, unter welchen Bedingungen eine Operation erlaubt oder verweigert wird. Diese Sichtweise knüpft an grundlegende Begriffe der Informationsschutzmechanismen an, bei denen Akteure, geschützte Objekte und Berechtigungen als zentrale Elemente der Zugriffskontrolle betrachtet werden [SS75].

Ein grundlegendes Konzept für die Durchsetzung von Zugriffskontrolle ist der Reference Monitor. Der Reference Monitor ist eine abstrakte Sicherheitskomponente, die jeden Zugriff eines Subjekts auf eine geschützte Ressource überprüft, bevor dieser Zugriff ausgeführt wird. Damit ein Reference Monitor wirksam ist, muss er insbesondere drei Eigenschaften erfüllen: Er muss bei jedem relevanten Zugriff durchlaufen werden, gegen Manipulation geschützt sein und hinreichend klein beziehungsweise nachvollziehbar sein, damit seine Korrektheit überprüft werden kann [And72, Abschnitt 3.4, S. 17].

Für Betriebssysteme ist dieses Konzept besonders relevant, da Zugriffe auf Dateien, Prozesse oder andere Systemressourcen letztlich über den Kernel vermittelt werden. Wird Zugriffskontrolle erst außerhalb des Kernels umgesetzt, besteht die Gefahr, dass sicherheitsrelevante Operationen nicht vollständig erfasst oder durch alternative Zugriffspfade

umgangen werden. Eine Zugriffskontrolle im Kernel kann dagegen an zentralen Stellen ansetzen, an denen Operationen auf geschützte Ressourcen tatsächlich ausgeführt werden. Dies steht im Zusammenhang mit dem Prinzip der vollständigen Vermittlung, nach dem jeder Zugriff auf eine geschütztee Ressource vor seiner Ausführung geprüft werden muss [SS75].

Im Kontext dieser Arbeit bildet das Reference-Monitor-Konzept die Grundlage für die Frage, wie ASBAC im Linux-Kernel umgesetzt werden kann. Der Prototyp soll Zugriffe nicht nur anhand statischer Regeln bewerten, sondern dynamische Attribute in die Entscheidung einbeziehen. Dadurch entsteht zusätzlich die Herausforderung, dass eine zunächst erlaubte Zugriffssituation später unzulässig werden kann, wenn sich entscheidungsrelevante Attribute ändern. Das klassische Reference-Monitor-Konzept erklärt daher zunächst die unmittelbare Kontrolle eines Zugriffs, während die spätere Arbeit zusätzlich betrachtet, wie bestehende Zugriffssituationen nachträglich überwacht werden können.

2.2 Attribute Based Access Control

Attribute-Based Access Control (ABAC) ist ein Zugriffskontrollmodell, bei dem eine Autorisierungsentscheidung nicht allein anhand einer festen Subjektidentität oder einer an der Ressource hinterlegten Access Control List (ACL) getroffen wird. Eine ACL ordnet einzelnen Subjekten oder Gruppen konkrete Berechtigungen für eine Ressource zu. Ebenso beruht die Entscheidung nicht ausschließlich auf einer zugewiesenen Rolle, wie bei Role-Based Access Control (RBAC). Stattdessen werden Attribute ausgewertet, die sich auf das Subjekt, die Ressource, die angeforderte Operation und gegebenenfalls auf die Umgebung beziehen. Das National Institute of Standards and Technology (NIST) beschreibt ABAC als logische Zugriffskontrollmethode, bei der solche Attribute gegen Policies, Regeln oder Beziehungen geprüft werden, um zu entscheiden, ob eine Operation zulässig ist [Hu+14].

Für diese Arbeit ist ABAC relevant, weil dadurch Zugriffsentscheidungen feiner modelliert werden können als bei rein rollenbasierten oder benutzerbasierten Ansätzen. Ein Subjekt kann beispielsweise über Attribute wie Position, Benutzerkennung oder Gruppenzugehörigkeit beschrieben werden. Eine Ressource kann Attribute wie Pfad, Typ oder Schutzstufe besitzen. Zusätzlich können Umwelt- oder Systemattribute, etwa Uhrzeit oder Systemzustand, in die Entscheidung einbezogen werden. Damit entsteht ein flexibles Modell, in dem Policies Bedingungen über mehrere Attributklassen ausdrücken können.

ABAC trennt damit die Beschreibung der Entscheidungsgrundlage von der konkreten technischen Durchsetzung. Die Policy beschreibt, unter welchen Bedingungen ein Zugriff erlaubt oder verweigert werden soll. Die Auswertung dieser Bedingungen und die Durchsetzung der Entscheidung können dagegen von unterschiedlichen Systemkomponenten übernommen werden. Diese Trennung ist für den Prototyp wichtig, weil die Policy-Verarbeitung im Userspace stattfinden kann, während die Zugriffsdurchsetzung später im Kernel betrachtet wird.

2.3 Attribute Stream-Based Access Control

ASBAC erweitert attributbasierte Zugriffskontrolle um fortlaufend veränderliche Entscheidungsgrundlagen und Autorisierungsentscheidungen. Während eine klassische ABAC-Anfrage typischerweise zu einer einzelnen Entscheidung führt, können in ASBAC Attribute, Policies und weitere Entscheidungsgrundlagen als veränderliche Datenströme betrachtet werden. Eine bestehende Autorisierung kann dadurch neu bewertet werden, sobald sich ein entscheidungsrelevanter Wert ändert.

Heutelbeck beschreibt ASBAC im Zusammenhang mit der SAPL als Publish-Subscribe-Modell. Ein Policy Enforcement Point stellt dazu eine Autorisierungsanfrage als Subscription an einen PDP. Der PDP bezieht die für die Entscheidung benötigten Datenströme ein und kann dem PEP aktualisierte Entscheidungen übermitteln, wenn sich Policies, Attribute oder weitere Entscheidungsgrundlagen ändern [Heu21].

Die fortlaufende Neubewertung steht inhaltlich in der Nähe von Usage Control. Unter Usage Control werden Zugriffskontrollmodelle verstanden, die eine Nutzung nicht nur zum Zeitpunkt der initialen Autorisierung betrachten, sondern auch Bedingungen während der laufenden Nutzung einbeziehen. Usage Control erweitert klassische Zugriffskontrolle unter anderem um die Veränderbarkeit von Attributen und die Fortdauer von Entscheidungen während einer Nutzung. Park und Sandhu verankern diese Eigenschaften als Mutabilität und Kontinuität im UCON-Modell, das Autorisierungen, Verpflichtungen und Bedingungen integriert [PS04]. Gouglidis et al. greifen diese Konzepte als *attribute mutability* und *continuity of decision* bei der formalen Untersuchung von UseCON auf [GGM18]. Usage Control dient in dieser Arbeit als verwandtes theoretisches Konzept, ersetzt jedoch nicht die Definition von ASBAC.

Der in dieser Arbeit entwickelte Prototyp übernimmt nicht die vollständige Publish-Subscribe-Architektur von SAPL. Insbesondere existiert kein zentraler PDP, der für jede bestehende Autorisierung einen individuellen Strom aktualisierter Entscheidungen an einen PEP übermittelt. Stattdessen werden Policies und dynamische Attribute im Userspace verarbeitet und über eBPF-Maps als generationsbasierter Zustand bereitgestellt. Neue Dateiöffnungen werden anhand dieses Zustands durch einen eBPF-LSM-basierten Kernelspace-Pfad bewertet. Nach der erfolgreichen Aktivierung einer neuen Policy- oder Attributgeneration stößt der Prototyp ereignisbasiert eine Neubewertung bestehender Dateizugriffe an; zeitabhängige Policies lösen eine solche Neubewertung beim Erreichen einer entscheidungsrelevanten Zeitgrenze aus.

Ein geöffneter FD wird dabei als technische Repräsentation einer fortdauernden Nutzungssituation betrachtet, nicht als ASBAC-Subscription. Der Prototyp registriert beim Öffnen keine fortdauernde Autorisierungsanfrage und verwaltet keinen individuellen Decision Stream. Der Userspace rekonstruiert vielmehr nach einem Aktivierungs- oder Zeitereignis aus den dann sichtbaren Prozess- und File-Descriptor-Informationen eine Zugriffsanfrage und bewertet diese lokal erneut. Der Prototyp realisiert somit keinen allgemeinen ASBAC-Decision-Stream, überträgt aber zentrale ASBAC-Eigenschaften auf einen begrenzten Zugriffskontrollmechanismus für Linux-Dateiöffnungen: veränderliche

Entscheidungsgrundlagen, eine durch deren Änderung ausgelöste Neubewertung und die Reaktion auf eine geänderte Autorisierungsentscheidung.

2.4 Policy-Beschreibungen für ASBAC

Policies beschreiben die Bedingungen, unter denen eine Zugriffsentscheidung getroffen wird. In einem attributbasierten Modell beziehen sich diese Bedingungen auf Attribute von Subjekt, Ressource, Aktion und Umgebung. NIST beschreibt Policies in ABAC als Regeln oder Beziehungen, gegen die Attribute ausgewertet werden [Hu+14]. Für ASBAC bleibt diese Grundidee erhalten, sie wird jedoch um dynamisch veränderliche Attribute erweitert.

Eine Policy-Beschreibung muss dafür ausdrücken können, welche Aktion auf welche Ressource bezogen ist und welche Attributbedingungen erfüllt sein müssen. Im Prototyp kann dies beispielsweise bedeuten, dass das Öffnen einer bestimmten Datei nur erlaubt ist, wenn ein Subjektattribut einen bestimmten Wert besitzt. Die Policy-Datei bleibt dabei eine textuelle Beschreibung im Userspace. Für die Auswertung im Kernel muss diese Beschreibung später in eine kernelgeeignete Repräsentation überführt werden.

Etablierte Zugriffskontrollarchitekturen unterscheiden häufig zwischen logischen Rollen zur Entscheidung und zur Durchsetzung. EXtensible Access Control Markup Language (XACML) beschreibt beispielsweise eine Architektur mit PDP und PEP sowie eine Policy-Sprache zur Beschreibung attributbasierter Entscheidungen [OAS17]. Der PDP wertet die für eine Zugriffsanfrage anwendbaren Policies aus und erzeugt eine Autorisierungsentscheidung. Der PEP schützt die Ressource, bildet eine Entscheidungsanfrage und setzt die erhaltene Entscheidung technisch durch. PDP und PEP bezeichnen damit logische Rollen und müssen nicht notwendigerweise als getrennte Prozesse implementiert sein.

Diese Arbeit übernimmt weder XACML als Policy-Sprache noch dessen vollständiges Architekturmodell. Im Prototyp sind Entscheidungs- und Durchsetzungsfunktion technisch eng gekoppelt. Im Kernelspace erfassen der Einstieg und der Rückgabepfad des eBPF-LSM-Programms eine neue `file_open`-Operation und setzen das Ergebnis über den LSM-Rückgabewert durch; sie erfüllen damit die PEP-Funktion. Die dazwischenliegenden Stufen zur Auswahl und Auswertung statischer und dynamischer Policies sowie zum Combining erfüllen logisch eine integrierte PDP-Funktion.

Entsprechend rekonstruiert die Userspace-Komponente bestehende Zugriffssituationen aus Prozess- und File-Descriptor-Informationen, wertet sie lokal anhand der aktiven Policies und Attribute aus und versucht bei einer Deny-Entscheidung, den betroffenen FD zu schließen. Auch diese Komponente verbindet somit eine Policy-Entscheidungsfunktion mit einer Durchsetzungsfunktion. In dieser Arbeit werden die Bezeichnungen *Kernelspace-PEP* und *Userspace-PEP* verwendet, um die jeweilige Durchsetzungsfunktion hervorzuheben. Ein eigenständiger, von beiden PEPs getrennter PDP ist nicht Bestandteil des Prototyps.

2.5 Userspace und Kernelspace in Linux

Linux trennt zwischen Userspace und Kernelspace. Anwendungen laufen im Userspace und greifen auf Betriebssystemfunktionen über Systemaufrufe oder andere definierte Schnittstellen zu. Der Kernelspace enthält dagegen den Kernel und damit Code, der privilegiert ausgeführt wird und zentrale Ressourcen wie Prozesse, Speicher, Dateien und Geräte verwaltet. Die Kernel-Dokumentation beschreibt unterschiedliche Ausführungskontexte, darunter Code, der einem Prozess im Kernelspace zugeordnet ist, sowie Prozesse, die im Userspace laufen [Rus26a].

Diese Trennung ist für Sicherheitsmechanismen wesentlich. Komplexe Verarbeitung, Dateiparsing und Validierung sind im Userspace leichter zu entwickeln, zu testen und bei Fehlern weniger kritisch für die Stabilität des Gesamtsystems. Code im Kernelspace hat dagegen unmittelbaren Einfluss auf das Betriebssystem. Fehler in Kernelcode können nicht nur eine Anwendung, sondern das gesamte System beeinträchtigen. Daher ist es sinnvoll, umfangreiche Policy-Verarbeitung im Userspace zu belassen und den Kernelspace auf die sicherheitskritische Durchsetzung vorbereiteter Entscheidungen zu beschränken.

Für diese Arbeit ergibt sich daraus eine Architekturgrenze. Der Userspace liest Policies und Attribute ein, validiert sie und übersetzt sie in eine Form, die im Kernel effizient ausgewertet werden kann. Der Kernelspace übernimmt dagegen die Entscheidung an den relevanten Zugriffspunkten. Diese Aufteilung reduziert die Komplexität im Kernel, ohne die Durchsetzung vollständig aus dem sicherheitsrelevanten Ausführungspfad herauszulösen.

2.6 LSM und Linux LSM-Hooks

Linux Security Modules (LSM) stellen ein Framework bereit, über das Sicherheitsmodule an sicherheitsrelevanten Stellen des Kernels Entscheidungen treffen können. Die Kernel-Dokumentation beschreibt LSM als Schnittstelle für Sicherheitsmodule wie SELinux, Smack, AppArmor oder TOMOYO [Ker26p]. Aus technischer Sicht werden dafür Hooks verwendet, die an bestimmten Punkten im Kernel-Ausführungspfad aufgerufen werden.

LSM-Hooks sind für Zugriffskontrolle besonders relevant, weil sie Operationen an Stellen erfassen können, an denen der Kernel ohnehin über die Zulässigkeit einer Operation entscheidet. Wright et al. beschreiben das LSM-Framework als allgemeine Sicherheitsunterstützung im Linux-Kernel, bei der Hooks Zugriff auf Kernelobjekte vermitteln und Rückgabewerte über Erfolg oder Fehler der Operation entscheiden können [Wri+02]. Damit eignen sich LSM-Hooks grundsätzlich als technische Grundlage für einen Reference-Monitor-ähnlichen Durchsetzungspunkt.

Für den Prototyp ist der LSM-Hook `file_open` maßgeblich. Er wird beim Öffnen einer Datei aufgerufen und deckt damit die in dieser Arbeit betrachtete Operation ab. Seine Entscheidung bezieht sich jedoch ausschließlich auf den aktuellen Öffnungsvorgang. Ändern sich Attribute erst danach, wird ein bereits geöffneter Dateizugriff nicht erneut

durch denselben Hook bewertet. Daher ist für solche bestehenden Zugriffssituationen eine ergänzende nachträgliche Überwachung erforderlich.

2.7 eBPF und eBPF-LSM

Der extended Berkeley Packet Filter (eBPF) ist ein Mechanismus, mit dem Programme in einer eingeschränkten Ausführungsumgebung im Linux-Kernel ausgeführt werden können. Die Programme werden vor der Ausführung durch den Kernel geprüft und können an verschiedene Ereignisse oder Hook-Typen gebunden werden. Historisch steht BPF für Berkeley Packet Filter. Die heute im Linux-Kernel verwendete erweiterte Form wird meist als eBPF bezeichnet; die Kernel-Dokumentation verwendet für das moderne Subsystem jedoch häufig weiterhin die Bezeichnung BPF. In dieser Arbeit wird deshalb eBPF verwendet, wenn der moderne Mechanismus für Programme, Maps und Verifier (vgl. Abschnitt 2.9) gemeint ist. Der Begriff BPF wird nur dort aufgegriffen, wo er in offiziellen Bezeichnungen wie `bpf()`-Systemaufruf, BPF-Dateisystem oder in der Kernel-Dokumentation selbst verwendet wird. Die Kernel-Dokumentation beschreibt BPF als im Kernel verwendetes Instruktionsformat und betont, dass BPF-Programme auf definierte Hilfsfunktionen und Kernelobjekte wie Maps beschränkt sind [Ker26k].

eBPF-LSM verbindet eBPF mit dem LSM-Framework. Die Kernel-Dokumentation beschreibt LSM-BPF-Programme als Möglichkeit, LSM-Hooks zur Laufzeit durch privilegierte Benutzer zu instrumentieren, um systemweite Mandatory-Access-Control- oder Audit-Policies mit eBPF umzusetzen [Ker26q]. Damit kann ein eBPF-Programm an einem LSM-Hook ausgeführt werden und dort eine Entscheidung über die angeforderte Operation zurückgeben.

Für das Laden und die typgerechte Anbindung solcher Programme ist außerdem das *BPF Type Format* (BTF) relevant. BTF ist ein Metadatenformat, das unter anderem Typ- und Funktionsinformationen zu Kernel- und eBPF-Strukturen beschreibt [Ker26m]. Ein Loader kann die vom Zielkernel unter `/sys/kernel/btf/vmlinux` bereitgestellten Informationen verwenden, um den vorgesehenen LSM-Hook und die Typen seiner Argumente aufzulösen. BTF ist damit keine Policy- oder Attributquelle, sondern eine technische Voraussetzung für das Laden des in dieser Arbeit eingesetzten eBPF-LSM-Programms.

In den Grundlagen wird eBPF noch nicht als endgültige Lösung begründet, sondern als technischer Mechanismus eingeordnet. Für diese Arbeit ist eBPF interessant, weil es eine Ausführung im Kernel ermöglicht, ohne ein klassisches Kernelmodul zu entwickeln. Gleichzeitig ist eBPF durch Verifier, eingeschränkte Sprachelemente, begrenzten Stack, fehlende allgemeine Heap-Allokation und definierte Hilfsfunktionen deutlich stärker beschränkt als allgemeiner Kernelcode. Diese Einschränkungen müssen in der späteren Konzeption gegen die Vorteile abgewogen werden.

2.8 eBPF-Maps und gepinnte Maps

eBPF-Maps sind Datenstrukturen im Kernel, die von eBPF-Programmen und Userspace-Anwendungen gemeinsam genutzt werden können. Die Kernel-Dokumentation beschreibt Maps als generischen Speicher für unterschiedliche Datentypen, der unter anderem den Datenaustausch zwischen Kernelspace und Userspace ermöglicht [Ker26l; Ker26a]. Über den Systemaufruf `bpf()` können Maps erzeugt sowie Einträge gelesen, aktualisiert oder gelöscht werden [Ker26a].

Für Zugriffskontrolle sind Maps relevant, weil eBPF-Programme nicht beliebige komplexe Userspace-Datenstrukturen direkt verwenden können. Policies und Attribute müssen daher in eine feste, kernelgeeignete Struktur gebracht werden. Hash-Maps oder Arrays können beispielsweise verwendet werden, um Policy-Einträge, Attributwerte, Generationen oder Statusinformationen bereitzustellen. Der Kernel kann diese Werte während einer Zugriffskontrolle auslesen, während der Userspace sie bei Policy- oder Attributänderungen aktualisiert.

Gepinnte Maps erweitern diese Idee um einen stabilen Zugriffspfad im BPF-Dateisystem. Pinning erlaubt es, eBPF-Objekte wie Maps über einen Pfad zugänglich zu machen, sodass andere Prozesse sie später wieder öffnen können [eBP26]. Dies ist für den Prototyp wichtig, weil nicht nur der Loader, sondern auch ein Userspace-PEP oder Administrationswerkzeug den aktuellen Zustand lesen können muss. Gepinnte Maps bilden damit eine zentrale Schnittstelle zwischen den getrennten Userspace-Komponenten und dem Kernelzustand.

2.9 eBPF-Verifier, Stack und Programmkomplexität

Bevor ein eBPF-Programm im Kernel ausgeführt werden darf, prüft der eBPF-Verifier dessen Sicherheit. Die Kernel-Dokumentation beschreibt die Prüfung als mehrstufigen Prozess, bei dem unter anderem Kontrollfluss, Registerzustände, Stackzugriffe und Speicherzugriffe analysiert werden [Ker26o]. Ein Programm wird nur geladen, wenn der Verifier zeigen kann, dass es die Sicherheitsregeln erfüllt.

Diese Prüfung hat direkte Auswirkungen auf die Gestaltung von eBPF-Programmen. Zugriffe auf Map-Werte, Stackbereiche oder Kontextdaten müssen für den Verifier nachvollziehbar sein. Der Verifier verfolgt unter anderem, ob Register initialisiert sind, welche Zeigerarten verwendet werden und ob Speicherzugriffe innerhalb bekannter Grenzen liegen. Unklare oder zu komplexe Zugriffsmuster können dazu führen, dass ein Programm abgelehnt wird, auch wenn die fachliche Logik korrekt erscheint. Darüber hinaus muss der Kontrollfluss eines eBPF-Programms für den Verifier analysierbar bleiben. Schleifen dürfen nicht unbeschränkt sein, Rekursion ist nicht als allgemeines Programmiermittel verfügbar, und Speicherzugriffe müssen durch bekannte Grenzen abgesichert sein. Auch der Zugriff auf Kernelinformationen erfolgt nicht beliebig, sondern über den Hook-Kontext, Maps oder für den jeweiligen Programmtyp zugelassene Hilfsfunktionen [Ker26o].

Zusätzlich bestehen praktische Begrenzungen bei Stack und Programmkomplexität. Die

BPF-Dokumentation nennt für Programme ein Stacklimit von 512 Byte und beschreibt interne Grenzen des Verifiers, darunter die Anzahl der während der Analyse betrachteten Instruktionen und Zustände [Ker26k]. Die offizielle Aya-Dokumentation beschreibt eBPF als eingeschränkte Laufzeitumgebung und weist darauf hin, dass eBPF-Programme nur 512 Byte Stack besitzen; bei Programmen, die Tail Calls verwenden, kann der verfügbare Stack auf 256 Byte begrenzt sein [Aya26a]. Der Stack eignet sich damit nur für kleine lokale Hilfsdaten, etwa Schlüssel für den Zugriff auf Map-Einträge oder wenige temporäre Werte. Größere lokale Puffer, dynamisch wachsende Listen oder Zeichenketten können dort nicht sinnvoll abgelegt werden. Ein allgemeiner Heap, wie er in Userspace-Programmen für dynamische Datenstrukturen wie Listen, Strings oder Objekte genutzt wird, steht eBPF-Programmen nicht zur Verfügung. Veränderliche oder größere Zustände müssen daher typischerweise in eBPF-Maps ausgelagert werden. Für Rust-basierte eBPF-Programme ergeben sich daraus weitere Einschränkungen. Die Standardbibliothek kann im eBPF-Programm nicht verwendet werden; stattdessen steht nur `core` zur Verfügung. Funktionen und Traits, die auf `core::fmt` beruhen, etwa `Display` oder `Debug`, sind nicht nutzbar. Da kein Heap verfügbar ist, können auch `alloc` und darauf aufbauende Collections nicht eingesetzt werden. Außerdem darf ein eBPF-Programm nicht auf Panics angewiesen sein, weil die eBPF-Laufzeit weder Stack-Unwinding noch eine Abort-Instruktion unterstützt. Ein eBPF-Programm besitzt zudem keine gewöhnliche `main`-Funktion, sondern wird über einen programmspezifischen Einstiegspunkt an einen Hook gebunden. Da eBPF-Programme häufig direkt Kernel-Speicher oder Hook-Kontextdaten lesen, ist außerdem ein relevanter Teil des Codes als `unsafe` zu behandeln [Aya26a]. Für die Arbeit bedeutet dies, dass komplexe Policy-Logik möglichst im Userspace vorbereitet werden sollte, während das eBPF-Programm nur eine kompakte, vorher strukturierte Repräsentation auswertet.

2.10 Aya als Rust-Framework für eBPF

Aya ist ein Rust-Framework zur Entwicklung von eBPF-Anwendungen. Die offizielle Aya-Dokumentation beschreibt Aya als Möglichkeit, eBPF-Programme mit Rust zu entwickeln und dabei Cargo, Rust-Strukturen sowie gemeinsame Codeanteile zwischen Userspace und eBPF-Programm zu nutzen [Aya26a]. Neben Rust-basierten Frameworks existieren etablierte eBPF-Werkzeuge mit C- sowie Hochsprachen-Schnittstellen. libbpf ist eine C-Bibliothek für die Entwicklung und das Laden von BPF- beziehungsweise eBPF-Programmen, während BCC (BPF Compiler Collection) ein Toolkit für BPF-basierte Analyse-, Tracing- und Monitoring-Werkzeuge ist [lib26; IOV26]. Aya stellt dazu eine Rust-basierte Alternative dar.

Für den Prototyp ist Aya vor allem deshalb relevant, weil Userspace-Komponenten und eBPF-Programm in derselben Programmiersprache entwickelt werden können. Gemeinsame Datentypen können in einer gemeinsamen Bibliothek definiert werden, wodurch sich die Gefahr verringert, dass Userspace und Kernelspace unterschiedliche Annahmen über die Struktur von Map-Schlüsseln oder Map-Werten treffen. Gerade bei Policies und Attributen ist diese Konsistenz wichtig, weil dieselben binären Strukturen auf beiden Seiten der Map-Schnittstelle interpretiert werden.

Gleichzeitig hebt Aya die Einschränkungen von eBPF nicht auf. Auch mit Rust gelten die Vorgaben des eBPF-Verifiers, die Stackbegrenzung, der Verzicht auf Heap-Allokation im eBPF-Programm und die Beschränkung auf unterstützte Hilfsfunktionen. Aya ist daher als Entwicklungswerkzeug zu verstehen, nicht als fachliches Zugriffskontrollmodell. Die Entscheidung für Aya betrifft die spätere Implementierung, während die fachlichen Anforderungen weiterhin aus ASBAC und der Kernel-Durchsetzung abgeleitet werden.

2.11 Dynamische Attribute und Policy-Generationen

Dynamische Attribute sind Attributwerte, die sich zur Laufzeit ändern können. Im Unterschied zu statischen Eigenschaften, etwa einem festen Dateipfad, können dynamische Attribute zeitabhängig oder durch externe Ereignisse beeinflusst sein. Beispiele sind Uhrzeit, Systemzustand oder vom Nutzer frei definierte Subjektattribute. Die Idee veränderlicher Attribute knüpft an Usage-Control-Modelle an, in denen Attributmutabilität als ausdrückliches Merkmal betrachtet wird [GGM18, S. 52].

Für den Prototyp bedeutet dies, dass Attribute nicht nur beim Start einmalig geladen werden dürfen. Änderungen müssen vom Userspace erkannt, validiert und in die kernelgeeignete Repräsentation übertragen werden. eBPF-Programme können Map-Einträge während ihrer Ausführung lesen, während Userspace-Anwendungen dieselben Maps aktualisieren können [Ker26a]. Der Prototyp nutzt diese Eigenschaft, um aktuelle Attributwerte für Zugriffsentscheidungen bereitzustellen. Die Attributnamen und Werte müssen dabei im Userspace so verarbeitet werden, dass das eBPF-Programm mit festen Schlüssel- und Wertstrukturen arbeiten kann.

Policy-Generationen beschreiben zusammengehörige gültige Stände von Policies. Dieser Begriff ist für den Prototyp wichtig, weil Policy-Dateien im Userspace geparst und validiert werden, bevor sie für Zugriffsentscheidungen verwendet werden. Eine neue Generation sollte erst dann aktiv werden, wenn alle dazugehörigen Map-Einträge vollständig erzeugt und konsistent bereitgestellt wurden. Dadurch wird vermieden, dass der Kernel während einer Aktualisierung auf einen unvollständigen oder widersprüchlichen Policy-Zustand zugreift.

2.12 Überwachung bestehender Zugriffe und File Descriptors

Ein Dateizugriff in Linux wird häufig über einen FD repräsentiert. Der Systemaufruf `open()` liefert bei Erfolg einen nichtnegativen FD zurück, der in nachfolgenden Systemaufrufen wie `read()`, `write()` oder `lseek()` als Referenz auf die geöffnete Datei verwendet wird [Ker26g]. Damit entsteht nach dem Öffnen eine bestehende Zugriffssituation, die unabhängig vom ursprünglichen Pfad weiter über den FD genutzt werden kann.

Für eine Zugriffskontrolle über `file_open` ist das wichtig, weil der LSM-Hook beim Öffnen der Datei greift. Wenn sich danach entscheidungsrelevante Attribute ändern, wird derselbe bereits geöffnete FD nicht automatisch erneut durch `file_open` bewertet. Eine

ASBAC-orientierte Zugriffskontrolle muss deshalb zusätzlich betrachten, wie bestehende FDs überwacht und bei geänderter Entscheidungsgrundlage behandelt werden können.

Linux stellt über `/proc/<pid>/fd` Informationen über die geöffneten FDs eines Prozesses bereit. Die man-pages beschreiben die Einträge in diesem Verzeichnis als symbolische Links auf die geöffneten Dateien oder andere referenzierte Objekte; der Zugriff darauf unterliegt eigenen Berechtigungsprüfungen [Ker26h]. Für den Prototyp kann ein Userspace-PEP diese Informationen nutzen, um zu erkennen, welcher Prozess über welchen FD auf welche Datei zugreift. Wird eine bestehende Zugriffssituation durch neue Attribute oder Policies unzulässig, kann der Userspace-PEP gezielt den betroffenen Zugriff behandeln, ohne notwendigerweise alle Zugriffe desselben Prozesses zu beenden.

Eine solche nachträgliche Behandlung ist jedoch nicht atomar. Zwischen dem Ermitteln eines File Descriptors, der Policy-Auswertung und dem späteren Entzugsversuch kann sich der Zustand des Prozesses verändern. Eine derartige Zustandsänderung zwischen Prüfung und Verwendung wird als Time-of-check-to-time-of-use-Race-Condition (TOCTTOU) bezeichnet [MIT26]. Linux kann eine geschlossene File-Descriptor-Nummer erneut vergeben; bei mehrthreadigen Prozessen kann zudem eine bereits laufende Ein-/Ausgabe (E/A) -Operation auch nach dem Schließen des Descriptors noch abgeschlossen werden [Ker26b].

Weitere Referenzen auf dieselbe geöffnete Dateibeschreibung können unabhängig vom betrachteten Descriptor bestehen. `dup()` erzeugt einen zusätzlichen FD für dieselbe geöffnete Dateibeschreibung, und ein durch `fork()` erzeugter Kindprozess übernimmt entsprechende Deskriptorreferenzen [Ker26c; Ker26e]. Das Schließen eines einzelnen File Descriptors beseitigt diese weiteren Referenzen nicht. Ebenso bleibt eine mit `mmap()` erzeugte Speicherabbildung nach dem Schließen des zugehörigen FDs bestehen [Ker26f]. Der für den Prototyp verwendete `ptrace`-Mechanismus ermöglicht zwar die Steuerung eines Zielprozesses, unterliegt jedoch Berechtigungsprüfungen und arbeitet auf der Ebene einzelner Threads [Ker26j]. Ein FD-basierter Entzug aus dem Userspace kann deshalb nur als nachträglicher, bestmöglicher Entzugsversuch verstanden werden.

2.13 NixOS

NixOS ist eine Linux-Distribution, deren Systemzustand deklarativ beschrieben wird. Kernel, Systemdienste, Pakete und weitere Einstellungen werden aus Nix-Konfigurationen erzeugt [Nix26]. Dadurch können System- und Entwicklungsumgebung gemeinsam mit dem Quellcode dokumentiert werden.

Für die Evaluation wird NixOS als Gastbetriebssystem in einer QEMU/KVM-VM eingesetzt.

2.14 Verwandte Arbeiten und Abgrenzung des eigenen Ansatzes

Fortdauernde Autorisierung und veränderliche Entscheidungsgrundlagen sind bereits durch Usage-Control-Modelle und ASBAC beschrieben. Das UCON-Modell berücksichtigt die Veränderbarkeit von Attributen und die Prüfung von Berechtigungen während einer Nutzung [PS04]. SAPL realisiert für ASBAC eine Publish-Subscribe-Architektur, in der der PDP veränderte Autorisierungsentscheidungen an den PEP übermittelt [Heu21; Heu]. Die fortlaufende Neubewertung einer Autorisierung ist daher keine neue Modellidee dieser Arbeit.

Auch die Durchsetzung dynamischer Dateinutzungskontrolle im Betriebssystemkernel ist Gegenstand bestehender Arbeiten. Teigão et al. implementieren einen UCON-orientierten Prototyp im OpenBSD-4.1-Kernel. Ein Parser überführt textuelle Regeln in eine interne Repräsentation; die laufenden Prüfungen erfolgen insbesondere bei Lese- und Schreiboperationen. Bei veränderten Bedingungen können weitere Operationen mit einem Berechtigungsfehler verweigert werden [TMS11, Abschnitte 7.1–7.3, S. 1349–1350]. Diese Vorarbeit zeigt bereits die grundsätzliche Machbarkeit dynamischer Dateikontrolle im Kernel, verwendet jedoch Kerneländerungen statt eBPF-LSM und prüft die laufende Nutzung an den jeweiligen Operationen.

Etablierte Linux-Sicherheitsmodule berücksichtigen ebenfalls Zugriffe nach dem Öffnen einer Datei. Im untersuchten Linux-Kernel v6.12 vergleicht SELinux in `selinux_file_permission` unter anderem die beim Öffnen gespeicherte Policysequenz mit dem aktuellen Stand und veranlasst gegebenenfalls eine erneute Berechtigungsprüfung [Ker24b]. Eine bereits geöffnete Datei ist damit nicht grundsätzlich von späteren Zugriffskontrollen ausgenommen. AppArmor setzt dagegen anwendungsbezogene Profile mit pfadbasierten Dateiregeln durch [Ker25]. Seine Profilverwaltungsdokumentation weist darauf hin, dass beim Ersetzen eines Profils bereits geöffnete Ressourcen weiterhin zugänglich bleiben können [App]. Der Kernelcode enthält zugleich eine Revalidierung bestehender Dateiobjekte, etwa wenn der aktuelle Sicherheitskontext nicht durch die zwischengespeicherten Berechtigungen abgedeckt ist [Ker24a]. Profilaktualisierung, erneute Operationsprüfung und aktives Schließen eines Deskriptors sind deshalb unterschiedliche Mechanismen; aus einer Profilaktualisierung folgt kein allgemeiner selektiver FD-Entzug.

KubeArmor und Tetragon zeigen die praktische Umsetzung deklarativer Sicherheitsregeln mit Linux-Kernelmechanismen und eBPF. Der BPF-LSM-Enforcer von KubeArmor übersetzt Policies in Map-Einträge, die an LSM-Hooks für unmittelbare Durchsetzungsentscheidungen ausgewertet werden [Kub]. Die Verwaltung von Policies im Userspace und deren Auswertung anhand von BPF-Maps im Kernel sind somit bereits etablierte technische Ansätze. Tetragon dokumentiert die Durchsetzung durch das Überschreiben geeigneter Funktionsrückgabewerte und durch Prozesssignale; seine Dateizugriffskontrolle berücksichtigt insbesondere `security_file_permission` für Lese- und Schreibzugriffe [Teta; Tetb]. Die Verweigerung einer weiteren Operation und die Beendigung eines Prozesses unterscheiden sich in ihrer Wirkung vom selektiven Schließen eines einzelnen FD.

Die vorliegende Arbeit untersucht vor diesem Hintergrund die Übertragung ausgewählter ASBAC-Eigenschaften auf einen begrenzten eBPF-LSM-Prototyp für Linux-Dateiöffnungen. Ihr Beitrag umfasst die Bereitstellung dynamischer Entscheidungsgrundlagen, die Abstimmung von Kernel- und Userspace-Auswertung sowie die ereignisabhängige Nachbewertung bestehender File Descriptors mit einem selektiven Schließversuch. Die konkrete Verbindung dieser Mechanismen wird in Kapitel 4 konzipiert und in Kapitel 6 evaluiert. Dabei wird insbesondere untersucht, welche Einschränkungen aus der Rekonstruktion bestehender Zugriffe und dem nachträglichen Eingriff in den Zielprozess entstehen. Die verbleibende Fragestellung betrifft damit die technische Integration und ihre Grenzen, nicht die erstmalige Einführung fortdauernder Autorisierung oder dynamischer Kernelzugriffskontrolle. Der Prototyp übernimmt keine vollständige ASBAC-Subscription-Architektur und gewährleistet keinen vollständigen oder atomaren Entzug sämtlicher Nutzungsformen.

3 Anforderungsanalyse

In diesem Kapitel werden die Anforderungen an den zu entwickelnden Prototypen beschrieben. Die Anforderungen ergeben sich aus der Zielsetzung, zu untersuchen, wie ASBAC im Linux-Kernel prototypisch umgesetzt werden kann. Sie beschreiben das Verhalten, das der Prototyp im Rahmen der Arbeit zeigen soll.

Für den Prototyp wird ein vertrauenswürdig administriertes Linux-Zielsystem vorausgesetzt. Der Administrator und der Kernel gehören zur Vertrauensbasis. Der Administrator darf Policies, Attributquellen, eBPF-Maps und Programme verändern sowie die Runtime beenden; Schutz gegen solche administrativen Eingriffe ist nicht Gegenstand der Anforderungen. Die Integrität der Policy- und Attributquellen sowie der bereitgestellten Zeitinformationen wird vorausgesetzt und ist durch die Administration sicherzustellen. Vorgesehen ist der Betrieb einer einzelnen Runtime; eine Absicherung gegen konkurrierende administrative Schreibzugriffe wird nicht gefordert.

Die zusätzliche Zugriffskontrolle des Prototyps wird nur nach erfolgreicher Initialisierung und solange das Hauptprogramm läuft bereitgestellt. Eine Fortsetzung dieses Schutzes nach Beendigung oder Ausfall der Runtime ist nicht Bestandteil des Prototypumfangs.

3.1 Funktionale Anforderungen

Funktionale Anforderungen beschreiben, welche fachlichen Funktionen der Prototyp bereitstellen muss. Sie sind im Regelfall direkt testbar, da geprüft werden kann, ob eine Funktion vorhanden ist und das erwartete Verhalten zeigt. Im Kontext dieser Arbeit betreffen sie insbesondere das Einlesen und Verarbeiten von Policies, die Zugriffskontrolle über LSM-Hooks sowie die nachträgliche Überwachung bestehender Zugriffe.

FA-01 Policy-Einlesen: Der Prototyp muss ASBAC-Policies aus einer definierten Quelle im Userspace einlesen können.

Akzeptanzkriterium: Eine im Policy-Verzeichnis abgelegte Policy wird erkannt und für die weitere Verarbeitung bereitgestellt.

FA-02 Policy-Verwaltung: Der Prototyp muss über das vorgesehene Veröffentlichungsverfahren bereitgestellte Policyänderungen sowie das Entfernen aktiver Policy-Dateien erkennen und in die Zugriffskontrolle übernehmen können.

Akzeptanzkriterium: Wird eine neue oder geänderte Policy über das vorgesehene Verfahren veröffentlicht oder eine aktive Policy-Datei entfernt, wird diese Änderung vom System verarbeitet. Direktes Überschreiben aktiver Dateien während des Betriebs ist kein unterstützter Aktualisierungsweg. Vor dem Start kann der anfängliche Dateibestand regulär vorbereitet werden.

FA-03 Zugriffskontrolle bei Dateiöffnungen: Der Prototyp muss Dateiöffnungen über

den LSM-Hook `file_open` abfangen können.

Akzeptanzkriterium: Das Öffnen einer durch eine Policy erfassten Datei wird durch den LSM-Hook `file_open` kontrolliert.

FA-04 Entscheidungsfindung anhand von Policies: Der Prototyp muss Zugriffsentscheidungen auf Grundlage der geladenen Policies treffen können.

Akzeptanzkriterium: Ein Zugriff wird abhängig von einer Policy erlaubt oder verweigert.

FA-05 Unterstützung dynamischer Attribute: Der Prototyp muss dynamische Attribute, beispielsweise Zeitinformationen oder externe Zustandsinformationen, in Zugriffsentscheidungen einbeziehen können.

Akzeptanzkriterium: Eine Änderung eines dynamischen Attributs kann zu einer veränderten Zugriffsentscheidung führen. Für dateibasierte Attribute werden Änderungen über das vorgesehene Veröffentlichungsverfahren sowie das Entfernen aktiver Attributdateien verarbeitet; direktes Überschreiben aktiver Dateien während des Betriebs ist kein unterstützter Aktualisierungsweg.

FA-06 Überwachung und Enforcement bestehender Dateizugriffe: Der Prototyp muss bereits bestehende Dateizugriffe überwachen und versuchen, sie durch Schließen des betroffenen FDs zu beenden, wenn sie durch entscheidungsrelevante Policy- oder Attributänderungen unzulässig werden. Als bestehender Zugriff gilt im Rahmen des Prototyps ein geöffneter FD eines laufenden Prozesses auf eine durch eine Policy erfasste Datei. Der Userspace-PEP ermittelt den zugehörigen Prozess und FD, bewertet den Zugriff anhand der aktuellen Policy- und Attributlage und führt bei einer erkannten Verletzung einen gezielten Schließversuch im Zielprozess aus. Im Rahmen des Prototyps wird dieser Entzug als Best-Effort-Maßnahme verstanden: Der Schließversuch ist erforderlich, sein Erfolg kann insbesondere bei einem nicht erreichbaren Zielprozess jedoch nicht garantiert werden. Ein fehlgeschlagener Versuch muss protokolliert werden und darf die Behandlung weiterer festgestellter Verletzungen nicht verhindern. Eine reine Ausgabe der Verletzung ohne Schließversuch genügt der Anforderung nicht.

Akzeptanzkriterium: Öffnet ein Prozess eine Datei zunächst zulässig und wird dieser Zugriff anschließend durch eine Policy- oder Attributänderung unzulässig, führt der Userspace-PEP bei der Nachbewertung einen gezielten Schließversuch für den betroffenen FD aus. In einem kontrollierten Erfolgsfall wird der Deskriptor tatsächlich geschlossen; andere FDs desselben Prozesses bleiben geöffnet, sofern sie nicht selbst eine Policy verletzen. In einem kontrollierten Fehlerfall mit nicht erreichbarem Zielprozess wird der fehlgeschlagene Versuch protokolliert und die Behandlung weiterer festgestellter Verletzungen fortgesetzt. Ein fehlgeschlagener Schließversuch wird dabei nicht als erfolgreicher Entzug gewertet.

FA-07 Administrationsschnittstelle: Der Prototyp soll eine einfache Administrationsschnittstelle bereitstellen, über die zentrale Informationen zum Zustand des Prototyps lesend eingesehen werden können.

Akzeptanzkriterium: Ein Administrator kann zentrale Informationen zum Zustand des Prototyps abrufen, ohne über die Administrationsschnittstelle Policies, Attribute oder Kernelzustand zu verändern.

FA-08 Gültige Policy-Generationen: Der Prototyp muss sicherstellen, dass nur erfolgreich erzeugte und validierte Generationen von Policies für Zugriffsentscheidungen verwendet werden. Fehlerhafte, unvollständige oder nicht erfolgreich verarbeitete Policy-Generationen dürfen nicht als aktive Entscheidungsgrundlage gelten.

Akzeptanzkriterium: Eine neue Policy-Generation wird erst dann aktiviert, wenn sie vollständig verarbeitet und erfolgreich validiert wurde. Andernfalls bleibt die zuletzt gültige Policy-Generation aktiv.

FA-09 Validierung von Policies: Der Prototyp muss eingelesene Policies vor ihrer Übersetzung in die Kernelrepräsentation syntaktisch und semantisch validieren. Policies, die nicht dem unterstützten Policyformat entsprechen oder nicht in die vorgesehene Kernelrepräsentation überführt werden können, dürfen nicht für Zugriffsentscheidungen aktiviert werden.

Akzeptanzkriterium: Eine syntaktisch oder semantisch ungültige Policy wird mit einer nachvollziehbaren Fehlermeldung abgelehnt und nicht in die aktiven Policy-Maps übernommen. Die bislang aktive, gültige Policy-Generation bleibt unverändert.

FA-10 Beliebige Attributnamen: In Policies muss es für den Nutzer möglich sein, Attributnamen frei zu vergeben, ohne dass diese vorher im Prototyp fest definiert sein müssen. Zulässige Attributnamen dürfen nicht leer sein und bestehen aus American Standard Code for Information Interchange (ASCII)-Buchstaben (`A-Z`, `a-z`), Ziffern (`0-9`), Unterstrich (`_`) und Bindestrich (`-`).

Akzeptanzkriterium: Eine Policy mit einem gültigen, zuvor nicht bekannten Attributnamen, beispielsweise `subject.position == "engineer"`, wird erfolgreich verarbeitet. Eine Policy mit einem Attributnamen außerhalb des zulässigen Zeichensatzes wird abgelehnt.

3.2 Operative Anforderungen

Operative Anforderungen beschreiben, wie gut der Prototyp seine Funktionen im Betrieb erfüllen soll. Sie beziehen sich auf Eigenschaften, die für Nutzung, Betrieb und Evaluation sichtbar sind, beispielsweise Stabilität, Beobachtbarkeit, Nachvollziehbarkeit, Performance und Reproduzierbarkeit.

OA-01 Stabilität: Der Prototyp darf die Stabilität des Linux-Kernels nicht gefährden.

Akzeptanzkriterium: Fehlerhafte oder ungültige Eingaben aus dem Userspace führen nicht zu einem Kernel-Absturz.

OA-02 Beobachtbarkeit und Nachvollziehbarkeit: Der Prototyp soll zentrale Zustände und Entscheidungen so sichtbar machen, dass Betrieb und Evaluation nachvollziehen können, welche Policies, Attribute oder Zustände eine Zugriffsentscheidung beeinflusst haben.

Akzeptanzkriterium: Für zentrale Zugriffsentscheidungen ist erkennbar, welche Policy oder welcher Zustand die Entscheidung beeinflusst hat.

OA-03 Performance: Der Prototyp muss Dateiöffnungen mit begrenztem zusätzlichem Laufzeitaufwand kontrollieren und Änderungen der Entscheidungsgrundlagen zeitnah durchsetzen.

Akzeptanzkriterium: Unter den dokumentierten Bedingungen auf dem Referenzsystem dürfen Median und p95 der Dateiöffnungsdauer mit aktivem Prototyp sowohl ohne Policy als auch mit passender Permit-Policy höchstens 20 % über der Baseline ohne aktiven Prototyp liegen. Das p95 der Reaktionszeit nach Policy- und Attributänderungen sowie der Zeit von einer Policyänderung bis zum erfolgreichen Entzug bestehender File Descriptors darf jeweils 50 ms nicht überschreiten. Alle im Entzugsszenario untersuchten unzulässig gewordenen File Descriptors müssen erfolgreich geschlossen werden. Messbeginn, Messende, Messintervalle, Wiederholungszahlen und Testlast werden dokumentiert.

OA-04 Reproduzierbarkeit: Die Evaluation des Prototyps soll unter dokumentierten Bedingungen wiederholbar sein.

Akzeptanzkriterium: Testumgebung, Policies und Testszenarien werden so beschrieben, dass die Ergebnisse nachvollzogen und wiederholt werden können.

3.3 Entwicklungsbezogene Anforderungen

Entwicklungsbezogene Anforderungen beschreiben Eigenschaften, die vor allem für die Entwicklung und spätere Erweiterung des Prototyps relevant sind. Sie sind für Nutzer nicht immer unmittelbar sichtbar, beeinflussen aber, wie verständlich, änderbar und erweiterbar das System bleibt.

EA-01 Wartbarkeit: Die Implementierung soll modular aufgebaut sein, sodass Userspace-Komponenten, Kernel-Komponenten und Kommunikationsschnittstelle getrennt betrachtet werden können.

EA-02 Erweiterbarkeit: Der Prototyp soll so entworfen werden, dass weitere Attribute,

Policies oder LSM-Hooks ergänzt werden können.

EA-03 Begrenzter Kernel-Anteil: Komplexe Policy-Logik soll möglichst im Userspace verbleiben, um die Komplexität im Kernel zu reduzieren.

4 Konzeption

Dieses Kapitel entwickelt die Konzeption des Prototyps auf Grundlage der zuvor formulierten Anforderungen sowie der technischen Randbedingungen von eBPF. Zunächst werden die praktischen Einschränkungen der Ausführung im Kernelspace eingeordnet. Darauf aufbauend werden die Architektur mit ihren Zuständigkeiten, die Datenflüsse und das fachliche Datenmodell beschrieben. Abschließend werden die wesentlichen Entwurfsentscheidungen, ihre Alternativen und ihre jeweiligen Konsequenzen begründet.

4.1 Entwurfsfolgen der eBPF-Einschränkungen

Die in Abschnitt 2.9 beschriebenen Grenzen erfordern für den Prototyp feste Datenlayouts, begrenzte Schleifen und eine überschaubare Auswertungslogik. Daraus folgen drei Entwurfsentscheidungen: Textuelle Policies werden im Userspace vorbereitet, der Kernel erhält kompakte Regeln und Attribute über Maps, und die Auswertung wird auf mehrere Tail-Call-Stufen verteilt. Die jeweiligen Begründungen folgen in den Entwurfsentscheidungen dieses Kapitels.

4.2 Architektur

Die Architektur des Prototyps ist in zwei zentrale Bereiche unterteilt: den Userspace und den Kernelspace. Die Gründe für diese Aufteilung werden in Abschnitt 4.5.1 erläutert. Die im Folgenden verwendeten Bezeichnungen Kernelspace-PEP und Userspace-PEP heben die jeweilige Durchsetzungsfunktion hervor. Wie in den Grundlagen erläutert, enthalten beide Pfade zugleich eine integrierte PDP-Funktion, weil sie die anwendbaren Policies lokal auswerten und daraus eine Entscheidung ableiten. Ein separater PDP-Prozess und ein Nachrichtenaustausch von Entscheidungsanfragen und Entscheidungen zwischen PDP und PEP sind nicht Bestandteil der Architektur.

Die Zuständigkeiten verteilen sich auf folgende Komponenten:

Hauptprogramm: Initialisiert Maps und eBPF-Programme, hängt den LSM-Hook an und steuert die Userspace-Komponenten.

Policyverwaltung: Liest und validiert Policyquellen, übersetzt sie in Regeln und aktiviert neue Policy-Generationen.

Attributloader: Stellt die aktuelle Zeit sowie externe System-, Subject- und Ressourcenattribute bereit.

Kernelspace-PEP: Bewertet neue Dateiöffnungen am Hook `file_open` und setzt die Entscheidung über dessen Rückgabewert durch.

Userspace-PEP: Rekonstruiert nach entscheidungsrelevanten Ereignissen bestehende

Dateizugriffe und versucht, unzulässige FDs gezielt zu schließen.

Administrationstool: Zeigt geladene Policies, Attribute und Generationen ausschließlich lesend an.

Gepinnte eBPF-Maps verbinden die Komponenten durch gemeinsame Entscheidungsdaten. Entscheidungen verbleiben im jeweiligen Durchsetzungspfad; der Userspace-PEP erhält keine fertigen Entscheidungen vom Kernel, sondern bewertet bestehende Zugriffe selbst. Die Zuordnung dieser Aufgaben zu Rust-Crates behandelt Abschnitt 5.1.

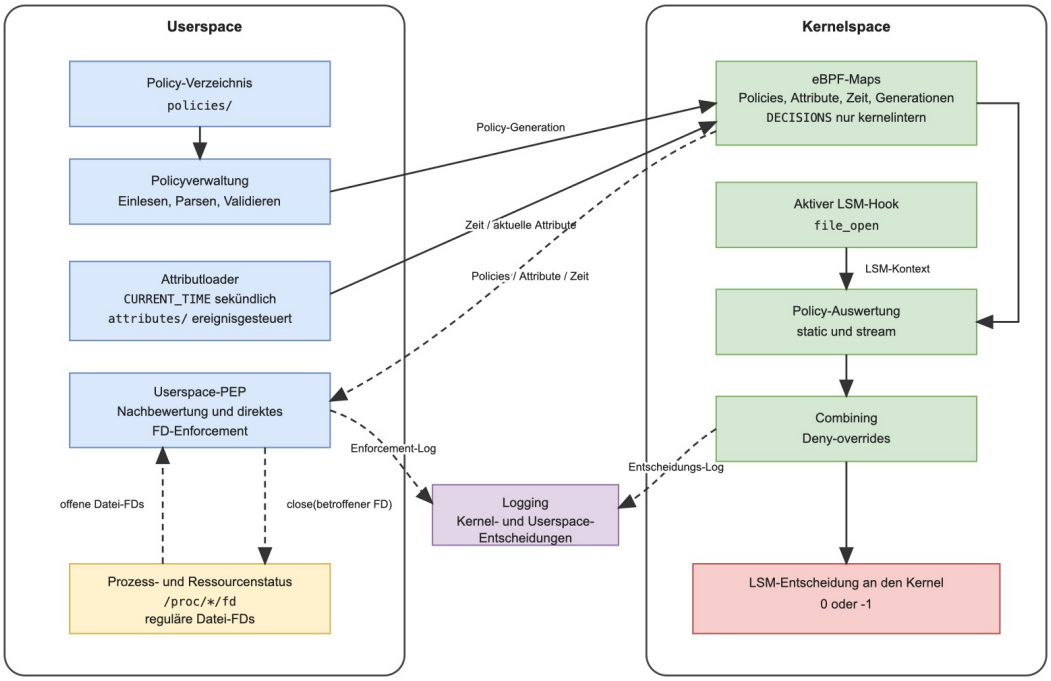


Abbildung 1: Konzeptionelle Architektur des Prototyps

4.3 Datenfluss

Die vier Abläufe verbinden die Eingabedaten mit der unmittelbaren und nachträglichen Zugriffskontrolle.

4.3.1 Policy-Datenfluss

Die Policyverwaltung liest den veröffentlichten Dateibestand, validiert ihn und erzeugt kernelgeeignete Regeln. Erst nach erfolgreicher Vorbereitung wird die neue Policy-Generation aktiviert; Updatefehler vor der Aktivierung lassen die bisherige Generation bestehen. Das Generationenmodell und seine Grenzen erläutert Abschnitt 4.5.5.

4.3.2 Datenfluss dynamischer Attribute

Die aktuelle Zeit wird regelmäßig bereitgestellt, externe Attribute nach Veröffentlichung oder Entfernung aktiver Quelldateien. Das Veröffentlichungsprinzip ist die atomare Ersetzung einer vollständig vorbereiteten Datei; es bildet keine gemeinsame Transaktion mehrerer Dateien oder einen atomaren Verzeichnissnapshot. Das konkrete Verfahren beschreibt Abschnitt 5.3.4. Nach erfolgreicher Verarbeitung wird der neue Attributbestand unabhängig von den Policies aktiviert.

4.3.3 Zugriffsentscheidungen im Kernel

Der Hook `file_open` liefert den Kontext einer neuen Dateiöffnung. Der Kernelspace-PEP bildet daraus eine Anfrage, prüft die geladenen Regeln und kombiniert deren Ergebnisse. Bei einer Ablehnung unterbindet der Kernel die Öffnung. Die gewählte Combining-Semantik erläutert Abschnitt 4.5.8.

4.3.4 Nachträgliche Kontrolle bestehender Zugriffe

Eine erfolgreiche entscheidungsrelevante Aktivierung oder eine relevante Zeitgrenze löst die Nachbewertung bestehender FDs aus. Der Userspace-PEP rekonstruiert diese Zugriffe aus Betriebssysteminformationen, wertet die gepinnten Entscheidungsdaten aus und versucht, betroffene FDs zu schließen. Begründung und Best-Effort-Grenzen stehen in Abschnitt 4.5.10.

4.4 Fachliches Datenmodell

Das Datenmodell unterscheidet Eingaben, vorbereitete Entscheidungsgrundlagen und eine konkrete Auswertung:

Policyquelle: Eine menschenlesbare Datei beschreibt Entscheidung, Zugriffstyp, Filter und Bedingungen.

Vorbereitete Regeln: Validierte Policyquellen werden in statische Regeln und Streampolicies übersetzt. Eine Policy-Generation umfasst die gemeinsam aktivierte Regelmenge.

Attribute: Zeitinformationen und externe Werte beschreiben den veränderlichen Kontext. Externe Attribute gehören zum System, zu einem Subjekt oder zu einer Ressource und besitzen eine eigene Generation.

Zugriffsanfrage: Subjekt, Kommando und Dateiobjekt beschreiben eine neue Öffnung oder einen rekonstruierten bestehenden Zugriff. Für den nachträglichen Entzug kommen Prozess-ID und FD-Nummer hinzu.

Entscheidungszustand: Treffer der Regelauswertung werden als Permit- und Deny-Markierungen gesammelt und zu einer Gesamtentscheidung kombiniert.

Die konkreten Structs, Feldgrößen und Map-Typen dieser Objekte beschreibt Abschnitt 5.2.

4.5 Entwurfsentscheidungen

Die folgenden Entwurfsentscheidungen leiten aus den fachlichen Anforderungen und den technischen Rahmenbedingungen die grundlegende Gestaltung des Prototyps ab. Für jede Entscheidung werden das Ausgangsproblem, die gewählte Lösung, berücksichtigte Alternativen und die daraus entstehenden Konsequenzen dargestellt.

4.5.1 Policy-Verarbeitung im Userspace und Auswertung im Kernelspace

Das Reference-Monitor-Konzept verlangt eine manipulationssichere, verifizierbare Komponente zur Überprüfung der Zugriffe auf geschützte Ressourcen [And72, Abschnitt 3.4]. LSM-Hooks erlauben die Entscheidung unmittelbar im Kernel-Ausführungspfad [Wri+02]. Der Prototyp trifft daher die Entscheidung über eine neue Öffnung im Kernel, bereitet ihre Eingabedaten aber im Userspace vor.

Textuelles Parsen und Validieren benötigt Zeichenkettenverarbeitung, Zwischendaten und aussagekräftige Fehlerbehandlung. Eine solche Verarbeitung im eBPF-Programm würde dessen begrenzten Stack und die Verifier-Prüfung belasten. Gewählt wird deshalb eine Userspace-Verarbeitung, die lesbare Policydateien in feste Regeln mit Subjekt-, Kommando-, Ressourcen- und Bedingungsfeldern übersetzt. Der Kernel beschränkt sich auf Vergleiche, Map-Lookups und begrenzte Schleifen. Dies hält den unmittelbaren Durchsetzungspfad klein und nachvollziehbar.

Eine Alternative wäre das direkte Schreiben binärer Regeln in Maps, etwa mit `bpftool`. Dabei müssten Nutzer Layout und Validierung selbst sicherstellen; ganze Policy-Stände wären schwerer nachvollziehbar auszutauschen. Eine mächtigere Policy-Engine im Kernel würde mehr Ausdrucksstärke bieten, aber die eBPF-Grenzen stärker belasten. Eine Beschränkung auf rein statische Regeln würde dagegen die für ASBAC wesentlichen dynamischen Bedingungen auslassen.

Die gewählte Trennung erhält die nutzerseitige Policybeschreibung und begrenzt zugleich das unterstützte Kernelmodell. Erweiterungen müssen in dessen feste Repräsentation passen oder zusätzliche Vorverarbeitung erhalten. Der Userspace übernimmt außerdem die Attributbereitstellung und die nachträgliche Kontrolle bestehender Zugriffe; die unmittelbare Öffnungsentscheidung bleibt im Kernel.

4.5.2 Einsatz von eBPF-LSM statt eines klassischen LSM

Klassische LSM werden im Mainline-Linux in den Kernel eingebaut und können nicht wie gewöhnliche Kernelmodule zur Laufzeit nachgeladen werden [Ker26p]. Änderungen an ihrer Implementierung erfordern daher üblicherweise einen erneuten Kernelbuild und den Start des angepassten Kernels. Davon zu unterscheiden sind Änderungen an den Policies eines bestehenden LSM, etwa SELinux, die keinen erneuten Kernelbuild erfordern. Für den experimentellen Prototyp ist insbesondere die Möglichkeit relevant, den eigenen Zugriffskontrollcode ohne erneuten Kernelbuild zu ändern.

Gewählt wurde daher die Implementierung als eBPF-LSM. Diese Implementierung erlaubt es, eBPF-Programme an LSM-Hooks auszuführen und damit Zugriffskontrollentscheidungen im Kernel-Ausführungspfad zu treffen [Ker26q]. Bei einem entsprechend konfigurierten Kernel mit aktiviertem BPF-LSM können die Programme zur Laufzeit geladen und an unterstützte LSM-Hooks angehängt werden; vor der Ausführung werden sie durch den eBPF-Verifier geprüft [Ker26q; Ker26o].

Ein weiterer Grund für diese Entscheidung ist die Möglichkeit, eBPF-Programme mit Rust zu entwickeln. Der Prototyp nutzt dafür Aya (siehe Abschnitt 2.10). Dadurch können Userspace- und eBPF-Anteile innerhalb desselben Rust-Workspaces entwickelt werden. Zusätzlich können gemeinsame Datenstrukturen und Teile der Policy-Auswertungslogik in einer gemeinsamen Rust-Kompilationseinheit (Crate) abgebildet werden. Dies verbessert die Konsistenz zwischen Userspace und Kernelspace und reduziert doppelte Implementierungen.

Eine Alternative wäre die Entwicklung eines klassischen LSM gewesen. Diese Alternative bietet grundsätzlich größere Freiheiten im Kernelcode, erhöht jedoch Entwicklungs- und Wartungsaufwand. Für den Prototyp ist eBPF-LSM geeigneter, da Änderungen schneller getestet werden können und gleichzeitig ein sicherheitsrelevanter Ausführungspunkt im Kernel genutzt wird.

Konsequenzen dieser Entscheidung sind, dass die Implementierung die Einschränkungen von eBPF beachten muss, insbesondere die Prüfung durch den Verifier, die begrenzte Stack-Nutzung und die eingeschränkte Programmkomplexität. Diese Einschränkungen beeinflussen unter anderem die Aufteilung der eBPF-Programme in verschiedene Tail-Calls und die reduzierte Policy-Repräsentation im Kernelspace.

4.5.3 Auswahl des LSM-Hooks

Für den Prototyp musste ein LSM-Hook ausgewählt werden, an dem sich die unmittelbare Zugriffskontrolle und die spätere Überwachung bestehender Zugriffssituationen nachvollziehbar untersuchen lassen. Die Wahl fiel auf den Hook `file_open`. Dieser Hook wird im regulären Öffnungspfad über `security_file_open` aufgerufen und passt damit unmittelbar zu der Anforderung, Dateiöffnungen zu kontrollieren. Eine ablehnende Entscheidung bricht den Öffnungsvorgang ab, sodass dem aufrufenden Prozess kein FD für diese Öff-

nung zurückgegeben wird. Für den aufrufenden Prozess liefert open bei einer Ablehnung -1 statt eines neuen FDs zurück [Lin24a; Lin24b; Ker26g].

Eine unmittelbare Alternative für die Kontrolle von Dateizugriffen ist der Hook file_permission. Dieser wird über security_file_permission vor Lese- und Schreiboperationen auf einer bereits geöffneten Datei aufgerufen und ermöglicht insbesondere eine erneute Rechteprüfung nach Policy-Änderungen [Lin24b]. Würde die Policy ausschließlich an diesem Hook durchgesetzt, könnte die reguläre Dateiöffnung zunächst erfolgreich sein und einen FD zurückgeben, sofern die übrigen Berechtigungsprüfungen den Vorgang erlauben. Eine ablehnende Entscheidung würde dann die nachfolgende, vom Hook erfasste Lese- oder Schreiboperation verhindern; der bereits geöffnete FD würde dadurch nicht geschlossen [Lin24a; Lin24c]. Der Unterschied liegt somit zwischen der Verhinderung einer erfolgreichen Dateiöffnung und der erneuten Autorisierung einzelner Operationen über einen bestehenden FD.

Für den Prototyp wird file_open gewählt, weil bereits der Aufbau der betrachteten Zugriffssituation verhindert werden soll, wenn die Policy ihn nicht erlaubt. Nach einer erlaubten Öffnung kann der entstandene FD dagegen als Ansatzpunkt für die spätere Überwachung dienen. Wird seine weitere Nutzung durch eine Policy- oder Attributänderung unzulässig, veranlasst der Userspace-PEP das Schließen des betroffenen FDs. Bei erfolgreichem Schließen steht diese Referenz auf die Datei für neue Operationen nicht mehr zur Verfügung [Ker26b]. Damit werden die anfängliche Zulassung im Kernel und der nachträgliche Entzug im Userspace an derselben, über einen FD repräsentierten Zugriffssituation untersucht. Diese Aufteilung ersetzt keine synchrone Policy-Prüfung vor jeder Lese- oder Schreiboperation: Zwischen einer Änderung und dem nachträglichen Schließen verbleibt ein Zeitfenster.

Dateiöffnungen und fortdauernde FDs lassen sich mit Linux-Bordmitteln sowie /proc/<PID>/fd reproduzierbar beobachten [Ker26h]. Netzwerkhooks wie socket_bind würden einen anderen Zugriffstyp und zusätzliche Zustandsbeobachtung erfordern, ohne die hier untersuchte Fragestellung besser abzudecken.

Die Konsequenz dieser Entscheidung ist eine bewusste Begrenzung des Prototyps auf Dateiöffnungen. Der Prototyp trifft damit keine allgemeine Aussage über alle möglichen LSM-Hooks, sondern untersucht ASBAC exemplarisch anhand eines gut beobachtbaren und reproduzierbar testbaren Zugriffstyps.

4.5.4 Kommunikation über eBPF-Maps

Der Prototyp benötigt eine gemeinsame Datenbasis zwischen Userspace und Kernelspace. Der Userspace muss Policies, Attribute, Zeitinformationen und Generationenstände bereitstellen können. Das eBPF-Programm muss diese Daten im LSM-Hook lesen können, ohne direkt auf Userspace-Speicher oder komplexe Userspace-Datenstrukturen zuzugreifen. Zusätzlich müssen weitere Userspace-Komponenten, insbesondere Userspace-PEP und Administrationswerkzeug, den aktuellen Zustand lesen können.

Die Entscheidung lautet daher, die Kommunikation über eBPF-Maps umzusetzen. Policy-Einträge werden in Array-Maps abgelegt, dynamische Attribute in der Hash-Map `ATTRIBUTES`; `ATTRIBUTE_GENERATION` kennzeichnet den aktiven Attributstand. Die gemeinsame Zeitbasis liegt als Unix-Zeit in der einzigen Zeit-Map `CURRENT_TIME`. Der gemeinsame Typ `PolicyTime` leitet daraus für Kernelspace-PEP und Userspace-PEP identisch die benötigten UTC-Komponenten ab. Für Zustände, die über mehrere Komponenten hinweg zugreifbar sein müssen, werden gepinnte Maps verwendet. Dadurch erhalten Loader, Userspace-PEP und Administrationswerkzeug einen stabilen Zugriffspfad auf dieselben Kernelobjekte, während das eBPF-Programm die Werte direkt im Kernel auswerten kann. Die Map `DECISIONS` ist davon ausgenommen: Sie ist nicht gepinnt und speichert nur Zwischenergebnisse zwischen den Tail-Call-Stufen des Kernelspace-PEP. Diese Entscheidungen werden nicht in den Userspace übertragen, sondern erscheinen ausschließlich in den jeweiligen Logs.

Diese Entscheidung folgt aus dem eBPF-Ausführungsmodell. eBPF-Programme können nicht beliebige Speicherbereiche des Userspace lesen und sollen im Zugriffspfad mit festen, vom Verifier nachvollziehbaren Datenstrukturen arbeiten. eBPF-Maps sind der dafür vorgesehene Mechanismus, um Daten zwischen Userspace und eBPF-Programmen auszutauschen. Sie ermöglichen es, Policy- und Attributzustände im Kernel bereitzustellen und dennoch aus dem Userspace heraus zu aktualisieren. Zugleich passen Maps zu der reduzierten Policy-Repräsentation, weil sie feste Schlüssel- und Werttypen erzwingen.

Eine gleichwertige Alternative gibt es im Rahmen dieses Prototyps nicht. Direkte Übergaben über Funktionsparameter sind für dauerhafte Policy- und Attributzustände nicht geeignet, weil der LSM-Hook bei jedem Zugriff unabhängig ausgeführt wird. Ein erneutes Laden des eBPF-Programms bei jeder Policy- oder Attributänderung wäre unpraktisch und würde den laufenden Betrieb unnötig stören. Userspace-Dateien können vom eBPF-Programm ebenfalls nicht als normale Dateien gelesen und interpretiert werden. Damit bleiben eBPF-Maps die naheliegende und technisch vorgesehene Schnittstelle.

Die Konsequenz ist, dass alle zwischen Userspace und Kernel geteilten Zustände in feste Map-Layouts übersetzt werden müssen. Änderungen an diesen Layouts betreffen beide Seiten und müssen deshalb kontrolliert erfolgen. Gleichzeitig entsteht dadurch eine klare Schnittstelle: Der Userspace schreibt validierte Zustände in Maps, der Kernel liest diese Zustände bei der Zugriffskontrolle, und externe Komponenten können über gepinnte Maps lesend auf denselben Zustand zugreifen.

4.5.5 Generationenmodell für Policy-Aktualisierungen

Policies können während des Betriebs geändert, hinzugefügt oder entfernt werden. Dabei soll eine neue Policy-Generation erst nach vollständiger Vorbereitung zur Auswertung ausgewählt werden können. Problematisch wäre beispielsweise, wenn einzelne Map-Einträge bereits überschrieben wurden, andere Einträge derselben Policy-Menge aber noch zum vorherigen Stand gehören. In diesem Fall könnte eine Zugriffsentscheidung auf einer widersprüchlichen Mischung aus Einträgen der bisherigen und der neu vorbereiteten Policy-

Generation beruhen.

Die Entscheidung lautet daher, Policy-Aktualisierungen über ein Generationenmodell umzusetzen. Als Bank wird im Folgenden einer von zwei logisch getrennten Speicherbereichen innerhalb derselben Map bezeichnet. Eine Bank ist somit weder ein eigener eBPF-Map-Typ noch eine separate Map. Jede Bank kann einen vollständigen Policy- beziehungsweise Attributstand aufnehmen. Der aktuelle Generationenwert kennzeichnet eine Bank als aktiv. Die andere Bank kann für die nächste Generation vorbereitet werden. Eine bereits laufende Auswertung kann jedoch noch auf die zuvor ausgewählte Bank zugreifen. Die Policy-Maps werden in zwei Bänke aufgeteilt. Eine Bank enthält die aktuell aktive Policy-Generation, während die andere Bank als inaktive Bank für die nächste Generation vorbereitet wird. Bei einer Änderung schreibt der Userspace zunächst alle vorbereiteten Policy-Einträge in die inaktive Bank. Erst wenn dieser Schreibvorgang erfolgreich abgeschlossen ist, wird der Generationenzähler aktualisiert. Das eBPF-Programm liest den aktiven Generationenwert und bestimmt daraus, welche Bank bei der Auswertung verwendet wird.

Dieses Vorgehen reduziert das Risiko inkonsistenter Zustände. Solange die neue Bank noch nicht vollständig geschrieben ist, bleibt die bisherige Generation aktiv. Schlägt das Parsen, Validieren oder Schreiben der neuen Policies fehl, wird der Generationenzähler nicht umgeschaltet und die letzte gültige Generation bleibt weiter wirksam. Damit passt das Modell zu der Anforderung, nur erfolgreich verarbeitete Policy-Stände als Entscheidungsgrundlage zu verwenden. Dasselbe Grundprinzip wird auch für dynamische Attribute verwendet: Attribute werden bankweise geschrieben und anschließend über eine eigene Attributgeneration aktiviert. Policy- und Attributgeneration werden unabhängig voneinander aktiviert. Sie bilden daher keine gemeinsame Transaktion und garantieren keine zusammengehörige Lesesicht beider Bestände.

Eine Alternative wäre, Policy-Einträge direkt an ihrer aktiven Position zu überschreiben. Diese Variante wäre einfacher, könnte aber während der Aktualisierung zu Zwischenzuständen führen, die weder der alten noch der neuen Policy-Menge vollständig entsprechen. Eine weitere Alternative wäre, die Zugriffskontrolle während einer Aktualisierung anzuhalten oder das eBPF-Programm vollständig neu zu laden. Beides wäre für einen laufenden Prototyp unnötig invasiv und würde den Betrieb stärker beeinflussen.

Die Konsequenz des Generationenmodells ist ein zusätzlicher Verwaltungsaufwand. Die Maps müssen Platz für mehrere Bänke bereitstellen, und Userspace sowie Kernelspace müssen denselben Mechanismus zur Berechnung der aktiven Bank verwenden. Die Umschaltlogik stellt sicher, dass eine neue Generation erst nach vollständiger Vorbereitung aktiviert wird. Eine unveränderte Lesesicht während einer laufenden Kernel-Auswertung garantiert das Zwei-Bank-Verfahren allein jedoch nicht: Bei weiteren Aktualisierungen kann die von dieser Auswertung verwendete Bank erneut beschrieben werden.

4.5.6 Getrennte Repräsentation statischer Policies und Streampolicies

Statische Policies verwenden nur Eigenschaften der aktuellen Anfrage; Streampolicies berücksichtigen zusätzlich Zeit oder externe Attribute. Sie werden für `file_open` in getrennten Maps gespeichert und in getrennten Stufen ausgewertet. Der statische Pfad benötigt dadurch keine dynamische Zusatzlogik, während der Stream-Pfad Zeitwerte und Attribut-Lookups behandelt. Anschließend werden die Ergebnisse zusammengeführt.

Eine gemeinsame Map mit einem Typfeld würde einen Auswertungspfad für beide Fälle und zusätzliche Verzweigungen erfordern. Auch statische Regeln ausschließlich als Streampolicies ohne dynamische Bedingungen abzubilden würde den einfachen Fall durch diesen komplexeren Pfad führen. Die getrennten Maps verursachen Verwaltungsaufwand, halten aber Einträge und Kontrollfluss auf ihren jeweiligen Zweck begrenzt.

Nur `file_open` ist umgesetzt. Weitere Hooks könnten eigene Map-Layouts benötigen, weil sie andere Ressourcen- und Kontextdaten bereitstellen.

4.5.7 Tail-Call-basierte Aufteilung der eBPF-Programme

Die Policy-Auswertung besteht aus mehreren Teilschritten. Zunächst muss der Hook-Einstieg die Anfrage erfassen. Danach werden statische Policies ausgewertet, anschließend Streampolicies, und zum Schluss werden die Zwischenergebnisse zu einer finalen Entscheidung kombiniert. Würden alle Schritte in einem einzigen eBPF-Programm umgesetzt, würde dieses Programm größer und für den Verifier schwerer analysierbar. Gerade bei eBPF sind Programmgröße, Kontrollfluss und Stacknutzung praktische Grenzen, die den Entwurf unmittelbar beeinflussen.

Die Entscheidung lautet daher, die Auswertung über Tail Calls in mehrere eBPF-Programme aufzuteilen. Der Einstieg am LSM-Hook springt über eine Program-Array-Map in den statischen Auswertungsschritt. Dieser springt anschließend in den Stream-Auswertungsschritt, und dieser wiederum in das Combining-Programm. Jeder Schritt übernimmt damit eine klar begrenzte Aufgabe.

Diese Aufteilung reduziert die Komplexität der einzelnen eBPF-Programme. Der statische Pfad muss keine dynamischen Attribute auswerten, der Stream-Pfad muss nicht die gesamte Hook-Initialisierung enthalten, und das Combining kann auf bereits gespeicherte Zwischenergebnisse zugreifen. Gleichzeitig entspricht die Aufteilung der fachlichen Struktur der Entscheidung: Anfrage erfassen, Policy-Gruppen auswerten, Ergebnisse zusammenführen. Sie erleichtert außerdem die spätere Anpassung einzelner Schritte, ohne die gesamte Auswertung in einem großen Programm ändern zu müssen.

Eine Alternative wäre ein monolithisches eBPF-Programm. Diese Variante hätte weniger Tail-Call-Verwaltung benötigt, würde aber die Verifier-Komplexität erhöhen und frühere Probleme mit zu großen Programmen begünstigen. Eine weitere Alternative wäre, weniger Policy-Funktionalität im Kernel abzubilden, um ein einzelnes Programm klein genug

zu halten. Das würde jedoch den Anspruch des Prototyps einschränken, statische und dynamische Bedingungen gemeinsam untersuchen zu können.

Die Konsequenz ist, dass eine zusätzliche Tail-Call-Tabelle gepflegt werden muss. Außerdem muss beachtet werden, dass Tail Calls eigene eBPF-Einschränkungen mit sich bringen, insbesondere hinsichtlich Stacknutzung. Dafür bleibt jeder Auswertungsschritt kleiner und besser kontrollierbar.

4.5.8 Combining-Strategie für Policy-Entscheidungen

Mehrere Policies können auf dieselbe Zugriffssituation anwendbar sein. Dabei können einzelne Policies zu unterschiedlichen Aussagen kommen, etwa *Permit* oder *Deny*. Der Prototyp benötigt daher eine Strategie, mit der die Zwischenergebnisse der Policy-Auswertung zu einer finalen Hook-Entscheidung zusammengeführt werden. Eine solche Regel wird als Combining-Algorithmus bezeichnet. XACML unterscheidet unter anderem *Deny-overrides*, bei dem eine Deny-Entscheidung Vorrang hat, *Permit-overrides*, bei dem eine Permit-Entscheidung Vorrang hat, und *First-applicable*, bei dem das Ergebnis der ersten anwendbaren Policy übernommen wird [OAS17]. Diese Arbeit übernimmt damit nur die etablierte Terminologie zur Einordnung der Konfliktauflösung, nicht die XACML-Policy-Sprache oder deren vollständiges Entscheidungsmodell.

Die Entscheidung lautet, Zwischenergebnisse zunächst in einem kleinen Entscheidungszustand abzulegen und die finale Entscheidung in einem separaten Combining-Schritt zu treffen. Die statische und die streambasierte Auswertung halten fest, ob während ihres jeweiligen Auswertungspfades eine passende Permit- oder Deny-Policy gefunden wurde. Das Combining-Programm liest diesen Zustand und entscheidet daraus, ob der Hook den Zugriff erlaubt oder verweigert. Im aktuellen Prototyp wird eine vereinfachte, an *Deny-overrides* angelehnte Strategie verwendet: Sobald eine Deny-Entscheidung vorliegt, wird der Zugriff verweigert; andernfalls wird er erlaubt. Die XACML-Termini *Indeterminate* und *NotApplicable* werden dabei nicht übernommen.

Diese Aufteilung trennt Policy-Matching und finale Konfliktauflösung. Die einzelnen Auswertungspfade müssen nicht selbst die vollständige globale Entscheidung treffen, sondern liefern nur ihre Zwischenergebnisse. Das macht die Struktur des eBPF-Programms nachvollziehbarer und erlaubt es, die Combining-Logik an einer klar abgegrenzten Stelle zu betrachten.

Eine Alternative wäre eine reihenfolgeabhängige *First-applicable*-Strategie, bei der das Ergebnis der ersten anwendbaren Policy als Gesamtentscheidung übernommen und die Auswertung anschließend beendet wird. Diese Variante wäre für eine feste Strategie einfacher, würde aber die spätere Untersuchung anderer Combining-Algorithmen erschweren. Außerdem würde die Reihenfolge der Policy-Auswertung stärker in die Semantik eingehen.

Die Konsequenz ist, dass ein zusätzlicher Entscheidungszustand zwischen den Tail-Call-Schritten benötigt wird. Dafür bleibt die Konfliktauflösung zentralisiert. Der Prototyp

verwendet aktuell die vereinfachte, an *Deny-overrides* angelehnte Strategie, ohne die Policy-Auswertung selbst fest auf einen frühen Abbruch nach der ersten passenden Policy zu beschränken.

Für fachlich nicht erfüllte Bedingungen wird keine Deny-Markierung gesetzt. Fehlt ein benötigtes Attribut oder passt sein Typ nicht, greift die betreffende Policy nicht. Tabelle 1 zeigt dies für eine einzelne Deny-Policy mit `system.clearance <= 2`. Vorausgesetzt werden ein technisch fehlerfreier Auswertungspfad, ein passender Ressourcenfilter und keine weiteren Deny-Policies.

Attributzustand	**Wirkung der Policy**	**Ergebnis**
`clearance = 2`	Bedingung erfüllt; Deny greift	Verweigert
`clearance = 5`	Bedingung nicht erfüllt	Erlaubt
`clearance` fehlt	Bedingung nicht erfüllt	Erlaubt
`clearance = low"`	Zeichenkette statt erwarteter Zahl; Bedingung nicht erfüllt	Erlaubt

Tabelle 1: Wirkung einer Deny-Policy mit der Bedingung `system.clearance <= 2` ohne weitere Deny-Policies

Ohne passende Deny-Policy gilt Default-Allow. Eine alleinige Permit-Policy beschränkt Zugriffe daher nicht auf Fälle mit erfüllter Bedingung; eine passende Deny-Policy überstimmt sie. Ein syntaktisch gültiger, aber zur Bedingung typabweichender Attributwert ist von einer ungültigen Quelldatei zu unterscheiden: Letztere wird nicht aktiviert und lässt die bisherige Generation bestehen. Technische Fehler im Durchsetzungspfad werden gesondert behandelt (Abschnitt 5.9).

4.5.9 Identifikation von Dateien über Device und Inode

Policies werden für Nutzer über Dateipfade formuliert. Für die Auswertung im Kernel ist ein Pfadvergleich jedoch ungünstig. Pfade sind Zeichenketten, können unterschiedlich geschrieben werden und sind nicht zwingend eine eindeutige Beschreibung des zugrunde liegenden Dateiobjekts. Beispiele sind symbolische Links, relative Pfade oder mehrere Pfade, die auf dasselbe Dateiobjekt verweisen können. Ein Zeichenkettenvergleich im eBPF-Programm wäre zudem unnötig aufwendig und würde die Policy-Auswertung vergrößern.

Die Entscheidung lautet daher, Dateipfade im Userspace in die Dateiidentität aus Device und Inode zu übersetzen. Beim Verarbeiten der Policy wird der angegebene Pfad über Dateimetadaten aufgelöst. In der Policy-Map werden anschließend Device und Inode gespeichert. Beim `file_open`-Hook liest das eBPF-Programm die entsprechenden Werte aus dem Kernelkontext der geöffneten Datei und vergleicht diese mit den vorbereiteten Policy-Werten.

Diese Repräsentation passt besser zum Kernelkontext. Beim geöffneten Dateiobjekt liest der Kernel die Inode-Nummer aus dem Feld `i_ino` des Inodes und die Device-Kennung aus dem Feld `s_dev` des zugehörigen Superblocks. Die Kombination beider Werte dient

dem Prototyp zur Identifikation der Datei. Dadurch muss das eBPF-Programm keinen vollständigen Pfad rekonstruieren und keine langen Zeichenketten vergleichen. Außerdem werden unterschiedliche Pfadschreibweisen reduziert, weil die Entscheidung auf der identifizierten Datei und nicht auf der Schreibweise des Pfades basiert.

Eine Alternative wäre ein Pfadvergleich im Kernel. Dafür müssten Pfade oder Pfadbestandteile in Maps gespeichert und im eBPF-Programm verglichen werden. Diese Variante wäre komplexer, anfälliger für Uneindeutigkeiten und stärker von Pfadnormalisierung abhängig. Eine weitere Alternative wäre ein Hash des Pfades. Auch dieser würde jedoch weiterhin an der Pfadschreibweise hängen und nicht am tatsächlich geöffneten Dateiobjekt.

Die Konsequenz ist, dass Policies beim Laden an das zu diesem Zeitpunkt vorhandene Dateiobjekt über dessen Device-Inode-Identität gebunden werden. Diese Identität ist kein Hash des Dateiinhalts; Inhaltsänderungen am selben Dateiobjekt ändern sie nicht. Wird eine Datei später ersetzt, kann sich die Inode ändern und die Policy muss entsprechend neu verarbeitet werden. Dies betrifft die geschützte Ressource und ist von der atomaren Veröffentlichung einer Policy- oder Attributdatei zu unterscheiden. Dafür wird die Auswertung im Kernel deutlich einfacher und näher an der tatsächlichen Dateiidentität durchgeführt.

4.5.10 Nachbewertung und selektiver FD-Entzug

Nach einer erlaubten Öffnung prüft `file_open` den bestehenden Zugriff nicht erneut. Für ASBAC ist jedoch relevant, dass spätere Policy- oder Attributänderungen dessen Zulässigkeit verändern können. Gewählt wird deshalb ein ergänzender Userspace-PEP, der nach erfolgreichen entscheidungsrelevanten Aktivierungen und an relevanten Zeitgrenzen laufende Prozesse und offene FDs rekonstruiert. Er bewertet sie lokal anhand derselben gepinnten Policies und Attribute; dies ist seine integrierte PDP-Funktion. Ein gefundener FD ist eine technische Repräsentation fortdauernder Nutzung, keine registrierte ASBAC-Subscription.

Bei einem Verstoß versucht der Prototyp, den betroffenen FD im Zielprozess zu schließen. Ein Prozessabbruch wäre einfacher, würde aber auch legitime Zugriffe desselben Prozesses beenden. Reines Protokollieren oder der Verzicht auf eine Nachbewertung würde die geforderte nachträgliche Durchsetzung nicht abdecken. Der selektive Entzug erhält dagegen andere, nicht betroffene Deskriptoren. Seine konkrete `ptrace`-Umsetzung für `x86_64`-Linux beschreibt Abschnitt 5.7.3; sie benötigt weitreichende Berechtigungen und kann den Zielprozess kurz anhalten.

Neubewertung und Entzugsversuch bilden keine atomare Operation: Zwischen dem Einlesen der Prozessinformationen, der Auswertung und dem Schließen kann sich die FD-Zuordnung verändern. Das Schließen eines Descriptors beendet zudem weder bereits laufende E/A noch Zugriffe über duplizierte oder vererbte FDs oder bestehende Speicherabbildungen (vgl. Abschnitt 2.12). Ein vollständiger oder verzögerungsfreier Entzug ist deshalb nicht garantiert. Der Userspace-PEP ergänzt die unmittelbare Öffnungskontrolle durch eine nachträgliche Best-Effort-Maßnahme.

4.5.11 Administration und Diagnose über ein separates Tool

Für Betrieb und Evaluation des Prototyps muss sichtbar sein, welche Policies, Generationen und Attribute aktuell im Kernelzustand vorliegen. Diese Informationen liegen nicht nur in den ursprünglichen Textdateien, sondern auch in der übersetzten Map-Repräsentation. Gerade bei dynamischen Attributen und gehashten Attributnamen ist eine Diagnosemöglichkeit wichtig, um nachvollziehen zu können, welche Werte tatsächlich geladen wurden.

Die Entscheidung lautet, Administration und Diagnose über ein separates, lesendes Werkzeug umzusetzen. Das Administrationswerkzeug öffnet die gepinnten Maps und gibt aktive Policy-Bänke, statische Policies, Streampolicies und Attribute aus. Für gehashte Attributnamen und Zeichenkettenwerte baut es zusätzlich ein Wörterbuch aus den Policy- und Attributdateien auf. Dadurch können viele Hashwerte wieder einer lesbaren Bezeichnung zugeordnet werden, ohne dass die Kernel-Maps selbst Klartextnamen speichern müssen.

Diese Trennung verhindert, dass Diagnosefunktionen mit der eigentlichen Policyverwaltung vermischt werden. Der Loader bleibt für das Verarbeiten und Schreiben gültiger Zustände zuständig. Das Administrationswerkzeug dient dagegen der Sichtbarkeit des aktuellen Zustands. Es arbeitet bewusst lesend, damit es den Kernelzustand nicht unbeabsichtigt verändert und nicht in Konkurrenz zum Policy-Sync tritt.

Eine Möglichkeit wäre die direkte Nutzung allgemeiner Werkzeuge wie `bpftool`. Damit lassen sich Maps zwar grundsätzlich inspizieren, die Ausgabe ist jedoch stärker an der binären Map-Struktur orientiert und für die fachliche Bewertung weniger komfortabel. Eine weitere Alternative wäre, Diagnose direkt in das Hauptprogramm oder den Userspace-PEP einzubauen. Dies würde jedoch Zuständigkeiten vermischen und die Beobachtung des Zustands stärker an einen laufenden Prozess binden.

Die Konsequenz ist ein zusätzliches Werkzeug im Prototyp. Dafür können geladene Policies und Attribute unabhängig von der Policyverwaltung eingesehen werden. Die Diagnose bleibt allerdings teilweise best-effort: Wenn ein Hashwert nicht aus den vorhandenen Policy- oder Attributdateien rekonstruiert werden kann, kann das Tool ihn nur als Hash beziehungsweise unbekannten Wert anzeigen.

5 Entwurf und Umsetzung

Auf Grundlage der zuvor entwickelten Konzeption beschreibt dieses Kapitel deren technische Ausgestaltung im entwickelten Prototyp. Zunächst wird die Implementierungsstruktur des Rust-Workspaces dargestellt. Anschließend werden die gemeinsamen Datenstrukturen und die gepinnten eBPF-Maps als Schnittstelle zwischen Userspace und Kernelspace erläutert. Darauf aufbauend werden die Initialisierung des Prototyps, die Verarbeitung von Policies und dynamischen Attributen sowie die Zugriffskontrolle für neue und bereits bestehende Dateizugriffe beschrieben. Den Abschluss bilden das Administrationstool, die Fehlerbehandlung sowie die technischen Grenzen von Build und Zielsystem.

5.1 Implementierungsstruktur

Die Implementierungsstruktur gliedert den Prototyp in mehrere Rust-Crates. Die folgende Darstellung erläutert zunächst ihre Aufgaben und anschließend die Abhängigkeiten zwischen ihnen.

5.1.1 Aufteilung des Rust-Workspaces

Die Umsetzung ist in mehrere Rust-Crates mit klar abgegrenzten Verantwortlichkeiten aufgeteilt. Diese Struktur bildet die zuvor begründete Trennung zwischen Userspace und Kernelspace technisch ab. Die Crate `tails-pdp-ebpf` enthält den eBPF-Programmcode, der im Kernel ausgeführt wird und deshalb den dort geltenden Einschränkungen unterliegt. Die Userspace-Crates übernehmen dagegen Initialisierung, Policy- und Attributverarbeitung, Überwachung bestehender Zugriffe sowie Administration. Gemeinsame Datenstrukturen befinden sich in einer separaten Crate, damit Userspace und eBPF-Programm dieselben Map-Layouts verwenden. Tabelle 2 fasst die Aufteilung zusammen.

5.1.2 Abhängigkeiten zwischen den Crates

Die folgenden Abhängigkeiten beziehen sich auf die im Workspace definierten internen Crates. Zunächst ist zwischen einer regulären Cargo-Abhängigkeit und einer Build-Abhängigkeit zu unterscheiden. Reguläre Cargo-Abhängigkeiten stehen im Manifestabschnitt `dependencies` und dem Programmcode der jeweiligen Crate zur Verfügung. Build-Abhängigkeiten werden dagegen im Abschnitt `build-dependencies` angegeben und können ausschließlich durch das Build-Skript `build.rs` verwendet werden; sie stehen dem eigentlichen Programmcode nicht automatisch zur Verfügung [Rus26b].

`tails-pdp-ebpf` ist eine Build-Abhängigkeit des Hauptprogramms zur Erzeugung des eingebetteten eBPF-Artefakts; reguläre Rust-Funktionsaufrufe zwischen diesen Crates finden nicht statt. Die technische Umsetzung beschreibt Abschnitt 5.10.

Die regulären internen Abhängigkeiten sind wie folgt aufgebaut:

Crate	Aufgabe
`tails-pdp`	Hauptprogramm zum Laden des eBPF-Objekts, Einrichten der gepinnten Maps und Tail Calls sowie Starten der Userspace-Komponenten.
`tails-pdp-policy-loader`	Einlesen, Validieren und Übersetzen von Policydateien sowie Aktivieren vollständiger Policy-Generationen.
`tails-pdp-attribute-loader`	Bereitstellung und Aktualisierung von Zeit-, System-, Subject- und Ressourcenattributen.
`tails-pdp-userspace-common`	Gemeinsame Hilfsfunktionen und Zugriff auf die gepinnten eBPF-Maps für Userspace-Komponenten.
`tails-pdp-userspace-pep`	Überwachung bestehender Dateizugriffe und gezielter Entzug unzulässiger FDs.
`tails-pdp-common`	Gemeinsam verwendeter Code mit Datenstrukturen und Auswertungslogik, insbesondere Policy-, Attribut- und Map-Strukturen.
`tails-pdp-ebpf`	Enthält die im Kernel ausgeführten eBPF-LSM-Programme für den Hook `file_open`, die Policy-Auswertung, Tail Calls und die Durchsetzung der Zugriffsentscheidung.
`tails-pdp-admintool`	Administrationstool zur Anzeige aktiver Policies, Attribute und Generationen aus gepinnten eBPF-Maps.

Tabelle 2: Aufteilung des Rust-Workspaces in Crates und Verantwortlichkeiten

- Das Hauptprogramm `tails-pdp` bindet `tails-pdp-attribute-loader`, `tails-pdp-policy-loader`, `tails-pdp-userspace-common`, `tails-pdp-userspace-pep` und `tails-pdp-common` ein.

- `tails-pdp-attribute-loader`, `tails-pdp-policy-loader` und `tails-pdp-userspace-pep` verwenden jeweils `tails-pdp-common` für gemeinsame Datenstrukturen sowie `tails-pdp-userspace-common` für den Zugriff auf die Userspace-seitigen Map-Funktionen.

- Das Kernelprogramm `tails-pdp-ebpf` hängt ausschließlich von `tails-pdp-common` ab. Dadurch bleiben die gemeinsamen Map-Layouts und Policy-Strukturen zwischen Userspace und Kernelspace konsistent, ohne dass das eBPF-Programm von Userspace-spezifischen Crates abhängt.

- Das eigenständig gestartete `tails-pdp-admintool` verwendet `tails-pdp-common`, liest die gepinnten Maps jedoch selbst und hängt deshalb nicht von `tails-pdp-userspace-common` ab.

- `tails-pdp-common` und `tails-pdp-userspace-common` besitzen keine weiteren internen Crate-Abhängigkeiten.

Die gemeinsamen Crates trennen die Datenrepräsentation von der Infrastruktur im Userspace. `tails-pdp-common` enthält die von eBPF-Programm und Userspace verwendeten Datenstrukturen, während `tails-pdp-userspace-common` ausschließlich Funktionen zum Öffnen und Verwenden gepinnter Maps im Userspace bereitstellt. Abbildung 2 fasst die daraus entstehenden Abhängigkeiten zusammen.

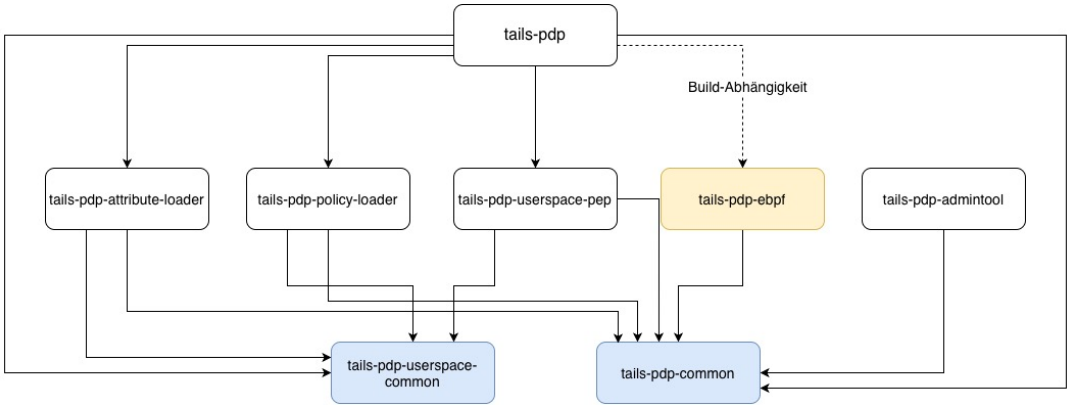


Abbildung 2: Interne Abhängigkeiten der Rust-Crates

Durchgezogene Pfeile kennzeichnen reguläre Cargo-Abhängigkeiten, der gestrichelte Pfeil die Build-Abhängigkeit zur Erzeugung des eingebetteten eBPF-Artefakts. Pfeile zeigen von der abhängigen Crate zur verwendeten Crate. Externe Abhängigkeiten sind nicht dargestellt.

5.2 Gemeinsame Datenstrukturen und Map-ABI

Die eBPF-Maps bilden die gemeinsame Datenschnittstelle zwischen Userspace und Kernelspace. Sie speichern Schlüssel und Werte als Bytefolgen, denen keine Typinformationen aus Rust beigefügt sind. Damit ein im Userspace geschriebener Map-Eintrag vom eBPF-Programm eindeutig gelesen werden kann, müssen beide Ausführungsbereiche dieselbe Binärrepräsentation der verwendeten Datenstrukturen zugrunde legen. Dies betrifft insbesondere Feldreihenfolge, Feldgrößen und Ausrichtung der Schlüssel und Werte.

Diese gemeinsame Repräsentation bildet die Map-ABI des Prototyps. Die hierfür benötigten Strukturen sind in `tails-pdp-common` gebündelt und werden sowohl vom eBPF-Programm als auch von den Userspace-Komponenten verwendet. Die folgenden Unterabschnitte beschreiben die kernelgeeignete Darstellung von Policies und Attributen, die Identifikation von Ressourcen in Map-Schlüsseln sowie Aufbau und Pinning der verwendeten Maps. Das Einlesen, Validieren und Übersetzen der Policy- und Attributdateien wird dagegen in den nachfolgenden Abschnitten behandelt.

5.2.1 Kernelgeeignete Policy-Strukturen

Die Policy-Repräsentation ist auf eine konstante Größe und einen vorhersehbaren Kontrollfluss im eBPF-Programm ausgelegt. Für den Hook `file_open` existieren deshalb getrennte Rust-Structs für statische Policies und Streampolicies. Beide Varianten enthalten die Zugriffsentscheidung, die Aktion, einen Aktivierungsstatus, das Subject sowie die Bedingungen für Kommando und Ressource. Die Kommandobezeichnung wird als Byte-Array mit 16 Einträgen und der vom Nutzer angegebene Ressourcenpfad als Byte-Array mit 64 Einträgen gespeichert. Zusätzlich enthalten beide Policy-Typen Device und Inode als numerische Ressourcenidentität. Der Kernel verwendet für den Ressourcenvergleich ausschließlich diese beiden Werte; der Pfad bleibt für die Darstellung im Userspace erhalten.

Der im Rust-Code als `struct` mit festem Speicherlayout definierte Map-Werttyp `FileOpenStaticPolicy` enthält nur die Bedingungen, die unmittelbar mit dem Öffnungsvorgang verglichen werden können. `FileOpenStreamPolicy` erweitert diese Repräsentation um eine optionale Zeitbedingung und ein fest dimensioniertes Feld für Attributbedingungen. Jede dieser Bedingungen besteht aus Namespace, Vergleichsoperator, Werttyp, Hash des Attributnamens und dem Vergleichswert. Die Anzahl der aktiven Bedingungen wird gesondert gespeichert; das zugrunde liegende Array besitzt dennoch stets eine feste Obergrenze von vier Einträgen. Damit kann die Auswertung im eBPF-Programm mit begrenzten Schleifen erfolgen, ohne Speicher dynamisch reservieren zu müssen.

Beide Policy-Typen liegen in separaten Array-Maps; nicht belegte Einträge enthalten deaktivierte Policies. Ihre Erzeugung und Aktivierung beschreibt Abschnitt 5.4, das Matching Abschnitt 5.6.

5.2.2 Repräsentation dynamischer Attribute

Das Attributmodell muss frei wählbare Namen und Werte abbilden, obwohl eBPF-Maps nur Strukturen fester Größe speichern können. Ein Attribut wird deshalb durch zwei getrennte Rust-Structs repräsentiert. `AttributeKey` ist ausschließlich der Map-Schlüssel und enthält die Attributbank, den Namespace, zwei Objektidentifikatoren und einen 128-Bit-Hash des Attributnamens. Er legt damit fest, zu welchem System-, Subject- oder Ressourcenobjekt ein Attribut gehört und unter welchem Namen es angesprochen wird. Der typisierte Inhalt ist nicht Teil dieses Schlüssels, sondern wird separat als Map-Wert `AttributeValue` gespeichert.

`AttributeValue` unterscheidet numerische, boolesche und Zeichenkettenwerte. Numerische Werte werden direkt als `u64` gespeichert. Boolesche Werte belegen dasselbe `u64`-Feld und verwenden darin `0` für `false` und `1` für `true`. Obwohl dafür ein einzelnes Bit ausreichen würde, hält das gemeinsame Zahlenfeld den Map-Wert fest dimensioniert und vermeidet unterschiedliche ABI-Layouts für Zahlen und boolesche Werte. Für Zeichenketten wird nicht der Text selbst, sondern sein 128-Bit-Hash abgelegt. Die Namens- und Zeichenkettenhashes sind deterministische 128-Bit-Werte aus zwei 64-Bit-Komponenten. Sie begrenzen Speicherbedarf und Vergleichsaufwand, ersetzen aber keine kryptographische Kollisionsgarantie.

Policybedingungen speichern denselben Namenshash und den erwarteten typisierten Wert. Das Einlesen und Validieren der Quelldateien erläutert Abschnitt 5.5; die Verwendung dieser Felder beim Matching folgt in Abschnitt 5.6.

5.2.3 Objektidentitäten im Map-Schlüssel

Die beiden Objektidentifikatoren im Attributschlüssel werden abhängig vom Namespace belegt. Für Systemattribute sind beide Werte `0`, da sie systemweit gelten. Subject-Attribute

verwenden die numerische UID als primären Identifikator; der zweite Wert bleibt null. Ressourcenattribute verwenden dagegen Device- und Inode-Wert der Datei. Diese Zuordnung wird sowohl beim Einlesen der Attributdateien im Userspace als auch bei der eBPF-Auswertung verwendet.

Device und Inode liegen als getrennte `u64`-Werte vor, nicht als Hash. Ihre Bedeutung als Dateiobjektidentität erläutert Abschnitt 4.5.9; die Auflösung aus einem Ressourcenpfad erfolgt beim Laden der jeweiligen Quelldateien. Eine Änderung von Größe, Reihenfolge oder Bedeutung dieser Schlüsselfelder wäre eine Änderung der Map-ABI.

5.2.4 Aufbau und Pinning der eBPF-Maps

Tabelle 3 zeigt die gepinnten Maps, die den dauerhaften Datenaustausch zwischen den Komponenten ermöglichen. Die Policy- und Attributloader schreiben gültige Daten in diese Maps. Das eBPF-Programm liest sie bei der Zugriffsentscheidung; Userspace-PEP und Administrationstool öffnen dieselben Maps bei Bedarf erneut zur Überwachung beziehungsweise Anzeige.

Map	Typ	Schlüssel und Wert	Aufgabe
`POLICY_GENERATION`	Array	`u32` → `u32`	Speichert die aktive Policy-Generation und bestimmt damit die auszuwertende Policy-Bank.
`FILE_OPEN_STATIC_POLICIES`	Array	`u32` → `FileOpenStaticPolicy`	Enthält die statischen `file_open`-Policies beider Policy-Banken.
`FILE_OPEN_STREAM_POLICIES`	Array	`u32` → `FileOpenStreamPolicy`	Enthält die attribut- und zeitabhängigen `file_open`-Policies beider Policy-Banken.
`CURRENT_TIME`	Array	`u32` → `u64`	Stellt den aktuellen Unix-Zeitstempel für zeitabhängige Bedingungen bereit.
`ATTRIBUTE_GENERATION`	Array	`u32` → `u32`	Speichert die aktive Attributgeneration und damit die aktive Attributbank.
`ATTRIBUTES`	Hash-Map	`AttributeKey` → `AttributeValue`	Enthält die dynamischen System-, Subject- und Ressourcenattribute.

Tabelle 3: Gepinnte eBPF-Maps als Map-ABI des Prototyps

Beim Start legt `tails-pdp` das Pin-Verzeichnis `/sys/fs/bpf/tails-pdp` an und lädt das eBPF-Objekt mit diesem Verzeichnis als Standardpfad für Maps. Gepinnte Maps bleiben im BPF-Dateisystem erreichbar und können deshalb von später gestarteten Userspace-Komponenten über ihren Pfad geöffnet werden. Die nicht gepinnten Maps für Tail Calls, temporäre Entscheidungen und Debug-Konfiguration gehören dagegen ausschließlich zum geladenen eBPF-Objekt und sind nicht Teil der gemeinsamen Map-ABI.

Gepinnte Maps können einen Programmneustart überdauern. Ihre Kompatibilitätsprüfung und die Startreihenfolge beschreibt Abschnitt 5.3.

5.3 Initialisierung und Laufzeitsteuerung

Die Ausführungssteuerung ist im initialen Programm `tails-pdp` gebündelt. Es richtet die Voraussetzungen für den Kernelspace-PEP ein, stellt die anfänglichen Policy- und Attributdaten bereit und startet anschließend die begleitenden Userspace-Komponenten. Damit wird die Durchsetzung nicht an einen noch unvollständig initialisierten Bestand aus Maps oder Eingabedateien angehängt.

5.3.1 Laden und Prüfen des eBPF-Programms

Zu Beginn initialisiert das Hauptprogramm die Protokollierung. Anschließend wird das Pin-Verzeichnis `/sys/fs/bpf/tails-pdp` angelegt und das Layout bereits vorhandener, aus einer eventuell vorherigen Version des Prototyps stammenden, gepinnter Maps geprüft. Sollten hier Maps mit einem falschen Layout vorhanden sein, wird der Prototyp beendet. Die Prüfung erfolgt vor dem Laden des eBPF-Objekts, damit eine veraltete Map-ABI nicht unbemerkt mit den aktuellen Rust-Strukturen kombiniert wird. Bei einer Abweichung von Schlüsselgröße, Wertgröße oder maximaler Eintragszahl bricht die Initialisierung ab. Die Fehlermeldung nennt Pin-Pfad sowie erwartete und gefundene Werte. Vorhandene Maps werden nicht automatisch gelöscht; nach einer bewussten ABI-Änderung muss der Betreiber inkompatible Maps entfernen. Dies verhindert eine fehlerhafte Interpretation vorhandener Map-Bytes.

Der Aya-Loader lädt das in Abschnitt 5.10 beschriebene eBPF-Artefakt mit dem Pin-Verzeichnis als Standardpfad für seine Maps. Anschließend liest er die zuvor eingeführten BTF-Metadaten des Zielkernels und verwendet sie beim Laden der LSM-Programme, um den vorgesehenen Hook und dessen Typinformationen aufzulösen. Gleichzeitig fordert der Loader ausführliche Verifier-Informationen an. Fehlt beispielsweise `/sys/kernel/btf/vmlinux`, ist der Hook nicht verfügbar oder lehnt der Verifier ein Programm ab, beendet das Hauptprogramm den Start mit einer kontextbezogenen Fehlermeldung. Die folgenden Schritte bereiten Maps, Tail Calls und Eingabedaten vor dem Anheften des Hooks vor.

5.3.2 Initialisierung der Maps und Tail Calls

Nach dem Laden öffnet das Hauptprogramm die zum Objekt gehörenden Maps. Die nicht gepinnte Array-Map `DEBUG_LOGGING` enthält einen Schalter für optionale Diagnoseausgaben des eBPF-Programms. Das Hauptprogramm setzt ihn anhand der Umgebungsvariablen `TAILS_PDP_EBPF_DEBUG`. Diese Map ist keine eigenständige fachliche Anforderung und nicht Teil der Policy- oder Attributdaten; sie ist ein Implementierungshilfsmittel zur Fehlersuche und unterstützt die in OA-02 geforderte Nachvollziehbarkeit während Entwicklung und Evaluation. Standardmäßig bleibt die Ausgabe deaktiviert.

Für die Aufteilung der Policy-Auswertung wird die nicht gepinnte Map `FILE_OPEN_JUMP_TABLE` mit den Dateideskriptoren der drei nachfolgenden eBPF-Programme befüllt. Die festen Indizes stehen für die Auswertung statischer Policies, die Auswertung attribut- und

zeitabhängiger Policies sowie das Zusammenführen der Teilergebnisse. Diese Programme werden nicht eigenständig an einen Hook angehängt, sondern ausschließlich über Tail Calls aus dem anfänglichen `file_open`-Programm erreicht. Die Ausgestaltung der Kette und ihre Bedeutung für die Verifier-Grenzen wird im Abschnitt zur Implementierung des Kernelspace-PEP erläutert.

Danach öffnet das Programm die gepinnten Maps für Zeitinformationen und Attribute und erzeugt die Policy-Synchronisation, welche die drei gepinnten Policy-Maps verwendet. Bevor der LSM-Hook angehängt wird, müssen die anfänglichen Policies erfolgreich eingelesen, validiert und als vollständige Generation aktiviert sein. Ebenso schreibt der Attributloader den aktuellen Attributbestand sowie einen ersten Zeitstempel. Ein Fehler in einem dieser Schritte beendet den Start. Eine gültige Ausgangskonfiguration darf dabei auch eine leere Policy-Menge enthalten.

5.3.3 Start der Userspace-Komponenten

Erst nachdem die beschriebenen Initialschritte erfolgreich abgeschlossen sind, hängt das Hauptprogramm das als Einstiegspunkt vorgesehene LSM-Programm an den Hook `file_open`. Die nachfolgenden Stufen werden weiterhin nur über die bereits eingerichtete Tail-Call-Tabelle erreicht. Die Beschränkung auf diesen einen aktiven Hook entspricht dem Umfang des Prototyps und vermeidet, dass nicht betrachtete Zugriffsarten in die Durchsetzung einfließen.

Die weitere Laufzeitsteuerung erfolgt asynchron mit dem Rust-Makro `tokio::select!`. Der Zeitaktualisierer, die Verzeichnisbeobachtung für Attribute, die Verzeichnisbeobachtung für Policies und der Userspace-PEP werden dabei innerhalb desselben Tasks gesteuert. Die Zweige können beim asynchronen Warten die Ausführung abgeben; sie werden jedoch nicht als eigenständige parallele Tasks gestartet. Der Userspace-PEP führt die Nachbewertung synchron aus. Während eines laufenden Scans können die übrigen Zweige deshalb keine weiteren Aktualisierungen durchführen. Die beiden Loader aktualisieren dabei ihre jeweiligen gepinnten Maps selbstständig; der Userspace-PEP behandelt bereits bestehende Dateizugriffe und wird in einem späteren Abschnitt beschrieben. Eine Beendigung per `Ctrl-C` wird als reguläres Ende behandelt. Liefert dagegen eine der asynchronen Aufgaben einen Fehler zurück, wird dieser an das Hauptprogramm weitergegeben und der Prozess beendet. Beispiele sind ein fehlgeschlagenes Schreiben des aktuellen Zeitwerts, ein Fehler der Dateisystembeobachtung oder das fehlgeschlagene Öffnen einer vom Userspace-PEP benötigten gepinnten Map. Die Laufzeitsteuerung hält damit Laden, Durchsetzung und Aktualisierung in einem Prozess zusammen, ohne die fachliche Verarbeitung der Policies in den Kernel zu verlagern.

5.3.4 Gemeinsames Veröffentlichungsverfahren

Policy- und Attributloader überwachen nach der Initialisierung ihre jeweiligen Quellverzeichnisse rekursiv. Das folgende Verfahren gilt für `.policy`- und `.attributes`-

Dateien gleichermaßen. Vollständig vorbereitete Dateien werden mit dem Hilfsskript `scripts/publish-config.sh` veröffentlicht. Das Bash-Skript schreibt eine eindeutige temporäre Datei mit einer nicht geladenen Endung im Zielverzeichnis, schließt sie und ersetzt den aktiven Pfad atomar durch `mv` im selben Dateisystem mit Rename-Semantik [Lin26b]. Der Watcher berücksichtigt `IN_MOVED_TO` für aktive Dateiendungen sowie Entfernung und Verschiebung aus dem Verzeichnis; vorbereitende Schreibereignisse lösen keine Synchronisierung aus. Nach einem relevanten Ereignis wird der vollständige Bestand ohne feste Wartezeit erneut gelesen. Neue Unterverzeichnisse werden überwacht und erneut eingelesen; bei einem Ereignisüberlauf erfolgt ein erneuter Abgleich der Watches und des Dateibestands. Der Verlust des überwachten Wurzelverzeichnisses wird als Fehler behandelt. Direktes Überschreiben aktiver Dateien ist nicht unterstützt.

Die atomare Ersetzung veröffentlicht jeweils eine Datei. Sie garantiert weder einen atomaren Verzeichnissnapshot beim anschließenden Einlesen noch eine gemeinsame Transaktion von Policies und Attributen. Vor dem Start kann der anfängliche Dateibestand regulär vorbereitet werden.

5.4 Verarbeitung und Laden von Policies

Die Policy-Dateien bleiben im Userspace als les- und änderbare Quelldokumente erhalten. Der Policyloader liest sie ein, prüft ihre Gültigkeit und übersetzt sie in die festen Map-Werte des zuvor beschriebenen Map-ABI.

5.4.1 Einlesen der Policydateien

Beim Start durchsucht der Policyloader das Verzeichnis `policies/` im aktuellen Arbeitsverzeichnis rekursiv und berücksichtigt ausschließlich Dateien mit der Endung `.policy`. Der Inhalt jeder Datei wird zusammen mit ihrem relativen Pfad als Policy-Dokument erfasst. Eine feste Sortierung der gefundenen Pfade stellt sicher, dass derselbe Dateibestand bei wiederholtem Laden in derselben Reihenfolge verarbeitet wird.

Im laufenden Betrieb verarbeitet der Policyloader die Veröffentlichungen und Entfernungen aus Abschnitt 5.3.4. Der Loader vergleicht den aktuellen Dokumentbestand mit dem zuletzt erfolgreich beziehungsweise zuletzt fehlerhaft verarbeiteten Bestand. Unveränderte Daten werden nicht erneut in die Maps geschrieben; eine unverändert fehlerhafte Bearbeitung wird nicht bei jedem Dateisystemereignis wiederholt protokolliert.

5.4.2 Syntaktische Verarbeitung

Eine `.policy`-Datei enthält genau ein Policy-Dokument. Leerzeilen sowie Zeilen mit `//`- oder `#`-Kommentaren werden beim Einlesen ignoriert. Das erste wirksame Element muss einen quotierten Policy-Namen der Form `policy "..."` enthalten, gefolgt von der Entscheidung `permit` oder `deny`. Die weiteren Anweisungen müssen jeweils mit

einem Semikolon abgeschlossen werden. Der Parser erkennt eine Aktionsbedingung, eine optionale Subject-UID sowie optionale Gleichheitsbedingungen für Kommando und Ressourcenpfad.

Für die dynamischen Bedingungen akzeptiert der Parser die vordefinierte Zeitangabe `environment.time` sowie die UTC-Komponenten Stunde, Minute und Sekunde. Diese werden durch `environment.utc.hour`, `environment.utc.minute` und `environment.utc.second` angesprochen. Darüber hinaus erkennt der Parser Vergleiche von System-, Subject- und Ressourcenattributen mit den Präfixen `system.`, `subject.` und `resource.`. Unterstützt werden die Operatoren <, <=, ==, >= und >; Werte können numerisch, boolesch oder als quotierte Zeichenkette angegeben werden. Nicht zuordenbare Anweisungen, mehrfach definierte Einzelfelder oder unvollständige Ausdrücke führen zu einem Fehler mit Datei- und Zeilenangabe. Damit bleiben Fehler in der Eingabesprache im Userspace sichtbar, statt erst als uneindeutige Map-Daten im Kernel aufzufallen.

5.4.3 Semantische Validierung

Nach dem Parsen validiert der Loader den gesamten Dokumentbestand. Policy-Namen müssen innerhalb einer Generation eindeutig sein; jede Policy benötigt genau eine unterstützte Aktion. Der Prototyp akzeptiert für die Durchsetzung nur `file_open`. Nicht implementierte Aktionen oder aktionsfremde Felder werden zurückgewiesen. Optional nicht angegebene Filter, etwa `subject.uid`, Kommando oder Ressourcenpfad, werden als nicht einschränkende Bedingung behandelt.

Für jede `file_open`-Policy kontrolliert der Loader außerdem die Byte-Länge der Zeichenketten, die in Arrays fester Größe gespeichert werden. Das Kommando darf höchstens 15 Nutzbytes und der Ressourcenpfad höchstens 64 Byte umfassen. Das mit `COMMAND_LEN` definierte Kommandofeld bleibt 16 Byte groß; ein Byte ist für den NUL-Abschluss des Linux-Tasknamens reserviert. Die zulässige Eingabelänge wird getrennt durch `COMMAND_MAX_BYTES` festgelegt. Die Grenzen beziehen sich auf Bytes, nicht auf die Anzahl von Unicode-Zeichen. Längere Eingaben werden vor der Übersetzung zurückgewiesen, statt sie stillschweigend zu kürzen. Das mit `RESOURCE_LEN` definierte Ressourcenfeld bleibt unverändert. Ein angegebener Ressourcenpfad wird beim Laden über die Dateimetadaten in Device- und Inode-Wert aufgelöst. Existiert die Datei nicht oder kann ihre Identität nicht bestimmt werden, wird die Policy nicht aktiviert. Die zur Laufzeit verwendete Ressourcenidentität entspricht damit der für Ressourcenattribute verwendeten Identität, ohne im eBPF-Programm Pfade auflösen zu müssen.

Der Kommando-Filter verwendet den Linux-Tasknamen `comm`, keine manipulationsgeschützte Identität der ausgeführten Programmdatei. Threads eines Prozesses können unterschiedliche Namen besitzen und diese verändern [Lin26a]. Der Kernelspace-PEP liest den Namen der aktuell öffnenden Task, während der Userspace-PEP den Namen des Prozessleaders aus `/proc/<pid>/status` verwendet. Bei unterschiedlichen Threadnamen müssen beide Pfade daher nicht dieselbe Kommando-Bedingung erfüllen. Der Filter eignet sich im Prototyp zur Auswahl entsprechend benannter Tasks; eine zuverlässige Identifika-

tion eines bestimmten Executables oder eine Nachbewertung bei jeder Namensänderung wird damit nicht zugesichert.

Die Begrenzungen der kernelgeeigneten Repräsentation werden ebenfalls vor dem Schreiben geprüft. Pro Policy ist höchstens eine eingebaute Zeitbedingung zulässig. Zusätzlich darf eine Streampolicy höchstens vier dynamische Attributbedingungen enthalten. Zeichenkettenattribute sind auf den Gleichheitsvergleich == beschränkt, weil ihr Wert als Hash vorliegt. Schließlich darf jede der getrennten Mengen statischer Policies und Streampolicies die Größe einer Policy-Bank nicht überschreiten. Solche Grenzen werden damit früh und nachvollziehbar im Userspace durchgesetzt, statt den begrenzten Kontrollfluss des eBPF-Programms zu erweitern.

5.4.4 Übersetzung in Map-Einträge

Aus einer validierten Policy erzeugt der Loader die Map-Werte aus Abschnitt 5.2. Policies ohne Zeit- oder Attributbedingungen werden als `FileOpenStaticPolicy` eingeordnet, alle anderen als `FileOpenStreamPolicy`. Er überträgt die Filter und die zuvor aufgelöste Dateiidentität in deren feste Felder.

Für dynamische Bedingungen übernimmt er Namespace, Vergleichsoperator und typisierten Wert. Namen und Zeichenkettenwerte werden mit derselben Hashfunktion wie beim Attributloader übersetzt. So entstehen auf beiden Seiten übereinstimmende Schlüssel; deren Verwendung beim Matching beschreibt Abschnitt 5.6.

5.4.5 Generationenwechsel und Rollback

Die Policy-Maps enthalten jeweils zwei gleich große Bänke. Die aktive Generation in `POLICY_GENERATION[0]` bestimmt, aus welcher Bank das eBPF-Programm die statischen Policies und Streampolicies liest. Beim Laden einer neuen, vollständig validierten Menge erhöht der Loader die Generation und bestimmt daraus die bisher inaktive Bank. Er schreibt zuerst alle neuen Einträge in diese Bank und füllt die verbleibenden Plätze mit deaktivierten Policies. Die neu aktivierte Bank enthält dadurch keine Einträge entfernter Policy-Dateien.

Erst wenn beide Policy-Maps erfolgreich gefüllt wurden, schreibt der Loader die neue Generationsnummer in `POLICY_GENERATION[0]`. Dieser einzelne abschließende Schritt schaltet die neue Bank sichtbar. Schlägt vorher ein Schreibvorgang fehl, bleibt der Generationenwert unverändert; die unvollständig vorbereitete Generation wird nicht aktiviert. Diese Aktivierungsregel garantiert jedoch keine unveränderten Map-Inhalte für eine Kernel-Auswertung, deren zuvor ausgewählte Bank bei einer weiteren Aktualisierung wiederverwendet wird. Bei einem Updatefehler vor der Aktivierung protokolliert der Loader den Fehler und belässt die zuletzt erfolgreiche Generation aktiv. Die Bezeichnung Rollback meint hier das Beibehalten der bisherigen Generationsnummer; sie bezeichnet weder das Wiederherstellen bereits geschriebener Daten in der inaktiven Bank noch das Zurück-

nehmen einer erfolgreichen Aktivierung. Bei der ersten Initialisierung ist derselbe Fehler dagegen ein Startfehler, da noch keine vertrauenswürdige Generation für die Aktivierung des Hooks vorliegt.

5.4.6 Beispiel der Policy- und Attributsemantik

Das folgende Beispiel zeigt die Wirkung einer einzelnen Streampolicy. Die reguläre Datei `/tmp/protected.txt` muss vor dem Laden der Policy existieren, damit ihre Device- und Inode-Werte aufgelöst werden können. Das vollständige Dokument in `policies/example.policy` lautet:

```
policy "deny low clearance"
deny
    action == "file_open";
    resource.path == "/tmp/protected.txt";
    system.clearance <= 2;
```

Die zugehörige Datei `attributes/system.attributes` enthält beispielsweise:

```
clearance = 2
```

`clearance` ist ein frei benanntes numerisches Systemattribut. Bei einem Wert kleiner oder gleich zwei greift die Deny-Policy für eine neue Öffnung der angegebenen Datei. Die Entscheidungen für andere Werte, fehlende Attribute und Typabweichungen zeigt Tabelle 1 in Abschnitt 4.5.8. Änderungen werden über das Verfahren aus Abschnitt 5.3.4 veröffentlicht. Ein daraus folgender Entzug bestehender FDs unterliegt den Best-Effort-Grenzen des Userspace-PEP.

5.5 Bereitstellung dynamischer Attribute

Die dynamischen Attribute werden in einem von Policies getrennten Verzeichnis verwaltet und besitzen eine eigene Generation. Dadurch lassen sich beispielsweise `system.defcon` oder `subject.position` aktualisieren, ohne Policy-Dateien erneut zu verarbeiten. Wie bei den Policies bleiben die Quelldateien im Userspace lesbar; nur ihre validierte, feste Repräsentation wird in die gepinnten Maps übertragen.

5.5.1 Aufbau des Attributverzeichnisses

Der Attributloader verwendet das Verzeichnis `attributes/` im aktuellen Arbeitsverzeichnis. Beim Start legt er dieses sowie die Unterverzeichnisse `subjects/` und `resources/` bei Bedarf an. Die Datei `attributes/system.attributes` enthält systemweite Attribute. Fehlt sie bei der Initialisierung, wird eine Datei mit dem Standardwert

`defcon = 5` angelegt. Damit besitzt der Prototyp auch ohne manuelle Vorbelegung einen wohldefinierten Systemzustand.

Subject-Attribute liegen in Dateien der Form `attributes/subjects/<uid>.attributes`; der Dateiname bestimmt die zugehörige numerische UID. Ressourcenattribute werden rekursiv unter `attributes/resources/` abgelegt. Der relative Pfad ohne die Endung `.attributes` wird als absoluter Ressourcenpfad interpretiert. Beispielsweise beschreibt `attributes/resources/home/hntr/test.txt.attributes` die Datei `/home/hntr/test.txt`. Der Loader löst diesen Pfad beim Einlesen in Device- und Inode-Wert auf. Der Pfad ist damit die benutzerfreundliche Zuordnung in der Dateistruktur, während die Map den zuvor beschriebenen stabileren Objektidentifikator verwendet.

5.5.2 Parsen und Validieren der Attribute

Jede wirksame Zeile einer `.attributes`-Datei hat die Form `<attribut> = <wert>`. Leere Zeilen und Inhalte hinter # sowie Leerzeichen werden ignoriert. Der Wert kann eine vorzeichenlose Zahl, `true` oder `false` oder eine quotierte Zeichenkette sein. Die Typisierung wird beim Einlesen festgelegt und später bei der Policy-Auswertung berücksichtigt. Eine nicht geschlossene Zeichenkette, ein fehlendes Gleichheitszeichen oder ein nicht interpretierbarer Wert verhindert die Übernahme der betreffenden Gesamtkonfiguration.

Attributnamen dürfen aus ASCII-Buchstaben, Ziffern, Bindestrichen und Unterstrichen bestehen. Beispiele sind `position` und `clearance-level`. Die Namen können frei gewählt werden, ohne dass der Kernelspace-PEP sie vorab kennen muss. Ein Punkt ist nicht Teil des Namens, sondern trennt in einer Policy den Namespace vom Attributnamen, etwa in `subject.position`. Der Sonderfall `system.defcon` wird zusätzlich als numerischer Wert im Bereich eins bis fünf geprüft. Nach dem Einlesen sortiert der Loader die Attribute deterministisch und weist doppelte Definitionen desselben Namens für dasselbe Objekt zurück.

Auch die Zuordnung der Dateien wird validiert. Subject-Dateien müssen einen numerischen Dateinamen vor der Endung besitzen. Jede Ressourcendatei muss auf eine beim Laden vorhandene Datei verweisen, da nur so ihre Device- und Inode-Werte bestimmt werden können. Fehler bei einer laufenden Aktualisierung werden protokolliert und der bestehende Attributbestand bleibt aktiv. Während der Initialisierung bewirken sie dagegen einen Startabbruch, damit der LSM-Hook nicht mit einem nicht validierten Attributzustand aktiviert wird.

5.5.3 Erzeugung der Attribut-Map-Einträge

Der Loader setzt die validierten Attribute in `AttributeKey` und `AttributeValue` aus Abschnitt 5.2 um. Er belegt die Objektidentifikatoren entsprechend dem Namespace und hasht Namen sowie Zeichenkettenwerte mit derselben Funktion wie der Policyloader. Für `subject.position` entsteht damit bei derselben UID derselbe Schlüssel wie bei

der späteren Bedingungsauswertung. Die lesbare Darstellung dieser nicht umkehrbaren Hashwerte behandelt Abschnitt 5.8.

5.5.4 Aktualisierung der Attributgeneration

Nach der anfänglichen Übernahme verarbeitet der Attributloader Veröffentlichungen und Entfernungen nach Abschnitt 5.3.4. Er schreibt nur einen gegenüber dem zuletzt angewendeten Bestand veränderten Attributsatz. Die Attribute besitzen zwei Bänke innerhalb der `ATTRIBUTES`-Hash-Map; `ATTRIBUTE_GENERATION[0]` wählt die aktive Bank aus.

Der anschließende Generationenwechsel entspricht dem Vorgehen des Policyloaders aus Abschnitt 5.4.5. Für eine Aktualisierung löscht der Attributloader zunächst alle Einträge der inaktiven Bank. Danach schreibt er die vollständige Menge der neu eingelesenen Attribute mit dem zugehörigen Bankwert in den Schlüssel. Erst nach erfolgreichen Schreibvorgängen wird `ATTRIBUTE_GENERATION[0]` auf die nächste Generation gesetzt. Fehler während des Löschens oder Schreibens lassen die bisherige Generation unverändert aktiv; die unvollständig vorbereitete Attributgeneration wird nicht aktiviert. Für die Lesesicht einer laufenden Kernel-Auswertung gilt dieselbe Einschränkung bei der Wiederverwendung einer Bank wie bei den Policy-Generationen. Das Verfahren erlaubt auch das Entfernen einer Attributdatei, weil die neue vollständige Bank dann keinen entsprechenden Eintrag mehr enthält.

5.5.5 Bereitstellung der Zeitinformation

Zeitabhängige Bedingungen beziehen ihre Werte nicht aus einer `.attributes`-Datei. Stattdessen schreibt ein eigener Aktualisierer den aktuellen Unix-Zeitstempel. Der Wert liegt in `CURRENT_TIME[0]`. Er wird vor dem Anheften des Hooks gesetzt und während der Laufzeit in einem Intervall von einer Sekunde erneuert. Das eBPF-Programm kann daraus die in der Policy-Sprache vorgesehenen Zeit-, Stunden-, Minuten- und Sekundenbedingungen ableiten, ohne selbst auf die Systemzeit oder externe Schnittstellen zugreifen zu müssen.

Die Trennung von Zeitinformation und frei definierbaren Attributen vermeidet eine Sonderbehandlung in den Attributdateien. Die Zeit bleibt ein vom Laufzeitsystem bereitgestellter, regelmäßig aktualisierter Eingabewert, während `system.attributes`, Subject- und Ressourcenattribute durch den Nutzer verwaltete Konfigurationsdaten darstellen.

5.6 Implementierung des Kernelspace-PEP

Der Kernelspace-PEP entscheidet über neue Öffnungsvorgänge unmittelbar im LSM-Hook `file_open`. Die Bezeichnung als PEP hebt hervor, dass der Hook die kontrollierte Operation erfasst und das Ergebnis über seinen Rückgabewert durchsetzt. Die Auswertung der anwendbaren Policies und das Combining bilden zugleich eine integrierte PDP-Funktion;

ein separater PDP wird für den Kernelpfad nicht aufgerufen. Seine technische Aufgabe ist auf das Ermitteln der für die Policy relevanten Anfragewerte, das Auswerten der aktiven Map-Banken und die Rückgabe eines LSM-Ergebnisses begrenzt.

5.6.1 Verarbeitung des file_open-Hooks

Die Definition des LSM-Hooks `file_open` deklariert als Hookargument einen Zeiger auf das zu öffnende Kernelobjekt `struct file` [Lin25b]. Die UID wird davon unabhängig über Ayas eBPF-Kontext ermittelt. Als Subjektidentität legt der Prototyp ausdrücklich die Real-UID des aufrufenden Prozesses fest. Ayas `ctx.uid()` gibt die unteren 32 Bit des Ergebnisses von `bpf_get_current_uid_gid()` zurück [Aya26b]. Der Kernelhelper ermittelt dafür mit `current_uid_gid()` die Real-UID und Real-GID des aktuellen Tasks und übersetzt beide Werte in den initialen User-Namespace [Lin25a; Ker26n]. Effektive, Saved-Set- und Filesystem-UID sind damit keine Eingaben der Policyentscheidung. BPF-LSM ergänzt die Hook-Argumente um den Rückgabewert des zuvor ausgeführten BPF-LSM-Programms oder um `0` beim ersten Programm [Ker26q]. Die Verkettung allein bewahrt eine frühere Ablehnung nicht automatisch, wenn ein nachfolgendes Programm den Wert ignoriert. Das Einstiegspunktprogramm prüft ihn deshalb explizit und gibt einen Wert ungleich `0` unverändert zurück.

Für die eigene Auswertung liest der Kernelspace-PEP die UID des aufrufenden Prozesses und dessen Kommando. Die Ressourcenidentität wird nicht über einen Pfad bestimmt. Stattdessen liest eine Hilfsfunktion kontrolliert den Inode-Zeiger aus dem `file`-Objekt und daraus Device- sowie Inode-Wert. Diese Werte entsprechen der beim Policy- und Attributladen erzeugten Ressourcenidentität. Schlägt eine dieser Kernel-Leseoperationen fehl oder fehlt eine benötigte Information, gibt das Programm eine Ablehnung zurück. Ein fehlgeschlagener Zugriff auf Kernel-Daten kann damit nicht versehentlich einen unbeschränkten Zugriff erzeugen.

Das Einstiegspunktprogramm wertet selbst keine Policies aus. Es ruft die erste Auswertungsstufe über einen Tail Call auf. Ist der Eintrag in der Sprungtabelle nicht vorhanden oder schlägt der Tail Call fehl, wird ebenfalls abgelehnt. Diese Behandlung ist technisch von der fachlichen Standardentscheidung zu unterscheiden: Bei funktionierender Auswertung und keiner passenden Deny-Policy ist das Ergebnis erlaubt; ein unvollständiger Durchsetzungspfad führt dagegen zu einer Ablehnung.

5.6.2 Tail-Call-Kette

Die Auswertung ist in drei LSM-Programme aufgeteilt: die Prüfung statischer `file_open`-Policies, die Prüfung zeit- und attributabhängiger Streampolicies sowie die abschließende Kombination. Das Hauptprogramm trägt ihre Dateideskriptoren beim Start an festen Indizes in die nicht gepinnte `FILE_OPEN_JUMP_TABLE` ein. Der Ablauf lautet daher `file_open` → Auswertung statische Policies → Auswertung Streampolicies → Kombination.

Zwischen den Stufen liegt kein dynamischer Heap-Speicher. Der aktuelle Entscheidungszustand wird in der Per-CPU-Array-Map `DECISIONS` des Aya-Typs `PerCpuArray` abgelegt. Sie enthält getrennte Markierungen dafür, ob mindestens eine anwendbare Policy als Permit beziehungsweise als Deny ausgewertet wurde, sowie die für den Zugriff verwendete Policy-Generation. Da ein Tail Call im Kontext derselben CPU fortgesetzt wird, kann die folgende Stufe diesen Zustand lesen und erweitern. Die Map ist nicht gepinnt, weil ihr Inhalt nur während eines einzelnen Hook-Aufrufs benötigt wird.

Die Aufteilung reduziert die Größe und die Anzahl der durch den Verifier gleichzeitig zu betrachtenden Kontrollpfade pro Programm. Jede Stufe besitzt nur eine begrenzte Schleife über die Einträge einer Policy-Bank beziehungsweise über die höchstens vier Attributbedingungen. Zugleich entsteht eine klare Fehlerbehandlung an den Stufengrenzen: Kann eine Stufe ihren Zustand nicht lesen oder schreiben oder fehlt das nächste Tail-Call-Ziel, beendet sie die Kette mit einer Ablehnung. Die Tail Calls sind damit nicht nur eine Optimierung der Programmstruktur, sondern Teil der Durchsetzungslogik.

5.6.3 Auswertung statischer Policies

Die statische Stufe liest `POLICY_GENERATION[0]` und leitet daraus den Offset der aktiven Bank ab. Sie erstellt aus UID, Kommando sowie Device- und Inode-Wert eine `FileOpenRequest`. Anschließend durchläuft sie die feste Anzahl von Einträgen der aktiven Bank von `FILE_OPEN_STATIC_POLICIES`. Deaktivierte Einträge und Policies für andere Aktionen werden übersprungen.

Eine aktive Policy passt nur, wenn ihr Subject-Filter, ihr Kommando-Filter und ihre Ressourcenidentität mit der Anfrage übereinstimmen. Nicht belegte Filter wirken als Wildcards. Bei einem Treffer speichert die Stufe getrennt, ob eine Permit- oder Deny-Entscheidung vorliegt. Sobald beide Entscheidungsarten vorliegen, kann sie die verbleibenden Einträge dieser Stufe überspringen, weil die nachfolgende Combining-Regel dadurch nicht mehr verändert werden kann. Der Entscheidungszustand wird in `DECISIONS` geschrieben und anschließend an die Stream-Stufe übergeben.

5.6.4 Auswertung von Streampolicies

Die zweite Stufe übernimmt zunächst die von der statischen Stufe verwendete Policy-Generation. Damit verwenden beide Policy-Mengen dieselbe aktive Bank. Zusätzlich liest sie den aktuellen Zeitstempel aus `CURRENT_TIME` sowie `ATTRIBUTE_GENERATION` und bestimmt daraus die aktive Attributbank. Die Umrechnung des Unix-Zeitstempels in Stunde, Minute und Sekunde erfolgt mit der gemeinsamen Logik aus `tails-pdp-common`.

Für jeden Eintrag der aktiven Streampolicy-Bank werden zuerst dieselben statischen Filter wie in der ersten Stufe geprüft. Danach müssen alle hinterlegten Attributbedingungen erfüllt sein. Für jede Bedingung bildet der Kernelspace-PEP aus Namespace, aktueller UID beziehungsweise Ressourcenidentität, Hash und Attributbank einen `AttributeKey`.

Der zugehörige Wert wird aus `ATTRIBUTES` gelesen und mit Typ und Vergleichsoperator der Bedingung verglichen. Fehlt ein Attribut oder stimmt sein Typ oder Wert nicht überein, ist die gesamte Attributbedingung und damit die Policy nicht erfüllt. Eine optionale Zeitbedingung wird anschließend gegen den bereitgestellten Zeitwert geprüft.

Ein passender Stream-Eintrag ergänzt den Entscheidungszustand um Permit oder Deny. Sobald beide Markierungen gesetzt sind, kann auch diese Stufe beendet werden. Der resultierende Zustand wird anschließend über die dritte Tail-Call-Position an die Kombinationsstufe weitergegeben. Die frei wählbaren Attributnamen haben dabei keinen Einfluss auf den Ablauf, da im Kernel ausschließlich fest definierte Schlüsselstrukturen und die zugehörigen Hashwerte verglichen werden.

5.6.5 Combining und Durchsetzung

Die letzte Stufe liest den in `DECISIONS` gespeicherten Entscheidungszustand und wendet die im Prototyp festgelegte Combining-Regel *deny-overrides* an. Hat mindestens eine passende Policy eine Ablehnung geliefert, gibt sie den LSM-Fehlerwert `-EPERM` zurück. `EPERM` ist der symbolische Linux-Fehlercode für „Operation nicht erlaubt“; BPF-LSM verwendet den negativen Fehlerwert zur Ablehnung [Ker26q; Ker26d]. Andernfalls liefert die Stufe den LSM-Erfolgswert `0`. Eine Permit-Policy kann damit eine passende Deny-Policy nicht überstimmen; ohne passende Deny-Policy bleibt der Vorgang erlaubt.

Der Rückgabewert gilt unmittelbar für die neue `file_open`-Operation. Bei einer Ablehnung erhält der aufrufende Prozess daher keine erfolgreiche Öffnung und folglich keinen neuen FD. Die Auswertung betrifft ausschließlich den aktuellen Öffnungsvorgang. Bereits zuvor geöffnete Deskriptoren kann der Hook nicht rückwirkend entziehen; diese Lücke adressiert der folgende Userspace-PEP.

5.7 Implementierung des Userspace-PEP

Der Userspace-PEP ergänzt den Kernelspace-PEP für den Fall, dass sich Policies oder Attribute ändern, während ein Prozess eine Datei bereits geöffnet hält. Er ist kein zweiter LSM-Hook und kann einen historischen Öffnungsvorgang nicht nachträglich im Kernel blockieren. Stattdessen bewertet er bestehende Dateideskriptoren anhand derselben gepinnten Maps erneut und versucht, konkret festgestellte Verstöße am betroffenen Prozess durchzusetzen. Auch hier bezeichnet PEP primär die Durchsetzungsfunktion; Policy-Matching, Attributauswertung und Combining bilden eine in dieselbe Komponente integrierte PDP-Funktion.

5.7.1 Ermittlung bestehender Dateizugriffe

Beim Start öffnet der Userspace-PEP die gepinnten Policy-, Attribut- und Zeit-Maps und wartet anschließend auf interne Aktivierungsereignisse. Policy- und Attributloader senden

ein solches Ereignis ausschließlich nach dem vollständigen Schreiben der inaktiven Bank und der erfolgreichen Aktivierung der neuen Generation. Rohereignisse der Verzeichnisbeobachtung bleiben auf die Loader beschränkt. Zusätzlich bestimmt der Userspace-PEP aus der aktiven Streampolicy-Bank den nächsten Zeitpunkt, an dem eine Zeitbedingung ihren Wahrheitswert ändern kann. Nur eine erfolgreiche Aktivierung oder eine solche Zeitgrenze löst einen `/proc`-Durchlauf aus. Für jeden numerischen Prozesseintrag liest er aus `/proc/<pid>/status` das Kommando und das erste der vier UID-Felder. Dieses Feld bezeichnet die Real-UID; die folgenden Felder enthalten effektive, Saved-Set- und Filesystem-UID [Ker26i]. Der Userspace-Pfad verwendet somit dieselbe definierte Subjektidentität wie der Kernelspace-Pfad. Anschließend untersucht er die numerischen Einträge in `/proc/<pid>/fd`. Nur Deskriptoren, deren Ziel als reguläre Datei identifiziert werden kann, werden berücksichtigt; Verzeichnisse und nicht auflösbare Einträge bleiben unberücksichtigt.

Die dabei gefundenen FDs gelten als technische Repräsentationen fortdauernder Nutzungssituationen. Sie werden nicht als Subscriptions registriert oder dauerhaft verwaltet. Ein Aktivierungsereignis ist ebenfalls keine individuelle Aktualisierung eines Decision Streams, sondern löst eine erneute Rekonstruktion und lokale Bewertung der zu diesem Zeitpunkt über `/proc` sichtbaren Nutzungssituationen aus.

Für jeden berücksichtigten FD ermittelt der Userspace-PEP wieder Device- und Inode-Wert der geöffneten Datei. Die Umrechnung des Device-Werts entspricht der beim Policyloader und im Kernelspace-PEP verwendeten Darstellung. Zu Beginn eines Scans liest der Userspace-PEP die Nummer der aktiven Policy-Generation, die Nummer der aktiven Attributgeneration und den aktuellen Zeitwert jeweils einmal. Aus der Policy-Generation berechnet er den Offset einer der beiden Policy-Bänke; aus der Attributgeneration bestimmt er eine der beiden Attributbänke. Diese Auswahlwerte werden im `ScanContext` gespeichert und für alle in diesem Durchlauf betrachteten Deskriptoren wiederverwendet. Während des synchronen Scans können die Loader derselben Runtime keine neue Generation aktivieren. Änderungen an den Quelldateien können weiterhin auftreten, werden von diesen Loadern aber erst nach Rückkehr des Scans verarbeitet. Der Scan ist dennoch keine atomare Momentaufnahme des gesamten Systems: Prozesse und Deskriptoren können sich parallel ändern.

Der Trigger-Kanal ist auf einen ausstehenden Eintrag begrenzt; mehrere Aktivierungsereignisse können dadurch zusammengefasst werden. Der Userspace-PEP führt innerhalb einer Runtime jeweils nur einen `/proc`-Scan aus. Zeitaktualisierung und Scan erfüllen unterschiedliche Aufgaben: Der Zeitaktualisierer soll `CURRENT_TIME` sekündlich aktualisieren, löst damit aber selbst keine Nachbewertung aus. Während eines synchronen Scans ist auch dieser Aktualisierer blockiert. Neue Kernelzugriffe werden weiterhin anhand der zuletzt bereitgestellten Map-Werte bewertet. Die Dauer eines Scans kann daher die Übernahme von Policy- und Attributänderungen sowie die Aktualisierung der Zeitinformationen verzögern. Dies betrifft insbesondere das blockierende Warten mit `waitpid` beim `ptrace`-Entzug, für das die Implementierung keine explizite Zeitbegrenzung vorsieht. Zusätzlich können Prozesse, Dateien oder Deskriptornummern zwischen dem Lesen von `/proc` und der Durchsetzung ihren Zustand verändern. Ereignisgetriggerte Auslösung

bedeutet daher weder einen atomaren noch einen garantierten Entzug und ist kein gleichwertiger Ersatz für den unmittelbaren LSM-Hook.

5.7.2 Erneute Policy-Auswertung

Für jeden gefundenen Deskriptor erzeugt der Userspace-PEP dieselbe `FileOpenRequest`-Repräsentation wie der Kernelspace-PEP. Er wertet die statische und die Streampolicy-Bank mit den gemeinsamen Funktionen aus `tails-pdp-common` aus. Bei Streampolicies bildet er die Attributschlüssel mit denselben Objektidentifikatoren und Namenshashwerten wie das eBPF-Programm. Dadurch verwendet die nachträgliche Prüfung dieselben fachlichen Bedingungen und die gleiche *deny-overrides*-Semantik wie die Prüfung eines neuen Öffnungsvorgangs.

Für die nachträgliche Durchsetzung sind ausschließlich passende Deny-Policies relevant. Permit-Policies lösen keinen Entzug aus; eine passende Deny-Policy erzeugt einen Verstoß mit Policy-Art, Policy-Index, Prozess-ID und File-Descriptor-Nummer. Mehrere passende Policies für denselben Deskriptor führen nicht zu mehreren Entzugsversuchen innerhalb eines Scans. Die Deduplizierung erfolgt über das Tupel aus Prozess-ID und Deskriptor. Unterschiedliche Deskriptoren eines Prozesses bleiben dagegen unabhängig: Hält ein Prozess zwei Dateien geöffnet und verletzt nur eine davon eine Deny-Policy, wird nur deren Deskriptor als Verstoß behandelt.

Kann ein vollständiger Scan nicht ausgeführt werden, etwa weil eine Generation oder ein Map-Eintrag nicht gelesen werden kann, protokolliert der Userspace-PEP den Fehler und setzt den nächsten Scan fort. Ein Fehler beim Entzug eines einzelnen Deskriptors beendet ebenfalls nicht die Behandlung der übrigen Verstöße. Er löst keine eigenständige Wiederholung aus: Erst eine weitere erfolgreiche Aktivierung oder relevante Zeitgrenze veranlasst einen neuen Scan. Ohne ein solches Ereignis kann ein zuvor nicht entziehbarer FD geöffnet bleiben, auch wenn der Grund des Fehlversuchs inzwischen entfallen ist. Dies verhindert, dass ein nicht zugänglicher Prozess den gesamten Überwachungszyklus blockiert, ersetzt jedoch keine atomare Momentaufnahme des gesamten Systems.

5.7.3 Entzug des betroffenen File Descriptors

Der konkrete Entzug ist für das in dieser Arbeit festgelegte Zielsystem `x86_64`-Linux implementiert. Der Userspace-PEP hängt sich mit `PTRACE_ATTACH` an den Zielprozess an, wartet auf dessen Stopp und sichert Register sowie das Maschinenwort an der aktuellen Instruktionsadresse. Anschließend setzt er die Register für den Systemaufruf `close(fd)`, führt diesen durch einen einzelnen Ausführungsschritt aus und versucht danach, Instruktion und Register wiederherzustellen. Beim Verlassen des Hilfsobjekts wird der Prozess wieder von `ptrace` gelöst.

Der Entzug adressiert die konkrete File-Descriptor-Nummer, nicht pauschal die Datei oder den gesamten Prozess. Negative Deskriptoren sowie die Prozess-IDs `0`, `1` und die eigene

Prozess-ID werden vor dem Anheften abgewiesen. Dadurch schützt die Implementierung zentrale Prozesse und vermeidet Selbstbeeinflussung. Die Prüfung des selektiven Entzugs anhand zweier geöffneter Dateien behandelt Kapitel 6.

Der Mechanismus benötigt weitreichende Rechte und kann den Zielprozess kurz anhalten. Seine registerbasierte Ausführung von `close(fd)` ist architekturspezifisch und wird im Prototyp ausschließlich für `x86_64`-Linux betrachtet. Der Kernelspace-PEP bleibt die maßgebliche, unmittelbare Durchsetzung für neue Öffnungsvorgänge.

5.8 Implementierung des Administrationstools

Das Administrationstool dient ausschließlich der Beobachtung des geladenen Zustands. Es ersetzt weder den Policyloader noch schreibt es Policies oder Attribute in Maps. Seine getrennte Ausführung ist durch die gepinnten Maps möglich: Das Tool öffnet deren Pfade im BPF-Dateisystem und liest die Daten unabhängig vom laufenden Hauptprozess aus.

5.8.1 Auswahl und Ausgabe des aktiven Zustands

Zu Beginn liest das Tool `POLICY_GENERATION[0]` und `ATTRIBUTE_GENERATION[0]` und bestimmt daraus die aktiven Policy- und Attributbänke. Die Ausgabebefehle lesen damit den gegenwärtig für Entscheidungen verwendeten Map-Zustand. Sie können Policies und Attribute gemeinsam oder getrennt anzeigen; eine kompakte Variante blendet deaktivierte Policy-Plätze aus. Für Policies erscheinen die entscheidungsrelevanten Filter und Bedingungen, für Attribute Namespace, Objektidentität, Name und Wert. Die Ausgabe bildet somit die geladenen Map-Einträge ab und nicht lediglich den aktuellen Inhalt der Quelldateien. Bei parallelen Aktualisierungen ist die Ausgabe jedoch kein atomarer Snapshot: Die beiden Generationen werden unabhängig gelesen, und eine ausgewählte Bank kann während der Ausgabe erneut verwendet werden.

5.8.2 Darstellung nicht umkehrbarer Hashwerte

Attributnamen und Zeichenkettenwerte liegen in den Maps nur als 128-Bit-Hash vor. Das Tool kann daraus keinen Originaltext berechnen. Es erstellt deshalb beim Start ein Hilfswörterbuch, indem es die Dateien aus `policies/` sowie `attributes/` liest und daraus gesammelte Namen und Zeichenketten mit derselben Hashfunktion abbildet. Bei einem Treffer kann es beispielsweise `subject.position == ëngineer¨` lesbar ausgeben.

Dieses Wörterbuch ist ausschließlich eine Anzeigehilfe und durchläuft keine vollständige Loader-Validierung. Es bestätigt auch nicht, dass die Quelldateien zur aktiven Generation gehören. Die dargestellten Entscheidungsdaten stammen aus den gepinnten Maps; nur ihre Textbezeichnungen werden aus dem Wörterbuch ergänzt. Findet sich zu einem Hashwert kein Text im Wörterbuch, zeigt das Tool ihn hexadezimal an. Ein abweichender Quelldateibestand führt daher nicht zwangsläufig zu einer hexadezimalen Ausgabe, solange noch

ein passender Wörterbucheintrag vorhanden ist.

5.8.3 Read-only-Schnittstelle und Fehlerverhalten

Die Kommandozeilenschnittstelle (CLI) akzeptiert bei Bedarf abweichende Pfade für jede gepinnte Map sowie für die beiden Quellverzeichnisse. Ihre Map-Zugriffe beschränken sich auf Öffnen, Iterieren und Lesen; die Implementierung enthält keinen Pfad zum Setzen oder Entfernen von Map-Einträgen. Ungültige Kommandozeilenargumente, nicht erreichbare Maps oder nicht lesbare Hilfsdateien führen zu einer Fehlermeldung und verändern den laufenden Policy-Zustand nicht. Ob das Tool ohne `sudo` ausgeführt werden kann, hängt von den Berechtigungen des BPF-Dateisystems und der gepinnten Maps auf dem Zielsystem ab.

5.9 Fehlerbehandlung und Konsistenzsicherung

Die ausführlichen Abläufe stehen bei der Initialisierung, den Loadern und den Durchsetzungspfaden. Ihre Fehlerreaktionen lassen sich wie folgt zusammenfassen:

Startfehler: Inkompatible Maps, fehlende Kernelvoraussetzungen sowie Fehler beim Laden, Initialisieren oder Anheften brechen den Start ab (Abschnitt 5.3).

Updatefehler vor der Aktivierung: Fehler beim Einlesen, Parsen, Validieren oder Schreiben werden protokolliert; die bisherige Generationsnummer bleibt bestehen. Es werden weder geschriebene Daten in der inaktiven Bank zurückgesetzt noch erfolgreiche Aktivierungen rückgängig gemacht (Abschnitte 5.4.5 und 5.5).

Technische Fehler im Kernelpfad: Fehlende Kontrollwerte, nicht lesbare Kernelobjekte, nicht schreibbare Entscheidungswerte und eine unvollständige Tail-Call-Kette führen in den vorgesehenen Fehlerpfaden zur Ablehnung. Eine frühere BPF-LSM-Ablehnung wird erhalten (Abschnitt 5.6). Dies ist vom fachlichen Default-Allow bei fehlenden oder typabweichenden Attributen zu unterscheiden (Abschnitt 4.5.8).

Scan- und Entzugsfehler: Der Userspace-PEP protokolliert Fehler und behandelt weitere Verstöße. Gescheiterte Entzüge werden erst bei einem neuen auslösenden Ereignis erneut bewertet (Abschnitt 5.7).

Fatale Laufzeitfehler: Weitergegebene Fehler der Verzeichnisbeobachtung, der Zeitaktualisierung oder eines geschlossenen Enforcement-Trigger-Kanals können die Runtime beenden. Ein Benachrichtigungsfehler nach erfolgreicher Aktivierung nimmt diese nicht zurück. Policy-Lesefehler während eines Updates werden dagegen lokal behandelt.

Diese Reaktionen setzen die Grenzen des Generationenverfahrens nicht außer Kraft: Policy- und Attributgeneration werden unabhängig aktiviert, und die Wiederverwendung

einer Bank kann die Lesesicht einer noch laufenden Kernel-Auswertung verändern (Abschnitt 5.4.5).

Der Schutz besteht nur während der Laufzeit des Hauptprogramms. Es hält den von Aya verwalteten LSM-Link, pinnt ihn jedoch nicht. Beim Prozessende wird das Attachment freigegeben; Kernelspace-Prüfung und Userspace-Nachbewertung entfallen. Gepinnte Maps können bestehen bleiben, erhalten aber das Attachment nicht. Die lokale Fail-closed-Behandlung im Kernel ist daher keine ausfallsichere Fortsetzung nach dem Prozessende. Reguläre Linux-Dateiberechtigungen und andere LSM bleiben davon unberührt.

Das Administrationstool verändert bei Fehlern keinen Map-Zustand (Abschnitt 5.8). Optionales eBPF-Debug-Logging ist standardmäßig deaktiviert; bei Aktivierung können UID, Kommando und Ressourcenidentitäten in Kernel-Trace-Ausgaben erscheinen und müssen auf Mehrbenutzersystemen entsprechend geschützt werden.

5.10 Build und Deployment auf dem Zielsystem

Das Zielsystem des Prototyps ist ein `x86_64`-Linux-System mit Linux-Kernel 6.16.12. Der Kernel stellt BPF-LSM, BTF unter `/sys/kernel/btf/vmlinux` und ein eingehängtes BPF-Dateisystem bereit. Andere Prozessorarchitekturen und ältere Kernelversionen liegen außerhalb des betrachteten Implementierungsumfangs. Der Build erzeugt einerseits die Userspace-Programme für dieses Zielsystem und andererseits das eBPF-Objekt, das der Kernel nach erfolgreicher Verifier-Prüfung ausführt. Das Repository enthält dafür Cargo-Konfigurationen sowie ein Startskript für den Zielbetrieb.

5.10.1 Build-Prozess und Build-Artefakte

Die Crate `tails-pdp-ebpf` ist in `tails-pdp/Cargo.toml` als Build-Abhängigkeit eingetragen, damit Cargo Änderungen beim Neubau berücksichtigt. `tails-pdp/build.rs` ruft `aya-build` auf und erzeugt das eBPF-Objekt in einem eigenen Build-Schritt. Eine direkte Cargo-Artefaktabhängigkeit wird nicht verwendet, weil die dafür benötigte Manifestfunktion in der eingesetzten stabilen Cargo-Version nicht verfügbar ist. Das resultierende eBPF-Objekt wird anschließend als ausgerichtete Binärdatei in das Userspace-Hauptprogramm eingebettet. Für den `no_std`-eBPF-Teil setzt das Build-Skript die Panic-Strategie auf `abort`; die eBPF-VM unterstützt weder Panic-Unwinding noch eine Abort-Instruktion [Aya26a]. Das eBPF-spezifische Build-Skript verfolgt zusätzlich den verwendeten `bpf-linker`, damit eine Änderung des Linkers einen Neubau auslöst.

Die Cargo-Konfiguration enthält für das Zielsystem das Rust-Zieltripel `x86_64-unknown-linux-musl`. Die Aliasse `check-linux-x86_64` und `build-linux-x86_64` prüfen beziehungsweise bauen `tails-pdp` und `tails-pdp-admintool` für dieses Ziel. Dafür werden die Rust-Komponenten einschließlich `rust-src`, der `bpf-linker` und eine passende Musl-C-Toolchain benötigt.

5.10.2 Start auf dem Zielsystem

Auf dem Zielsystem bündelt `scripts/run.sh` den vorgesehenen Ablauf: Es wechselt unabhängig vom Aufrufverzeichnis in das Repository-Wurzelverzeichnis, baut das Administrationstool mit `cargo build --locked --bin tails-pdp-admintool --release` und startet anschließend `tails-pdp` mit `cargo run --locked --bin tails-pdp --release`. Bei einem Buildfehler wird der Start abgebrochen. Eine Aktualisierung des Arbeitsbaums mit `git pull` erfolgt bei Bedarf separat. Die Cargo-Konfiguration verwendet für reguläre Ausführungen den Runner `sudo E`; dadurch erhält der Hauptprozess die zum Laden von eBPF-Programmen, zum Anheften des LSM-Hooks und zum Pinnen der Maps erforderlichen Berechtigungen, während Umgebungsvariablen für Logging oder Debugging erhalten bleiben.

Die zur Überprüfung des Builds und des Laufzeitverhaltens verwendete Testumgebung sowie die automatisierten Testszenarien werden in Kapitel 6 beschrieben.

5.10.3 Implementierungsbedingte Grenzen

Der Zielkernel muss BPF-LSM, die zuvor erläuterten BTF-Metadaten unter `/sys/kernel/btf/vmlinux`, das BPF-Dateisystem und die für Aya benötigten eBPF-Funktionen bereitstellen. Fehlen diese Voraussetzungen oder reichen die Berechtigungen zum Laden und Anheften nicht aus, endet der Start vor der Aktivierung des Hooks. Unterschiede zwischen Kernelversionen sowie Verifier-Grenzen werden daher erst auf dem konkreten Zielsystem sichtbar. Ein erfolgreicher Build belegt noch nicht, dass der Verifier des eingesetzten Kernels das eBPF-Programm akzeptiert.

Die feste Map-Repräsentation begrenzt die Zahl der gleichzeitig aktiven statischen Policies und Streampolicies für `file_open` pro Bank auf jeweils 16 und die Zahl dynamischer Attributbedingungen pro Streampolicy auf vier. Die Attribut-Hash-Map besitzt eine feste Obergrenze von 1024 Einträgen. Diese Grenzen sind Teil des Entwurfs für vorhersagbaren Kontrollfluss und begrenzten Map-Speicher; ein Überschreiten wird bereits im Userspace abgewiesen.

Die Festlegung auf die Real-UID ist eine bewusste Modellgrenze. Sie bildet weder privilegienwirksame Wechsel der effektiven oder der Filesystem-UID noch sämtliche Feinheiten threadbezogener Linux-Credentials als eigene Policyattribute ab. Ändert ein Prozess seine Real-UID nach dem Öffnen einer Datei, verwendet eine spätere Userspace-Nachbewertung den dann in `/proc/<pid>/status` sichtbaren Wert und nicht historisch die Identität des ursprünglichen Öffnungsvorgangs. Auch User Namespaces werden im Prototyp nicht als eigenständige Identitätsdomänen modelliert. Die Konsistenzbehauptung bezieht sich deshalb auf die vom Zielsystem bereitgestellten Real-UID-Werte; weitergehende Credential-Semantiken erfordern zusätzliche, explizit typisierte Subjektattribute.

Schließlich ist der nachträgliche FD-Entzug des Userspace-PEP auf das Zielsystem `x86_64`-Linux zugeschnitten. Die verwendeten Register und die Vorbereitung des

`close`-Systemaufrufs folgen dessen Aufrufkonvention. Weitere Prozessorarchitekturen liegen außerhalb des in dieser Arbeit betrachteten Implementierungsumfangs.

6 Evaluation

Ziel der Evaluation ist es, zu untersuchen, ob der implementierte Prototyp die zentralen Prinzipien von ASBAC für Dateiöffnungen unter Linux wie vorgesehen umsetzt. Dazu werden die Verarbeitung von Policies und dynamischen Attributen, die Kontrolle neuer Dateiöffnungen sowie die Nachbewertung bereits bestehender Dateizugriffe überprüft. Zusätzlich werden der Laufzeitaufwand und die technischen Grenzen der prototypischen Umsetzung betrachtet.

6.1 Testumgebung

Die Evaluation erfolgt in einer ausschließlich für die Untersuchung verwendeten virtuellen Maschine. Dadurch lassen sich die für das Laden der eBPF-Programme erforderlichen privilegierten Operationen ausführen, ohne ein Produktivsystem zu beeinflussen. Während der privilegierten End-to-End-Tests läuft keine weitere Instanz des Prototyps, da die Tests vorübergehend die gepinnten BPF-Maps unter `/sys/fs/bpf/tails-pdp` verwenden. Die VM-Konfiguration, die NixOS-Systemkonfiguration und die Definition der Entwicklungsumgebung werden gemeinsam mit dem Quellcode archiviert. Die Dateien `nix-config/configuration.nix`, `nix-config/hardware-configuration.nix` und `nix-config/shell.nix` dokumentieren die für die Evaluation relevanten System-, Hardwareerkennungs- und Werkzeugvoraussetzungen.

Für die Kernel- und Durchsetzungstests muss BPF in der aktiven LSM-Reihenfolge enthalten sein. Darüber hinaus werden BTF-Informationen unter `/sys/kernel/btf/vmlinux` und ein unter `/sys/fs/bpf` eingehängtes BPF-Dateisystem vorausgesetzt. Das Laden und Anheften der eBPF-Programme, der Zugriff auf die gepinnten Maps und der `ptrace`-basierte Entzug eines File Descriptors benötigen erhöhte Berechtigungen. Root-Berechtigung und BTF-Verfügbarkeit werden vom Testskript explizit geprüft; die Verfügbarkeit von BPF-LSM und BPF-Dateisystem wird durch den erfolgreichen Runtime-Start, das Anhängen des Hooks und die erzeugten Map-Pins nachgewiesen.

6.1.1 Hard- und Softwareumgebung

Die virtuelle Maschine wird mittels QEMU/KVM auf einem Proxmox-VE-Host betrieben. Für die Reproduzierbarkeit sind dabei zwei Perspektiven zu unterscheiden: Die von Proxmox ausgegebene VM-Konfiguration beschreibt die vom Hypervisor bereitgestellten Ressourcen, während die von NixOS erzeugte `hardware-configuration.nix` die im Gastsystem erkannten Geräte und die zu ladenden Kernelmodule festhält. Keine der beiden Konfigurationen ersetzt daher vollständig die jeweils andere.

Als virtuelles Prozessormodell ist `x86-64-v2-AES` konfiguriert. Zwei virtuelle Sockel mit jeweils zwei Kernen stellen insgesamt vier virtuelle CPUs bereit; zusätzlich sind 8192 MiB Arbeitsspeicher zugewiesen. Das virtuelle Systemlaufwerk besitzt eine Kapazität von 64 GiB und ist über VirtIO-SCSI angebunden. Diese Angaben werden aus der VM-

Konfiguration übernommen, weil sie nicht Bestandteil der NixOS-Hardwarekonfiguration sind. Weitere betriebliche Kennungen und Gerätedetails der VM sind für die durchgeführten Funktions- und Laufzeittests nicht entscheidungsrelevant und werden daher nicht einzeln aufgeführt.

Als Gastbetriebssystem kommt NixOS zum Einsatz. Die archivierte Systemkonfiguration setzt `system.stateVersion` auf `25.05`. Diese Option steuert kompatibilitätsrelevante Voreinstellungen, um bestehende Anwendungsdaten bei Systemaktualisierungen weiterverwenden zu können. Sie bezeichnet daher nicht die tatsächlich installierte NixOS-Version [Nix25]. Der Kernel wird durch die Konfiguration aus den Quellen des stabilen Linux-Kernels 6.16.12 mit festgelegtem SHA-256-Hash gebaut. Auf dem Testsystem ist NixOS `25.05.810656.5b5be50345d4` installiert. Die Konfiguration aktiviert unter anderem die Kerneloptionen für das Security-Framework und SecurityFS. Die LSM-Auswahl wird zentral über die NixOS-Option `security.lsm` auf `landlock`, `yama` und `bpf` festgelegt. NixOS erzeugt daraus den Kernelparameter `lsm=landlock,yama,bpf`. Zusammen mit dem vom Kernel ergänzten Capability-LSM ergibt sich die auf dem Testsystem über `/sys/kernel/security/lsm` bestätigte aktive Reihenfolge `capability,landlock,yama,bpf`.

Die von NixOS generierte `hardware-configuration.nix` bindet das QEMU-Gastprofil ein und legt `x86_64-linux` als Plattform des Gastsystems fest. Für den Systemstart werden unter anderem die Module für VirtIO-PCI, VirtIO-SCSI und SCSI-Datenträger in die Initial Ramdisk aufgenommen. Das Root-Dateisystem verwendet ext4, die EFI-Systempartition vfat.

Die Entwicklungs- und Build-Umgebung wird mit `nix-shell` aus `nix-config/shell.nix` erzeugt. Für den regulären Projektbuild sind insbesondere Rustup und Cargo, LLVM 21 einschließlich seiner Laufzeitbibliothek sowie `bpf-linker` erforderlich. Aya führt den Build der eBPF-Crate für das Ziel `bpfel-unknown-none` aus. Das Build-Skript dieser Crate setzt die Verfügbarkeit des `bpf-linker` voraus; die Shell-Konfiguration sieht dafür Version 0.9.15 mit LLVM-21-Unterstützung vor. Die Rust-Abhängigkeiten werden durch `Cargo.lock` festgelegt; der verwendete Stand enthält Aya 0.13.2 und `aya-ebpf` 0.1.2.

Für die Ausführung der Skripte enthält die Umgebung außerdem Bash, Coreutils, GNU Grep, Procps und Python 3.

Tabelle 4 fasst die aus den archivierten Konfigurationsdateien ableitbaren Merkmale sowie die auf dem Testsystem ermittelte NixOS-Version und aktive LSM-Reihenfolge zusammen. Die konkrete Proxmox-, NixOS-, Nixpkgs-, Rust- und Cargo-Version sowie der getestete Git-Commit werden unmittelbar auf dem Testsystem erfasst und zusammen mit den Testergebnissen dokumentiert. Dies ist insbesondere für Nixpkgs erforderlich, weil die aktuelle `nix-config/shell.nix` das zum Ausführungszeitpunkt konfigurierte `<nixpkgs>` importiert und keine feste Nixpkgs-Revision enthält.

Da die Evaluation innerhalb einer virtuellen Maschine erfolgt, können insbesondere Zeitmessungen durch das Scheduling des Hypervisors und durch gleichzeitig auf dem

Komponente	Konfiguration
Virtualisierungsplattform	Proxmox VE (9.2.18) mit QEMU/KVM
Virtuelles CPU-Modell	`x86-64-v2-AES`
CPU-Konfiguration	4 vCPUs
Arbeitsspeicher	8192 MiB
Virtuelles Laufwerk	64 GiB
Speicheranbindung	VirtIO-SCSI
NixOS-Hardwareprofil	QEMU-Gast, Plattform `x86_64-linux`
Dateisysteme	ext4 für `/`, vfat für `/boot`
Gastbetriebssystem	NixOS `25.05.810656.5b5be50345d4`
Linux-Kernel	6.16.12, aus festgelegtem Quellarchiv gebaut
LSM-Reihenfolge	`capability,landlock,yama,bpf`
Entwicklungsumgebung	`nix-shell` auf Basis von `<nixpkgs>`
LLVM	LLVM-Paketfamilie 21 einschließlich `libLLVM`
Rust-Toolchain	`nightly-2025-08-01` mit `rust-src`
eBPF-Zielarchitektur	`bpfel-unknown-none`
`bpf-linker`	0.9.15
Aya / `aya-ebpf`	0.13.2 / 0.1.2 gemäß `Cargo.lock`

Tabelle 4: Hard- und Softwarekonfiguration der Testumgebung

Proxmox-Host ausgeführte Arbeitslasten beeinflusst werden. Gemessene Laufzeiten charakterisieren daher die dokumentierte Testumgebung und sind nicht ohne Weiteres als allgemeingültige Werte für andere Hard- und Softwarekonfigurationen zu verstehen.

6.2 Teststrategie und Testszenarien

Die Evaluation kombiniert automatisierte Unit- und Komponententests mit privilegierten End-to-End-Tests und ausgewählten Laufzeitmessungen. Die Teststufen liefern unterschiedliche Arten von Evidenz und werden deshalb getrennt betrachtet. Unit- und Komponententests prüfen isolierbare Logik unter kontrollierten Bedingungen. End-to-End-Szenarien untersuchen dagegen das Zusammenspiel der Userspace-Komponenten mit den BPF-Maps und dem tatsächlich geladenen eBPF-LSM-Programm. Die Laufzeitmessungen ergänzen die funktionalen Nachweise um quantitative Aussagen zur Reaktionszeit und zum zusätzlichen Aufwand bei kontrollierten Dateiöffnungen.

Dieser Abschnitt beschreibt zunächst den Versuchsaufbau und das jeweils erwartete Verhalten. Die tatsächlich beobachteten Resultate und ihre Bewertung werden davon getrennt in Abschnitt 6.3 dargestellt.

6.2.1 Teststufen und Bewertungskriterien

Die Unit-Tests untersuchen einzelne Funktionen mit fest vorgegebenen Eingaben und vergleichen deren Ausgabe durch Assertions mit dem erwarteten Ergebnis. Komponententests beziehen mehrere Funktionen einer Userspace-Komponente ein, ersetzen externe Abhängigkeiten jedoch teilweise durch Testdoubles. So prüfen die Generationstests die Reihenfolge zwischen dem Schreiben in die inaktive Bank und der anschließenden Aktivie-

rung mit einem speicherbasierten Ersatz für die BPF-Maps. Die Tests des Userspace-PEP verwenden einen `FakeFdCloser`, der Schließversuche protokolliert, aber keinen realen Systemaufruf in einem anderen Prozess ausführt. Diese Tests können die fachliche Logik und das lokale Fehlerverhalten präzise prüfen, belegen jedoch weder die Annahme des eBPF-Programms durch den Kernel-Verifier noch die reale Durchsetzung einer Zugriffsentscheidung.

Die End-to-End-Tests führen den vollständigen Pfad von einer Policy- oder Attributdatei über die Loader und gepinnten BPF-Maps bis zur beobachtbaren Wirkung auf einen Dateizugriff aus. Sie prüfen damit Eigenschaften, die sich in einer isolierten Testumgebung nicht nachweisen lassen, etwa das Laden des eBPF-Objekts, das Anhängen des `file_open`-Hooks und den tatsächlichen Entzug eines File Descriptors. Quantitative Messungen werden wiederholt ausgeführt und anhand der beschriebenen Messgrößen ausgewertet. Einzelne erfolgreiche Durchläufe werden nicht als Nachweis allgemeiner zeitlicher Garantien oder unbegrenzter Skalierbarkeit interpretiert.

Tabelle 5 fasst die Teststufen und ihre Aussagegrenzen zusammen.

Teststufe	**Gegenstand**	**Aussagegrenze**
Unit-Test	Einzelne Parser-, Vergleichs- und Hilfsfunktionen	Kein realer Kernel- oder Durchsetzungsnachweis
Komponententest	Zusammenwirkende Funktionen einer Userspace-Komponente, teilweise mit Testdoubles	Externe Schnittstellen wie BPF-Maps oder `ptrace` werden teilweise ersetzt
End-to-End-Test	Vollständiger Pfad auf dem Linux-Zielsystem	Nachweis gilt für das definierte Szenario und die dokumentierte Testumgebung
Laufzeitmessung	Reaktionszeit und Aufwand kontrollierter Dateiöffnungen	Keine allgemeine Echtzeit- oder Skalierbarkeitsgarantie

Tabelle 5: Teststufen der Evaluation

Für die abschließende Zuordnung zu den Anforderungen werden die Bewertungen *erfüllt*, *teilweise erfüllt*, *nicht erfüllt* und *nicht nachgewiesen* verwendet. Sie beziehen sich auf die ausdrücklich formulierten Akzeptanzkriterien und die dokumentierten Versuchsbedingungen:

Erfüllt: Alle Teilkriterien sind unter den definierten Bedingungen nachgewiesen.

Teilweise erfüllt: Mindestens ein Teilkriterium ist nachgewiesen erfüllt und mindestens ein anderes nachgewiesen nicht erfüllt. Die betroffenen Teilkriterien werden einzeln benannt.

Nicht erfüllt: Das geforderte Verhalten oder ein geforderter Grenzwert wird nachweislich verfehlt; bei zusammengesetzten Kriterien mit zugleich erfüllten Anteilen wird die Gesamtbewertung als teilweise erfüllt ausgewiesen.

Nicht nachgewiesen: Die vorhandene Evidenz erlaubt kein belastbares Urteil über das Kriterium. Ein fehlender Nachweis wird nicht als beobachtetes Fehlverhalten gewertet.

Sind einzelne Teilkriterien nachgewiesen, andere jedoch offen, wird die Anforderung insgesamt als nicht vollständig nachgewiesen eingeordnet; erfüllte, verfehlte und offene Teilkriterien werden ausdrücklich getrennt angegeben. Grenzen außerhalb des definierten Anforderungsumfangs führen für sich genommen nicht zu einer Abwertung. Die Bewertung stellt keinen vollständigen Sicherheitsnachweis dar.

6.2.2 Unit- und Komponententests

Das Skript `scripts/test.sh` bildet den zentralen Einstiegspunkt der gesamten Prüfkette. Sein erster, nicht privilegierter Teil führte im dokumentierten Lauf 65 automatisierte Rust-Tests in fünf Testbereichen sowie eine Bash-Prüfung des Veröffentlichungsverfahrens aus. Die Option `-locked` stellt sicher, dass Cargo dabei die in `Cargo.lock` festgehaltene Abhängigkeitsauflösung nicht verändert. Der in der Cargo-Konfiguration für reguläre Programmstarts eingestellte privilegierte Runner wird für diese Tests durch `env` ersetzt. Die Tests benötigen daher keine Root-Rechte, laden kein eBPF-Programm in den Kernel und verändern keine gepinnten BPF-Maps.

Die Tests der gemeinsamen Crate `tails-pdp-common` bilden mit 16 Testfällen die fachliche Entscheidungslogik ab. Sie prüfen das Matching statischer und zeitabhängiger Policies, die Ableitung von UTC-Komponenten, die Grenzen der Vergleichsoperatoren und die Combining-Regel *deny-overrides*. Darüber hinaus untersuchen sie dynamische Zahlen-, Boolean- und Stringattribute, Typabweichungen sowie die unterschiedlichen Objektidentifikatoren der System-, Subject- und Resource-Namespaces. Da Teile dieser Funktionen sowohl im Kernelspace- als auch im Userspace-Pfad verwendet werden, prüfen die Tests die gemeinsam genutzte Semantik, nicht jedoch die Einbindung in die jeweilige Laufzeitumgebung.

Der Policyloader besitzt 16 Tests für das Einlesen, Parsen, Übersetzen und Validieren textueller Policies. Abgedeckt sind unter anderem statische und zeitabhängige Policies, frei benannte dynamische Attribute, ungültige Attributnamen, doppelte Policynamen, fehlende Semikolons, unbekannte oder deaktivierte Actions, ungültige Zeitwerte und das Überschreiten der festgelegten Kapazitätsgrenzen. Zwei Tests prüfen für statische und dynamische Policies die Annahme von Kommandonamen mit 15 Bytes und die Ablehnung von 16 Bytes, jeweils auch mit Zeichen, deren UTF-8-Kodierung mehrere Bytes benötigt. Weitere Tests prüfen das rekursive Einlesen ausschließlich von Dateien mit der Endung `.policy`. Ein speicherbasierter `FakePolicyStore` kontrolliert, dass zuerst die inaktive Bank beschrieben und erst danach die neue Generation aktiviert wird. Simulierte Schreib- und Aktivierungsfehler müssen die bisherige Generationsnummer unverändert lassen; außerdem werden nicht mehr belegte Einträge einer neu vorbereiteten Bank deaktiviert.

Die 13 Tests des Attributloaders prüfen das Parsen der unterstützten Zahlen-, Boolean- und Stringwerte sowie die Validierung von Attributnamen und des zulässigen Defcon-Wertebereichs. Ergänzend werden ein vollständiges Verzeichnis mit System-, Subject- und Resource-Attributen sowie die erst nach erfolgreicher Vorbereitung erfolgende Aktivierung einer neuen Attributbank mit simulierten Lösch-, Schreib- und Aktivierungs-

fehlern untersucht. Weitere Fehlerfälle decken die Kapazitätsprüfung vor der ersten Map-Änderung, Fehler beim Lesen von Generation und Belegung sowie die Fortsetzung des laufenden Updaters nach einer fehlgeschlagenen Übernahme eines neuen Attributstands ab. Die sechs Tests in `tails-pdp-userspace-common` umfassen zwei Prüfungen des begrenzten Trigger-Kanals und vier Linux-Watcher-Tests. Die Trigger-Tests prüfen die Koaleszierung bei voller Kanalkapazität und den Fehler bei einem geschlossenen Kanal. Die Watcher-Tests untersuchen atomare Veröffentlichungen gegenüber vorbereitenden und direkten Schreibvorgängen, neue und verschobene Unterverzeichnisse, schnelle Folgeersetzungen mit Verlust des Wurzelverzeichnisses sowie die Wiederherstellung der Watches nach einem Ereignisüberlauf.

Die 14 Tests des Userspace-PEP prüfen die Auswertung von Prozess- und File-Descriptor-Informationen sowie die Steuerung der Nachbewertung. Dazu gehören das Filtern numerischer Einträge unter `/proc`, die Auswahl der Real-UID aus `/proc/<pid>/status`, die Erkennung regulärer Dateien und die gezielte Zuordnung eines Verstoßes zu Prozess und File Descriptor. Mit dem `FakeFdCloser` wird geprüft, dass ein File Descriptor innerhalb eines Scans höchstens einmal behandelt wird, unterschiedliche Deskriptoren unabhängig bleiben und ein fehlgeschlagener Schließversuch die Bearbeitung weiterer Verstöße nicht abbricht. Ein zusätzlicher Test bildet die Konjunktion mehrerer dynamischer Bedingungen im produktiven Lookup-Ablauf ab; ein weiterer weist nach, dass ein fehlgeschlagener Entzug die Verarbeitung eines anderen Zielprozesses nicht verhindert. Ergänzend testen asynchrone Fälle das Warten auf Aktivierungsereignisse sowie die Berechnung relevanter Stunden- und Modulo-Zeitgrenzen.

Neben den funktionalen Rust-Tests führt der nicht privilegierte Teil von `scripts/test.sh` drei Qualitätsprüfungen aus. Eine reine Formatprüfung kontrolliert mit `cargo fmt`, ob der Rust-Quellcode dem vorgegebenen Format entspricht. Clippy analysiert Produktions- und Testcode statisch; Warnungen werden dabei als Fehler behandelt. Abschließend erzeugt ein Release-Build die Userspace-Binaries und über das Build-Skript des Hauptprogramms auch das eingebettete eBPF-Objekt. Dieser Build belegt die Übersetzbarkeit des gemeinsamen Quellstands, ersetzt aber nicht die anschließende Prüfung durch den Kernel-Verifier. In der eBPF-Crate selbst werden keine nativen Unit-Tests ausgeführt, da ihr Code für die BPF-Zielarchitektur und eine `no_std`-Umgebung übersetzt wird. Die gemeinsame Policylogik wird daher nativ getestet; die tatsächliche Kernelausführung ist Gegenstand der folgenden End-to-End-Szenarien.

Ergänzend prüfen sechs nicht privilegierte Python-Tests die Laufzeit (PERF-03) Messauswertung. Sie prüfen unter anderem die zeitliche Reihenfolge, fehlende oder mehrfache Zeitmarkierungen, eine Aktivierung vor dem Ereignisempfang und noch laufende Scans vor Messbeginn.

6.2.3 Privilegierte End-to-End-Tests

Die End-to-End-Prüfung wird auf dem in Abschnitt 6.1 beschriebenen Linux-Zielsystem mit Root-Rechten ausgeführt. Der zentrale Runner `scripts/test.sh` fordert diese

Berechtigungen nach dem Release-Build über sudo an und ruft scripts/tests/test-e2e.sh auf. Dieses Skript bildet den gemeinsamen Einstiegspunkt der End-to-End-Stufe und ruft für die Test-IDs E2E-01 bis E2E-19 (siehe Tabelle 6) jeweils eine eigene Bashdatei unter scripts/tests/e2e/ auf. Vor dem Start bricht die Hauptdatei ab, wenn bereits eine Instanz des Prototyps läuft; anschließend entfernt sie die bekannten Test-Maps unter /sys/fs/bpf/tails-pdp. Deshalb darf die Suite ausschließlich auf dem dedizierten Testsystem und nicht parallel zu einer regulären Instanz oder einem weiteren Testlauf ausgeführt werden.

Für die ersten zehn Szenarien startet die Hauptdatei eine gemeinsame Runtime in einem temporären Arbeitsverzeichnis. Dieses enthält eigene Policy- und Attributverzeichnisse sowie eine geschützte und eine nicht geschützte Testdatei. Änderungen werden zunächst vollständig in eine temporäre Datei geschrieben und anschließend atomar aktiviert. Wo ein erlaubter Zugriff allein noch keinen Nachweis für die Verarbeitung einer Änderung liefern würde, wartet der Test zusätzlich auf einen Wechsel der Policy- oder Attributgeneration. Nach E2E-10 wird diese Runtime beendet und der gepinnte Testzustand entfernt.

Die Szenarien E2E-11 bis E2E-19 werden darin anschließend über scripts/tests/test-evaluation.sh mit jeweils eigener Runtime und isoliertem Arbeitsverzeichnis ausgeführt. Der zugehörige Runner sperrt parallele Evaluationsläufe, verwaltet ausschließlich die von ihm gestarteten Prozesse und verweigert den Start, wenn bekannte gepinnte Maps bereits vorhanden sind. Beim Aufräumen entfernt er nur die durch den jeweiligen Lauf erzeugten Maps. Für die Prüfung neuer Dateiöffnungen verwendet dieser Teil der Suite einen kleinen Syscall-Helfer. Ein Zugriff gilt nur dann als durch den LSM-Hook verweigert, wenn open() mit EPERM fehlschlägt; andere Ein-/Ausgabefehler werden nicht als erfolgreiches Deny gewertet.

Die Kennungen der ergänzenden Szenarien unterscheiden Komponententests (COMP), Charakterisierungstests (CHAR), Tests auf Wettlaufsituationen (RACE), Laufzeitmessungen (PERF), Kapazitätstests (LOAD) und Stabilitätstests (STAB). Die nachgestellte Nummer identifiziert das jeweilige Szenario innerhalb der Testgruppe.

Nach Abschluss aller 19 End-to-End-Szenarien ruft scripts/test.sh denselben Evaluationsrunner ein zweites Mal ausschließlich für COMP-04, CHAR-01, RACE-01, PERF-01 bis PERF-03, LOAD-01 und STAB-01 auf. Damit werden alle Teststufen durch einen Aufruf von scripts/test.sh ausgeführt, ohne die Szenarien E2E-11 bis E2E-19 doppelt zu starten. Die Teilrunner bleiben für gezielte Wiederholungen einzelner Szenarien separat aufrufbar.

Tabelle 6 fasst die grundlegenden Start-, Entscheidungs- und Attributszenarien zusammen. Die Kriterien beschreiben das erwartete Verhalten; die tatsächlich beobachteten Ergebnisse werden getrennt in Abschnitt 6.3 berichtet.

Die weiteren Szenarien in Tabelle 7 konzentrieren sich auf die Combining-Regel, bestehende File Descriptors, die Administrationsschnittstelle und kontrollierte Fehlerfälle. Beim nachträglichen Entzug hält ein Hilfsprozess mehrere Deskriptoren geöffnet und prüft über

Test-ID	Gegenstand	Bestehenskriterium
E2E-01	Runtime, Kernel-Verifier, LSM-Attach und Maps	Die Runtime bleibt aktiv und alle erwarteten Maps sind gepinnt.
E2E-02 bis E2E-04	Standardentscheidung, statische Deny-Policy und aktuelle UTC-Stunde	Der Zugriff ist ohne passende Deny-Policy erlaubt, wird bei passender statischer oder zeitabhängiger Policy verweigert und nach deren Entfernung wieder erlaubt.
E2E-05 bis E2E-07	System-, Subject- und Resource-Attribute	Änderungen von `system.defcon`, `subject.position` und `resource.classification` verändern die reale Zugriffsentscheidung in beide Richtungen.
E2E-08	Ungültige Policygeneration	Die Runtime meldet den Parserfehler, aktiviert die fehlerhafte Generation nicht und erzwingt weiterhin die letzte gültige Deny-Policy.
E2E-12	Konjunktion mehrerer Attribut-Namespaces	Die Deny-Policy greift nur bei gleichzeitig passenden System-, Subject- und Resource-Attributen; fehlende, typfalsche oder abweichende Einzelwerte heben das Deny auf.
E2E-13	Ungültige Attributgeneration	Ein unzulässiger Attributwert wird gemeldet, ohne die sichtbare Generation oder die zuvor gültige Entscheidung zu verändern.

Tabelle 6: Grundlegende End-to-End-Szenarien

`fstat()`, ob ausschließlich die von der neuen Entscheidung betroffenen Deskriptoren ungültig werden. Der Fehlercode `EBADF` (*Bad file descriptor*, ungültiger Dateideskriptor) zeigt dabei an, dass der geprüfte Deskriptor nicht mehr gültig ist [Ker26d].

Ein Szenario gilt nur dann als bestanden, wenn sein Skript ohne fehlgeschlagene Assertion endet, die Runtime während der vorgesehenen Prüfung aktiv bleibt und im neu hinzugekommenen Kernel-Log kein Hinweis auf *Kernel panic*, *Oops*, *BUG* oder einen General-Protection-Fault enthalten ist. Für die isolierten Szenarien werden Status, Parameter und gegebenenfalls erzeugte Detaildaten in `report.json` gespeichert. Runtime-, Szenario- und Kernel-Logs bleiben im temporären Verzeichnis unter `/tmp/tails-eval-*` erhalten. Diese Artefakte bilden die Grundlage für die Ergebnisdarstellung, sind aber selbst noch kein Nachweis über andere Kernel, Konfigurationen oder Lastsituationen hinaus.

Test-ID	Gegenstand	Bestehenskriterium
E2E-09 und E2E-16	Selektiver Entzug bestehender File Descriptors	Ein beziehungsweise mehrere verletzende Deskriptoren werden geschlossen; weiterhin zulässige Deskriptoren desselben Prozesses bleiben geöffnet.
E2E-10 und E2E-14	Administrationsschnittstelle und Nachvollziehbarkeit	Die Ausgabe enthält aktive Generation, Deny-Policy, Ressourcenidentität, Attributbedingung und Attributwert, ohne den Zustand zu verändern. Dies weist die rekonstruierbare Ursache im kontrollierten Szenario nach, jedoch kein individuelles Auditlog jeder Entscheidung.
E2E-11	Combining-Regel *deny-overrides*	Bei gleichzeitig passender Permit- und Deny-Policy wird der Zugriff unabhängig von der lexikalischen Policyreihenfolge verweigert; nach Entfernen des Deny wird er erlaubt.
E2E-15	Real und Effective UID	Bei privilegiertem Effective- und Filesystem-User wird die Entscheidung anhand der gesetzten Real-UID getroffen; eine andere Real-UID bildet die Negativkontrolle.
E2E-17	Nicht erreichbarer Zielprozess beim FD-Entzug	Der gezielt erzeugte `ptrace`-Attach-Konflikt wird protokolliert, ein anderer Zielprozess wird dennoch verarbeitet und weitere Policyaktivierungen sowie Kernelentscheidungen bleiben möglich.
E2E-18	Attributgetriggerter FD-Entzug	Eine reine Attributänderung schließt drei unzulässige FD; zwei erlaubte bleiben offen. Die Policygeneration bleibt unverändert.
E2E-19	Zeitgetriggerter FD-Entzug	Eine Zeitgrenze schließt drei unzulässige FD; zwei erlaubte bleiben offen. Policy- und Attributgeneration bleiben unverändert.

Tabelle 7: End-to-End-Szenarien für Enforcement und Fehlerverhalten

6.3 Ergebnisse

Die nachfolgend dargestellten Ergebnisse wurden auf dem in Abschnitt 6.1 beschriebenen Zielsystem erhoben.

6.3.1 Ergebnisse der funktionalen Prüfungen

Im aktuellen vollständigen Prüflauf wurden alle 65 Unit- und Komponententests erfolgreich ausgeführt. Davon entfallen 16 Tests auf die gemeinsame Policylogik, 16 auf den Policyloader, 13 auf den Attributloader, sechs auf Triggerkanal und Linux-Watcher sowie 14 auf den Userspace-PEP. Die Formatprüfung und der separate Release-Build der Runtime und des Administrationstools einschließlich des eBPF-Objekts waren ebenfalls erfolgreich. Damit ist nachgewiesen, dass der Prototyp übersetzbar ist und die in Abschnitt 6.2.2 beschriebenen Assertions erfüllt.

Die vollständige Ausführung von `scripts/test.sh` wurde mit Exitstatus `0` abgeschlossen. Clippy prüfte Produktions-, Test- und Binärziele mit als Fehler behandelten Warnungen erfolgreich. Die Bash-Prüfung der atomaren Konfigurationsveröffentlichung bestand ebenfalls. Auch die sechs ergänzenden Python-Tests zur Auswertung der PERF-03-Zeitmarkierungen bestanden. Alle acht ergänzenden Evaluationsszenarien für Map-Zugriff, Deskriptorgrenzen, Race-Verhalten, Performance, Kapazität und Stabilität wurden ebenfalls erfolgreich abgeschlossen. Damit waren sowohl die funktionalen Prüfungen als auch die definierte Qualitätsprüfkette vollständig erfolgreich.

Auf dem Zielkernel bestanden alle 19 End-to-End-Szenarien. Die Runtime wurde vom Kernel-Verifier akzeptiert, der `file_open`-Hook angehängt und die erwarteten Maps wurden gepinnt. Ohne passende Deny-Policy blieb der Zugriff erlaubt; statische, zeitabhängige und von dynamischen Attributen abhängige Deny-Policies führten dagegen zu `EPERM`. Änderungen von System-, Subject- und Resource-Attributen wurden in beide Entscheidungsrichtungen übernommen. Auch die Konjunktion mehrerer Attribut-Namespaces und die Combining-Regel *deny-overrides* zeigten das erwartete Verhalten.

Ungültige Policy- und Attributstände ersetzten nicht die jeweils letzte gültige Generation. Nach der Korrektur des Attributloaders wurde auch eine Überschreitung der realen Map-Kapazität kontrolliert abgelehnt: Je 512 Attribute pro vollständig belegter Bank wurden akzeptiert, ein Update auf 513 Attribute bei weiterhin 512 Einträgen der aktiven Bank dagegen vor einer Veränderung der Maps zurückgewiesen. Generation und aktive Attributwerte blieben unverändert; anschließende gültige Updates wurden ohne Neustart der Runtime verarbeitet. Für Policies wurden je 16 statische und 16 Streampolicies akzeptiert und der jeweils 17. Eintrag kontrolliert abgelehnt. Damit wurde nicht nur das Verhalten an den vorgesehenen Grenzen, sondern auch die Wiederherstellung des regulären Updatebetriebs nach einer Ablehnung beobachtet.

Der nachträgliche Entzug bestehender Dateizugriffe war in den vorgesehenen Szenarien ebenfalls erfolgreich. Sowohl ein einzelner als auch drei gleichzeitig verletzende File De-

scriptors wurden geschlossen, während die erlaubten Deskriptoren desselben Prozesses geöffnet blieben. E2E-18 bestätigte dieses Verhalten nach einer reinen Attributänderung bei unveränderter Policygeneration. E2E-19 bestätigte es beim Erreichen einer Zeitgrenze ohne Änderung der Policy- und Attributgeneration. In beiden Szenarien wurden drei unzulässige Deskriptoren geschlossen, während zwei erlaubte geöffnet blieben. War ein Zielprozess aufgrund eines bestehenden `ptrace`-Attachs nicht erreichbar, wurde der Fehler protokolliert und ein weiterer Prozess trotzdem erfolgreich verarbeitet. Nachfolgende Policyaktivierungen und Kernelentscheidungen blieben möglich. Die Administrationsschnittstelle zeigte aktive Generationen, Policies, Ressourcen und Attribute ohne beobachtete Zustandsänderung. Im kontrollierten Szenario ließ sich die Ursache einer Ablehnung aus der einzigen aktiven Deny-Policy, der Ressourcenidentität, der Attributbedingung und dem aktuellen Attributwert rekonstruieren. Ein individuelles Entscheidungsprotokoll für jede Dateiöffnung oder der ursprüngliche Policyname wird jedoch nicht ausgegeben.

Prüfung	**Ergebnis**	**Einordnung**
Rust-Tests	65 von 65 bestanden	Unit- und Komponentennachweise ohne realen Kernelpfad
End-to-End-Tests	19 von 19 bestanden	Vollständiger Pfad einschließlich Kernel-Verifier, LSM-Attach und Enforcement
Formatprüfung	Bestanden	Keine Formatabweichungen im Rust-Code
Bash-Veröffentlichungsprüfung	Bestanden	Atomare Ersetzung, Berechtigungen und Fehlerbereinigung
Python-Tests	6 von 6 bestanden	Prüfung der PERF-03-Messauswertung
Clippy-Gesamtprüfung	Bestanden	Statische Analyse mit als Fehler behandelten Warnungen
Release-Build	Bestanden	Userspace-Binaries und eBPF-Objekt wurden erzeugt
Kapazitätstest	Bestanden	Kontrollierte Ablehnung ohne Zustandsverlust und mit erfolgreichen Folgeupdates

Tabelle 8: Zusammenfassung der funktionalen Testergebnisse

6.3.2 Robustheit und Grenzen des Entzugsmechanismus

Im Stabilitätstest wurden 100 aufeinanderfolgende Zyklen mit gültigen und ungültigen Policy- und Attributänderungen ausgeführt. Die Runtime blieb bis zum regulären Testende aktiv. Die vom Runner ausgewertete Differenz des Kernel-Logs enthielt keinen Hinweis auf Kernel-Panic, Oops, BUG oder einen General-Protection-Fault. Der Stabilitätstest benötigte 31 Sekunden. Dieses Ergebnis belegt das Verhalten während des begrenzten Tests, stellt jedoch weder einen Langzeittest noch einen allgemeinen Nachweis der Kernelstabilität dar.

Die Charakterisierung duplizierter und durch `fork()` vererbter Deskriptoren zeigte, dass in Eltern- und Kindprozess jeweils alle vier betroffenen Deskriptoren geschlossen wurden, während jeweils zwei erlaubte Deskriptoren geöffnet blieben. Eine bereits bestehende `mmap()`-Abbildung der Datei blieb danach lesbar. Der Prototyp entzieht somit den File Descriptor, widerruft aber keinen zuvor eingerichteten speicherabgebildeten Zugriff. Diese Grenze folgt aus dem in Kapitel 3 ausdrücklich auf File Descriptors eingegrenzten Prototypumfang, ist für die sicherheitstechnische Interpretation des Ergebnisses jedoch

relevant.

Im nebenläufigen Stresstest wurden während zehn Policyzyklen 348 Wechsel der Ressource hinter derselben Deskriptornummer erzeugt. Dabei wurde kein erlaubter Deskriptor fälschlich geschlossen. Da der Test stochastisch ist, beweist dieses Ergebnis keine Race-Freiheit. Zwischen dem Erfassen der Deskriptoridentität und dem späteren `ptrace`-basierten Schließen verbleibt grundsätzlich ein Zeitfenster, in dem der Zielprozess die Deskriptornummer wiederverwenden kann.

6.3.3 Quantitative Ergebnisse

Tabelle 9 enthält die zentralen Messwerte. Bei PERF-01 wurden je 20 000 Dateiöffnungen mit warmem Cache gemessen. Jeder der drei Messzustände wurde zuvor mit 1000 ungemessenen Öffnungs- und Schließvorgängen vorbereitet. Der Median stieg von 4,223 μs ohne aktiven Prototyp auf 4,694 μs mit aktivem Prototyp ohne Policy und auf 4,604 μs bei passender Permit-Policy. Dies entspricht einem medianen Mehraufwand von 0,471 beziehungsweise 0,381 μs, entsprechend 11,2 beziehungsweise 9,0 Prozent. Die p95-Werte steigen gegenüber der Baseline um 0,481 beziehungsweise 0,399 μs, entsprechend 10,4 beziehungsweise 8,6 Prozent. Die Messung umfasst `os.open()` sowie den Aufwand der Python-Laufzeit und des Timers; `close()` liegt außerhalb des Messintervalls. Sie isoliert daher nicht ausschließlich die Ausführungszeit des LSM-Hooks. Dass der Zustand mit Permit-Policy einen geringeren Median und p95 als der Zustand ohne Policy aufweist, begründet keinen Leistungsvorteil der Policy-Auswertung: Die Zustände werden sequenziell in einem einzelnen Lauf gemessen und unterliegen Laufzeitschwankungen.

Messgröße	*n*	**Median**	**p95**
Dateiöffnung ohne Runtime	20 000	4,223 μs	4,645 μs
Dateiöffnung, Runtime ohne Policy	20 000	4,694 μs	5,126 μs
Dateiöffnung, passende Permit-Policy	20 000	4,604 μs	5,044 μs
Policyänderung bis neues Deny	10	10,42 ms	10,63 ms
Attributänderung bis neues Deny	10	10,54 ms	10,68 ms
Policyänderung bis `EBADF`	10	9,40 ms	14,24 ms

Tabelle 9: Quantitative Messergebnisse auf dem Zielsystem

Die Reaktionszeit vom atomaren Austausch einer Datei bis zur beobachtbaren neuen Entscheidung lag im Median bei 10,42 ms für Policyänderungen und 10,54 ms für Attributänderungen. Dabei wurde der Zugriff in Intervallen von 10 ms geprüft. Die Messung enthält somit neben Verarbeitung und Aktivierung auch Dateisystembeobachtung, Scheduling und Polling. Die Mediane liegen nahe am Beobachtungsabstand; sie dürfen daher nicht als präzise interne Aktivierungszeiten interpretiert werden. Eine Attributänderung wurde bereits nach 0,28 ms erkannt, die übrigen neun nach etwa 10,4 bis 10,7 ms. Dies verdeutlicht den Einfluss der ersten Zugriffsprüfung und des Pollings.

Für den Entzug bestehender Zugriffe betrug die Zeit von der Policydateiänderung bis zum beobachteten `EBADF` im Median 9,40 ms. PERF-03 bereitet die Policydatei außerhalb des überwachten Verzeichnisses vor und beginnt die Messung unmittelbar vor ihrer atomaren

Umbenennung in das Policyverzeichnis. Zuvor müssen der wartende Policy-Watcher und abgeschlossene Scans über 300 ms einen unveränderten Ruhezustand zeigen. Zeitmarkierungen prüfen, dass das Dateiereignis erst nach Messbeginn empfangen und die Generation anschließend aktiviert wurde. Eine feste Bündelungswartezeit besteht in diesem Stand nicht. Zwei weitere Zeitmarkierungen unmittelbar vor und nach dem Map-Systemaufruf zur Aktivierung grenzen deren Zeitpunkt ein. Die Messung enthält den Aufwand dieser Instrumentierung; der Entzug wird durch File Descriptor-Prüfungen im Abstand von 1 ms beobachtet. Über zehn Wiederholungen lagen die unteren Grenzen der Aktivierung-bis-Entzug-Intervalle zwischen 8,72 und 9,65 ms, die oberen Grenzen zwischen 8,73 und 9,66 ms. Eine scheinexakte Einzelzeit wird daraus nicht abgeleitet. Das 95. Perzentil (p95) bezeichnet den Wert, den mindestens 95 % der Messwerte nicht überschreiten. Bei zehn Wiederholungen entspricht es nach der verwendeten Berechnung bereits dem höchsten Messwert. Das angegebene p95 ist deshalb keine belastbare Schätzung seltener hoher Latenzen. OA-03 begrenzt den relativen Mehraufwand von Dateiöffnungen für Median und p95 auf jeweils 20 % und das p95 der Reaktions- und Entzugslatenzen auf jeweils 50 ms. Die archivierten Performance-Tests prüfen die erfolgreiche Messdurchführung und den beobachteten Zustandswechsel; ihr Bestehen allein weist die Einhaltung der Grenzwerte nicht nach. Der Vergleich mit den Grenzwerten erfolgt anhand der ausgewerteten Messwerte. Daraus folgt keine Eignung für einen bestimmten produktiven Einsatz.

6.3.4 Bewertung der Anforderungen

Die funktionalen Anforderungen FA-01 bis FA-10 werden innerhalb des in Kapitel 3 definierten Prototypumfangs als erfüllt bewertet. Die Tests weisen das Einlesen, Aktualisieren und Validieren von Policies, die Kontrolle neuer Dateiöffnungen, dynamische und frei benannte Attribute sowie den selektiven Entzug unzulässig gewordener File Descriptors nach. Diese Bewertung umfasst nur die getesteten Policyformen, Attributtypen und den Hook `file_open`.

FA-06: Nachbewertung und Entzug bestehender File Descriptors

Das Akzeptanzkriterium wird anhand der folgenden Teilprüfungen bewertet. Die Nachweise beziehen sich auf die in den Szenarien kontrolliert geöffneten Deskriptoren:

Policyänderung – erfüllt: E2E-09 und E2E-16 weisen nach, dass zuvor zulässige, durch eine Policyänderung unzulässig gewordene Deskriptoren tatsächlich geschlossen werden. Das beobachtete `EBADF` belegt den erfolgreichen Entzug im geprüften Fall.

Attributänderung – erfüllt: E2E-18 schließt nach einer reinen Attributänderung drei unzulässige Deskriptoren bei unveränderter Policygeneration. E2E-19 ergänzt den Nachweis für das Erreichen einer Zeitgrenze bei unveränderter Policy- und Attributgeneration.

Selektivität – erfüllt: E2E-16, E2E-18 und E2E-19 zeigen, dass die weiterhin zulässigen

Deskriptoren desselben Prozesses geöffnet bleiben. In E2E-18 und E2E-19 stehen jeweils drei geschlossenen unzulässigen Deskriptoren zwei weiterhin geöffneten zulässigen Deskriptoren gegenüber.

Protokollierung des Fehlerfalls – erfüllt: E2E-17 erzeugt einen kontrollierten `ptrace`-Attach-Konflikt. Das Szenario prüft die Fehlermeldung zum betroffenen Zielprozess im Runtime-Log.

Fortsetzung nach einem Fehler – erfüllt: Im selben Szenario werden die drei unzulässigen Deskriptoren eines anderen, erreichbaren Zielprozesses geschlossen, während dessen zwei zulässige Deskriptoren geöffnet bleiben. Weitere Policyaktivierungen und Kernelentscheidungen bleiben möglich.

Erfolgsbewertung – erfüllt: Der Report von E2E-17 weist für den blockierten Prozess drei nicht geschlossene Deskriptoren und für den erreichbaren Prozess drei geschlossene Deskriptoren aus. Die fehlgeschlagenen Schließversuche werden ausdrücklich nicht als erfolgreiche Entzüge gezählt. Das Bestehen von E2E-17 belegt die geforderte Fehlerbehandlung, nicht den erfolgreichen Entzug beim blockierten Prozess.

FA-06 ist damit im definierten Prototypumfang erfüllt. Die Nachweise der isolierten Szenarien sind im Report unter `eval-tails-pdp/test-results/20261010T172219Z-QZnscf/evaluation/tails-eval-1qatkm5o/report.json` archiviert; E2E-09 gehört zur gemeinsamen E2E-Umgebung desselben Laufs. Die Bewertung besagt nicht, dass jeder unzulässig gewordene Zugriff erfolgreich beendet werden kann. Insbesondere folgt daraus kein Widerruf bestehender `mmap()`-Abbildungen, da der bestehende Zugriff für den Prototyp als geöffneter File Descriptor definiert ist. Die zeitliche Bewertung des Entzugs erfolgt getrennt unter OA-03.

OA-01: Stabilität

OA-01 ist für die untersuchten Eingaben und die Dauer der Testläufe erfüllt. Ungültige Eingaben, Kapazitätsüberschreitungen und ein nicht erreichbarer Zielprozess führten in den Abschlussläufen weder zum Abbruch der Runtime noch zu einem vom Runner erkannten Kernelproblem. Eine allgemeine Stabilitätsgarantie kann daraus nicht abgeleitet werden.

OA-02: Beobachtbarkeit und Nachvollziehbarkeit

Maßgeblich ist das Akzeptanzkriterium aus Kapitel 3: Für zentrale Zugriffsentscheidungen muss erkennbar sein, welche Policy oder welcher Zustand die Entscheidung beeinflusst hat. E2E-10 und E2E-14 prüfen die lesende Ausgabe der Administrationsschnittstelle. Im kontrollierten Ablehnungsszenario sind die aktive Generation, die einzige aktive Deny-Policy, die Ressourcenidentität, die Attributbedingung und der aktuelle Attributwert sichtbar. Dadurch lässt sich die Entscheidungsgrundlage der geprüften Ablehnung rekonstruieren.

OA-02 wird deshalb für dieses kontrollierte Szenario als erfüllt bewertet; der Nachweis von E2E-14 ist im zuvor genannten Report der isolierten E2E-Szenarien archiviert.

Ein individuelles Auditlog jeder Dateiöffnung und eine unmittelbare Zuordnung zum ursprünglichen Policynamen werden nicht bereitgestellt. Diese Eigenschaften werden vom formulierten Akzeptanzkriterium nicht ausdrücklich verlangt und begründen deshalb keine Bewertung als teilweise erfüllt. Sie bleiben Grenzen des Nachweises: Insbesondere wird damit keine eindeutige Rekonstruktion beliebiger vergangener Entscheidungen bei zwischenzeitlich veränderter Policy- oder Attributlage nachgewiesen.

OA-03: Performance

Das zusammengesetzte Akzeptanzkriterium wird in Tabelle 10 einzeln mit den Messwerten verglichen. Für Median und p95 wird der relative Mehraufwand jeweils als $100\,\% \cdot (x_{\text{Prototyp}} - x_{\text{Baseline}})/x_{\text{Baseline}}$ berechnet. Die Baseline stammt aus demselben PERF-01-Lauf. Für den Vergleich werden die ungerundeten Werte verwendet; die Tabelle zeigt gerundete Ergebnisse.

Teilkriterium	**Soll**	**Ist**	**Nachweis**	**Urteil**
Median-Mehraufwand ohne Policy	≤ 20 %	11,2 %	PERF-01	Erfüllt
p95-Mehraufwand ohne Policy	≤ 20 %	10,4 %	PERF-01	Erfüllt
Median-Mehraufwand mit Permit-Policy	≤ 20 %	9,0 %	PERF-01	Erfüllt
p95-Mehraufwand mit Permit-Policy	≤ 20 %	8,6 %	PERF-01	Erfüllt
p95 nach Policyänderung	≤ 50 ms	10,63 ms	PERF-02	Erfüllt
p95 nach Attributänderung	≤ 50 ms	10,68 ms	PERF-02	Erfüllt
p95 des Entzugs nach Policyänderung	≤ 50 ms	14,24 ms	PERF-03	Erfüllt
Erfolgreicher Entzug im Messszenario	Alle	10 von 10	PERF-03	Erfüllt
Dokumentation des Messverfahrens	Vollständig	Vorhanden	Text und Artefakte	Erfüllt

Tabelle 10: Einzelbewertung der Akzeptanzkriterien von OA-03 anhand des archivierten Laufs

Die Messwerte und die zehn erfolgreichen Entzugsmessungen sind im Report unter `eval-tails-pdp/test-results/20261010T172219Z-QZnscf/evaluation/tails-eval-nd76qwck/report.json` archiviert. Die zugehörigen Szenarioartefakte enthalten die Detaildaten. Die Definition von Messbeginn und Messende, die Beobachtungsintervalle von 10 ms beziehungsweise 1 ms, die Wiederholungszahlen sowie die synthetische Testlast und der warme Cache sind im Versuchsaufbau, in der quantitativen Ergebnisdarstellung und den zugehörigen Artefakten dokumentiert. Der Erfolgsanteil bezieht sich ausschließlich auf die zehn kontrollierten Entzugsvorgänge von PERF-03; der absichtlich erzeugte Fehlerfall von E2E-17 gehört nicht zu dieser Messreihe. Die Grenze von 50 ms gilt für das gemessene p95 und ist keine obere Grenze jedes einzelnen Vorgangs. Die geringe Wiederholungszahl begrenzt insbesondere Aussagen über seltene Ausreißer.

Die Messwerte werden anhand der in Kapitel 3 definierten Akzeptanzkriterien bewertet. Für den untersuchten Stand liegen sämtliche Messwerte innerhalb der festgelegten Grenzen. Zusammen mit den nachgewiesenen erfolgreichen Entzügen und der dokumentierten Messmethodik ist OA-03 damit für den archivierten Stand und die untersuchten Szenarien erfüllt. Die PASS-Status der Skripte ersetzen den hier vorgenommenen Grenzwertvergleich nicht.

OA-04: Reproduzierbarkeit

OA-04 wird durch die archivierten Konfigurationen, Szenarioskripte, Testparameter und Rohdaten erfüllt.

Die entwicklungsbezogenen Anforderungen sind nicht allein durch Laufzeittests entscheidbar. Die Trennung in eBPF-Programm, gemeinsame Datentypen und Entscheidungslogik, Loader, Userspace-PEP und Administrationstool unterstützt EA-01. Frei benannte Attribute und getrennte Komponenten bieten Erweiterungspunkte im Sinne von EA-02; neue Attributtypen, Map-Strukturen oder LSM-Hooks erfordern jedoch weiterhin Änderungen an mehreren Schnittstellen, weshalb diese Anforderung nur teilweise erfüllt ist. EA-03 wird dadurch unterstützt, dass Einlesen, Validierung, Aktualisierung, Nachbewertung und Administration im Userspace liegen. Im Kernel verbleiben der notwendige LSM-Hook, Map-Lookups und die unmittelbar für eine Dateiöffnung erforderliche Auswertung.

Insgesamt zeigen die Ergebnisse, dass der Prototyp die für diese Arbeit ausgewählten ASBAC-Prinzipien auf dem untersuchten Linux-System funktional umsetzt: Zugriffsentscheidungen reagieren auf Policy- und Attributströme, neue Dateiöffnungen werden im Kernel kontrolliert und bereits geöffnete File Descriptors im Userspace nachbewertet. Die Ergebnisse sind ein Machbarkeitsnachweis für den definierten Prototyp, kein vollständiger Sicherheits-, Race-Freiheits-, Echtzeit- oder Produktionstauglichkeitsnachweis.

7 Zusammenfassung und Ausblick

7.1 Zusammenfassung und Beantwortung der Forschungsfrage

Diese Arbeit untersuchte, wie zentrale Prinzipien von ASBAC für Dateiöffnungen prototypisch mittels eBPF-LSM unter Linux umgesetzt werden können und welche technischen Einschränkungen sich insbesondere bei der nachträglichen Kontrolle bestehender Dateizugriffe ergeben. Hierzu wurde ein Prototyp entwickelt, der textuelle Policies und dynamische Attribute im Userspace verarbeitet, in eine feste Kernelrepräsentation übersetzt und über gepinnte eBPF-Maps bereitstellt. Neue Dateiöffnungen werden unmittelbar im LSM-Hook `file_open` bewertet. Bereits geöffnete File Descriptors werden nach einer erfolgreichen Policy- oder Attributaktivierung sowie beim Erreichen einer entscheidungsrelevanten Zeitgrenze im Userspace erneut untersucht. Ergibt die Nachbewertung eine Verletzung, versucht der Userspace-PEP, den betroffenen Deskriptor im Zielprozess zu schließen.

Die Forschungsfrage lässt sich damit wie folgt beantworten: Eine prototypische Übertragung ausgewählter ASBAC-Prinzipien ist durch eine hybride Architektur möglich. eBPF-LSM bildet einen kernelgestützten Durchsetzungspfad für neue Dateiöffnungen, während Userspace-Komponenten die komplexere Policyverarbeitung, die Aktualisierung dynamischer Attribute und die Nachkontrolle bestehender Deskriptoren übernehmen. Feste Datentypen und begrenzte Kontrollflüsse machen die Entscheidungslogik für eBPF ausführbar. Zwei Map-Bänke und getrennte Generationszähler sorgen dafür, dass neue Policy- und Attributstände erst nach ihrer vollständigen Vorbereitung aktiviert werden. Die Aufteilung über Tail Calls begrenzt die Komplexität der einzelnen eBPF-Programme. Kernelspace-PEP und Userspace-PEP verwenden dieselbe Policysemantik, dieselben Objektidentitäten und die Real-UID als definierte Subjektidentität.

Die Evaluation bestätigt die technische Machbarkeit für den untersuchten Funktionsumfang und das dokumentierte Zielsystem. Im vollständig archivierten Prüflauf vom 10. Oktober 2026 zum Commit `a9fd6c0` bestanden alle 65 Unit- und Komponententests, alle 19 privilegierten End-to-End-Szenarien und alle acht ergänzenden Evaluationsszenarien. Auch die sechs Python-Tests zur PERF-03-Messauswertung, die Bash-Prüfung des Veröffentlichungsverfahrens, die Formatprüfung, Clippy und der Release-Build waren erfolgreich. Dabei wurden unter anderem statische und dynamische Deny-Entscheidungen, die Combining-Regel *deny-overrides*, die Aktivierung vollständig vorbereiteter Generationen, die kontrollierte Ablehnung ungültiger Zustände und der selektive Entzug bestehender File Descriptors nach Policy- und Attributänderungen sowie beim Erreichen einer Zeitgrenze beobachtet. Der Kernel-Verifier akzeptierte das eBPF-Programm, und der LSM-Hook wurde erfolgreich angehängt. Die UID-Semantik stimmte auch bei voneinander abweichenden Real- und Effective-UIDs mit der festgelegten Real-UID-Semantik überein.

Die Laufzeitmessungen zeigen zugleich, dass die zusätzliche Kontrolle nicht kostenfrei ist. Der Median einer gemessenen Dateiöffnung stieg in der verwendeten Testumgebung von 4,223 μs ohne aktiven Prototyp auf 4,694 μs mit aktivem Prototyp ohne Policy und auf 4,604 μs bei einer passenden Permit-Policy. Dies entspricht in diesem Versuchsaufbau

einem medianen Mehraufwand von 11,2 beziehungsweise 9,0 Prozent. Policy- und Attributänderungen beeinflussten neue Entscheidungen im Median nach 10,42 beziehungsweise 10,54 ms. Vom Austausch einer Policydatei bis zum beobachteten `EBADF` eines zuvor geöffneten Deskriptors vergingen im Median 9,40 ms. Die Änderungszeiten werden wesentlich durch den 10-ms-Beobachtungsabstand mitbestimmt und sind keine präzisen internen Aktivierungszeiten. Diese Werte stammen aus demselben vollständigen Prüflauf ohne Debounce und charakterisieren die dokumentierte virtuelle Testumgebung; sie stellen weder eine Echtzeitgarantie noch einen isolierten Mikrobenchmark ausschließlich des LSM-Hooks dar.

Insgesamt erreicht der Prototyp damit das Ziel eines Machbarkeitsnachweises. Er zeigt, wie dynamische Entscheidungsgrundlagen in einen eBPF-LSM-Pfad einbezogen und bestehende Deskriptoren nach relevanten Zustandsänderungen erneut bewertet werden können. Er implementiert jedoch weder die vollständige Publish-Subscribe-Architektur von SAPL noch einen allgemeinen eigenständigen ASBAC-PDP. Ein geöffneter File Descriptor wird nicht als Subscription verwaltet, sondern bei einem Auslöser erneut aus dem zu diesem Zeitpunkt sichtbaren Systemzustand rekonstruiert. Die Bezeichnungen Kernelspace-PEP und Userspace-PEP beschreiben die jeweilige Durchsetzungsfunktion; beide Komponenten enthalten zugleich die für ihren Pfad erforderlichen Funktionen zur Policyentscheidung.

7.2 Grenzen der Ergebnisse

Der Schutzumfang setzt entsprechend Kapitel 3 einen vertrauenswürdigen Administrator und Kernel sowie integre Policy-, Attribut- und Zeitinformationen voraus. Ein Administrator mit ausreichenden Rechten kann die Entscheidungsgrundlagen verändern oder die Runtime beenden; gegen diese Eingriffe bietet der Prototyp keinen Schutz. Außerdem ist die zusätzliche Zugriffskontrolle an die Laufzeit des Hauptprogramms gebunden. Bei dessen Beendigung oder einem fatalen Fehler entfällt das nicht gepinnte LSM-Attachment, auch wenn die Maps weiterhin gepinnt sind. Eine ausfallsichere Weiterführung des Schutzes wird im Rahmen des Prototyps nicht beansprucht.

Die stärkste Durchsetzung bietet der Kernelspace-PEP für den eng definierten Zeitpunkt einer neuen Dateiöffnung. Die nachträgliche Behandlung eines bestehenden Zugriffs ist dagegen eine ereignisbasierte Best-Effort-Maßnahme. Entsprechend fordert FA-06 einen gezielten Schließversuch und die kontrollierte Behandlung von Fehlschlägen, keinen garantierten erfolgreichen Entzug. Das Durchlaufen von `/proc`, die erneute Policybewertung und der mittels `ptrace` ausgeführte `close`-Systemaufruf bilden keine atomare Operation. Zwischen Identifikation und Entzug kann der Zielprozess eine Deskriptornummer schließen oder neu belegen. Der nebenläufige Stresstest beobachtete zwar keinen Fehlentzug, kann Race-Freiheit aber nicht beweisen. Ein nicht erreichbarer Zielprozess kann nicht zum Schließen seines Deskriptors gezwungen werden. Fehlgeschlagene Schließversuche werden nicht selbstständig wiederholt. Ohne eine weitere Policy- oder Attributaktivierung oder eine relevante Zeitgrenze kann der betroffene Deskriptor daher geöffnet bleiben, selbst wenn der Zielprozess später wieder erreichbar ist.

Eine weitere Implementierungsgrenze betrifft die Laufzeitsteuerung: Der synchrone Userspace-Scan blockiert die übrigen Zweige desselben `tokio::select!`-Tasks. Dadurch können Policy- und Attributänderungen sowie die Aktualisierung von `CURRENT_TIME` verzögert werden, während der Kernelspace-PEP mit den zuletzt bereitgestellten Werten weiterarbeitet. Für das blockierende Warten beim `ptrace`-Entzug besteht keine explizite Zeitbegrenzung. Eine feste obere Grenze der Aktualisierungsverzögerung wird daher nicht zugesichert.

Das Schließen eines File Descriptors beendet außerdem nicht jede Nutzung, die aus einer früheren Dateiöffnung hervorgegangen ist. Die Evaluation zeigte, dass duplizierte und vererbte Deskriptoren in den untersuchten Prozessen gefunden und geschlossen wurden, eine bereits bestehende `mmap`-Abbildung jedoch weiterhin lesbar blieb. Bereits abgeschlossene Lesevorgänge können ebenso wenig rückgängig gemacht werden. Der Prototyp stellt deshalb keine vollständige fortdauernde Vermittlung sämtlicher Dateinutzungen und keinen vollständigen Reference Monitor dar.

Weitere Grenzen folgen aus dem festgelegten Implementierungsumfang. Kontrolliert wird ausschließlich der LSM-Hook `file_open`; andere Dateioperationen und Ressourcenarten werden nicht erfasst. Der Entzugsmechanismus ist an `x86_64`-Linux und ausreichende `ptrace`-Berechtigungen gebunden. Policies identifizieren Ressourcen über Device und Inode, sodass der Austausch einer Datei durch ein neues Kernelobjekt eine erneute Policyverarbeitung erfordert. Die Real-UID bildet effektive und Filesystem-UIDs sowie weitergehende Namespace-Semantiken nicht ab. Feste Map-Größen begrenzen pro Bank die Zahl statischer Policies und Streampolicies auf jeweils 16, die Zahl dynamischer Bedingungen pro Streampolicy auf vier und den gesamten Attributspeicher auf 1024 Einträge.

Auch die Evaluation besitzt begrenzte externe Validität. Die Versuche wurden auf einer einzelnen virtuellen Maschine, einem bestimmten Kernel und synthetischen Arbeitslasten ausgeführt. Die Latenzmessungen für Zustandsänderungen und Entzug umfassen jeweils nur zehn Wiederholungen; ihre p95-Werte sind daher keine belastbaren Schätzungen seltener Ausreißer. Ein Vergleich mit etablierten Linux-MAC-Systemen oder einer vollständigen SAPL-Laufzeit wurde nicht durchgeführt. Die Resultate belegen somit das Verhalten in den beschriebenen Szenarien, aber keine allgemeine Sicherheit, Skalierbarkeit oder Produktionstauglichkeit.

7.3 Ausblick

Ein vorrangiger Ansatzpunkt für weitere Arbeiten ist eine robustere Semantik für fortdauernde Zugriffe. Statt Nutzungssituationen nach einem Auslöser über `/proc` zu rekonstruieren, könnte eine Architektur Zugriffe und Entscheidungsänderungen ausdrücklich registrieren und als korrelierte Decision Streams verwalten. Damit ließe sich der Ansatz stärker an die Publish-Subscribe-Eigenschaften von ASBAC annähern. Zu untersuchen wäre, welche Ereignisse und Identifikatoren benötigt werden, um einen Zugriff über seinen Lebenszyklus stabil zu verfolgen, ohne einen File Descriptor selbst mit einer Subscription gleichzusetzen.

Für eine stärkere Durchsetzung müssten zusätzliche Zugriffspfade berücksichtigt werden. Weitere geeignete LSM-Hooks könnten Dateioperationen nach dem Öffnen vermitteln; ergänzende Mechanismen wären für Speicherabbildungen und bereits laufende E/A erforderlich. Besonders relevant ist eine Lösung, die das Zeitfenster zwischen der Identifikation eines Deskriptors und dessen Entzug verkleinert oder beseitigt. Dabei sind die Auswirkungen auf Nebenläufigkeit, Prozessstabilität, Berechtigungen und Portabilität systematisch zu untersuchen. Ein kooperativer Userspace-PEP im geschützten Prozess oder eine geeignete kernelgestützte Revocation-Schnittstelle könnten Alternativen zum derzeitigen `ptrace`-Eingriff bilden. Ergänzend könnten fehlgeschlagene Entzüge durch eine begrenzte Anzahl erneuter Versuche mit wachsenden Warteabständen behandelt werden. Vor jedem Versuch müssten die Prozess- und Ressourcenidentität sowie die aktuelle Policyentscheidung erneut geprüft werden, da Prozess- und Deskriptornummern zwischenzeitlich wiederverwendet werden können. Wiederholungsgrenzen, zusätzlicher Laufzeitaufwand und die verbleibenden Race-Bedingungen wären auf dem Zielsystem zu untersuchen.

Die Policy- und Identitätsmodelle bieten ebenfalls Erweiterungspotenzial. Zusätzliche LSM-Hooks und Ressourcenarten erfordern eigene Anfrage- und Map-Strukturen. Effektive-UID, Filesystem-UID, Capabilities, Cgroups sowie User- und Mount-Namespaces könnten als ausdrücklich typisierte Subjekt- oder Kontextattribute modelliert werden. Für größere Policybestände wären indexierte Suchstrukturen oder eine vorgelagerte Auswahl anwendbarer Policies zu untersuchen, da die derzeitigen festen linearen Scans bewusst auf kleine Mengen begrenzt sind.

Schließlich sollte eine Weiterentwicklung die Vertrauensgrenzen des Gesamtsystems härten. Dazu gehören minimale Capabilities für die Userspace-Komponenten, restriktive Rechte an Policy- und Attributquellen sowie an den gepinnten Maps, eine nachvollziehbare Herkunft und gegebenenfalls Signatur von Eingabedaten und ein individuelles Auditprotokoll für Entscheidungen und Entzugsversuche. Ergänzende Evaluationen auf physischer Hardware, weiteren Kernelversionen und unter realistischeren Mehrbenutzer- und Containerlasten könnten die Übertragbarkeit der Ergebnisse untersuchen. Größere Stichproben, isoliertere Hook-Mikrobenchmarks und Vergleiche mit bestehenden MAC- und ASBAC-Systemen würden zudem eine belastbarere Bewertung von Laufzeitaufwand und praktischem Nutzen ermöglichen.

Literaturverzeichnis

[And72] James P. Anderson. *Computer Security Technology Planning Study*. Techn. Ber. ESD-TR-73-51. Air Force Electronic Systems Division, 1972. URL: https://csrc.nist.gov/files/pubs/conference/1998/10/08/proceedings-of-the-21st-nissc-1998/final/docs/early-cs-papers/ande72.pdf.

[App] AppArmor Project. *AppArmor: Managing profiles*. Abschnitt Replace existing profiles. URL: https://apparmor-documentation-c38b15.gitlab.io/documentation/getting-started/managing-profiles/ (besucht am 05. 10. 2026).

[eBP26] eBPF Docs. *Pinning*. https://docs.ebpf.io/linux/concepts/pinning/. Zugriff am 12.06.2026. 2026.

[GGM18] Antonios Gouglidis, Christos Grompanopoulos und Anastasia Mavridou. „Formal Verification of Usage Control Models: A Case Study of UseCON Using TLA+“. In: *Electronic Proceedings in Theoretical Computer Science* 272 (2018), S. 52–64. DOI: 10.4204/EPTCS.272.5. URL: https://arxiv.org/abs/1806.09848.

[Heu21] Dominic Heutelbeck. „Demo: Attribute-Stream-Based Access Control (ASBAC) with the Streaming Attribute Policy Language (SAPL)“. In: *Proceedings of the 26th ACM Symposium on Access Control Models and Technologies*. Association for Computing Machinery, 2021, S. 95–97. DOI: 10.1145/3450569.3464397. URL: https://doi.org/10.1145/3450569.3464397.

[Heu] Dominic Heutelbeck. *SAPL: Introduction, Version 3.0.0*. URL: https://sapl.io/docs/3.0.0/1_1_Introduction/ (besucht am 05. 10. 2026).

[Hu+14] Vincent C. Hu u. a. *Guide to Attribute Based Access Control (ABAC) Definition and Considerations*. Techn. Ber. NIST Special Publication 800-162. National Institute of Standards und Technology, 2014. DOI: 10.6028/NIST.SP.800-162. URL: https://csrc.nist.gov/pubs/sp/800/162/upd2/final.

[IOV26] IOVisor Project. *BCC: BPF Compiler Collection*. https://github.com/iovisor/bcc. Zugriff am 15.06.2026. 2026.

[Ker26a] Michael Kerrisk. *bpf(2) Linux manual page*. https://man7.org/linux/man-pages/man2/bpf.2.html. Zugriff am 12.06.2026. 2026.

[Ker26b] Michael Kerrisk. *close(2) Linux manual page*. https://man7.org/linux/man-pages/man2/close.2.html. Zugriff am 03.10.2026. 2026.

[Ker26c] Michael Kerrisk. *dup(2) Linux manual page*. https://man7.org/linux/man-pages/man2/dup.2.html. Zugriff am 27.08.2026. 2026.

[Ker26d] Michael Kerrisk. *errno(3) Linux manual page*. https://man7.org/linux/man-pages/man3/errno.3.html. Zugriff am 08.08.2026. 2026.

[Ker26e] Michael Kerrisk. *fork(2) Linux manual page*. https://man7.org/linux/man-pages/man2/fork.2.html. Zugriff am 27.08.2026. 2026.

[Ker26f] Michael Kerrisk. *mmap(2) Linux manual page*. https://man7.org/linux/man-pages/man2/mmap.2.html. Zugriff am 27.08.2026. 2026.

[Ker26g] Michael Kerrisk. *open(2) Linux manual page*. https://man7.org/linux/man-pages/man2/open.2.html. Zugriff am 03.10.2026. 2026.

[Ker26h] Michael Kerrisk. *proc_pid_fd(5) Linux manual page*. https://man7.org/linux/man-pages/man5/proc_pid_fd.5.html. Zugriff am 12.06.2026. 2026.

[Ker26i] Michael Kerrisk. *proc_pid_status(5) Linux manual page*. https://man7.org/linux/man-pages/man5/proc_pid_status.5.html. Zugriff am 09.08.2026. 2026.

[Ker26j] Michael Kerrisk. *ptrace(2) Linux manual page*. https://man7.org/linux/man-pages/man2/ptrace.2.html. Zugriff am 27.08.2026. 2026.

[Kub] KubeArmor Project. *KubeArmor: Runtime Enforcer*. URL: https://docs.kubearmor.io/kubearmor/quick-links/kubearmor_overview/runtime_enforcer (besucht am 05. 10. 2026).

[lib26] libbpf Contributors. *libbpf*. https://github.com/libbpf/libbpf. Zugriff am 15.06.2026. 2026.

[Lin26a] Linux man-pages project. *proc_pid_comm(5) Linux manual page*. https://man7.org/linux/man-pages/man5/proc_pid_comm.5.html. Zugriff am 14.09.2026. 2026.

[Lin26b] Linux man-pages project. *rename(2): Linux manual page*. https://man7.org/linux/man-pages/man2/rename.2.html. Zugriff am 05.10.2026. 2026.

[MIT26] MITRE. *CWE-367: Time-of-check Time-of-use (TOCTOU) Race Condition*. https://cwe.mitre.org/data/definitions/367.html. Zugriff am 27.08.2026. 2026.

[Nix25] NixOS Contributors. *NixOS 25.05: Optionsdefinition system.stateVersion*. https://github.com/NixOS/nixpkgs/blob/nixos-25.05/nixos/modules/misc/version.nix. Optionsbeschreibung in version.nix, Zugriff am 16.09.2026. 2025.

[Nix26] NixOS Contributors. *NixOS Manual*. https://nixos.org/manual/nixos/stable/. Abschnitte zur deklarativen Systemkonfiguration, Zugriff am 21.09.2026. 2026.

[OAS17] OASIS. *eXtensible Access Control Markup Language (XACML) Version 3.0 Plus Errata 01*. https://docs.oasis-open.org/xacml/3.0/errata01/os/xacml-3.0-core-spec-errata01-os-complete.html. OASIS Standard incorporating Approved Errata, 12.07.2017; Zugriff am 13.09.2026. 2017.

[PS04] Jaehong Park und Ravi Sandhu. „The UCONABC Usage Control Model“. In: *ACM Transactions on Information and System Security* 7.1 (2004), S. 128–174. DOI: 10.1145/984334.984339.

[Rus26a] Rusty Russell. *Unreliable Guide To Hacking The Linux Kernel*. https://docs.kernel.org/kernel-hacking/hacking.html. Zugriff am 12.06.2026. 2026.

[SS75] Jerome H. Saltzer und Michael D. Schroeder. *The Protection of Information in Computer Systems*. https://web.mit.edu/Saltzer/www/publications/protection/. Online-Fassung, Zugriff am 12.06.2026. 1975.

[TMS11] Rafael Teigão, Carlos Maziero und Altair Santin. „Applying a usage control model in an operating system kernel“. In: *Journal of Network and Computer Applications* 34.4 (2011), S. 1342–1352. DOI: 10.1016/j.jnca.2011.03.019. URL: https://secplab.ppgia.pucpr.br/files/papers/2011-3.pdf (besucht am 05. 10. 2026).

[Teta] Tetragon Project. *Tetragon: Enforcement*. URL: https://tetragon.io/docs/concepts/enforcement/ (besucht am 05. 10. 2026).

[Tetb] Tetragon Project. *Tetragon: Filename access*. URL: https://tetragon.io/docs/use-cases/filename-access/ (besucht am 05. 10. 2026).

[Aya26a] The Aya Contributors. *Building eBPF Programs with Aya*. https://aya-rs.dev/book/. Zugriff am 12.06.2026. 2026.

[Aya26b] The Aya Contributors. *Implementation of the Aya eBPF Context*. https://github.com/aya-rs/aya/blob/b93ee8c26e96af85d77b040fa4fae8447c8fd7f8/ebpf/aya-ebpf/src/lib.rs. Commit b93ee8c26e96af85d77b040fa4fae8447c8fd7f8, Implementierung von EbpfContext::uid; Zugriff am 16.08.2026. 2026.

[Ker24a] The Kernel Development Community. *Linux v6.12: AppArmor file permission revalidation*. Funktion aa_file_perm. 2024. URL: https://github.com/torvalds/linux/blob/v6.12/security/apparmor/file.c (besucht am 05. 10. 2026).

[Ker24b] The Kernel Development Community. *Linux v6.12: SELinux file permission revalidation*. Funktion selinux_file_permission. 2024. URL: https://github.com/torvalds/linux/blob/v6.12/security/selinux/hooks.c (besucht am 05. 10. 2026).

[Ker25] The Kernel Development Community. *AppArmor*. 2025. URL: https://docs.kernel.org/admin-guide/LSM/apparmor.html (besucht am 21. 09. 2025).

[Ker26k] The Kernel Development Community. *BPF Design Q&A*. https://docs.kernel.org/bpf/bpf_design_QA.html. Zugriff am 12.06.2026. 2026.

[Ker26l] The Kernel Development Community. *BPF maps*. https://docs.kernel.org/bpf/maps.html. Zugriff am 12.06.2026. 2026.

[Ker26m] The Kernel Development Community. *BPF Type Format (BTF)*. https://docs.kernel.org/bpf/btf.html. Zugriff am 08.08.2026. 2026.

[Ker26n] The Kernel Development Community. *Credentials in Linux*. https://docs.kernel.org/security/credentials.html. Zugriff am 09.08.2026. 2026.

[Ker26o] The Kernel Development Community. *eBPF verifier*. https://docs.kernel.org/bpf/verifier.html. Zugriff am 12.06.2026. 2026.

[Ker26p] The Kernel Development Community. *Linux Security Module Usage*. https://docs.kernel.org/admin-guide/LSM/index.html. Zugriff am 12.06.2026. 2026.

[Ker26q] The Kernel Development Community. *LSM BPF Programs*. https://docs.kernel.org/bpf/prog_lsm.html. Zugriff am 12.06.2026. 2026.

[Lin24a] The Linux Kernel Developers. *Linux v6.12: Dateiöffnung in fs/open.c*. https://github.com/torvalds/linux/blob/v6.12/fs/open.c. do_dentry_open, Zeilen 855–907; do_sys_openat2, Zeilen 1317–1343; Zugriff am 03.10.2026. 2024.

[Lin24b] The Linux Kernel Developers. *Linux v6.12: Dokumentation und Implementierung der Datei-Sicherheitshooks in security/security.c*. https://github.com/torvalds/linux/blob/v6.12/security/security.c. security_file_permission, Zeilen 2650–2672; security_file_open, Zeilen 2911–2929; Zugriff am 03.10.2026. 2024.

[Lin24c] The Linux Kernel Developers. *Linux v6.12: Lese- und Schreibprüfungen in fs/read_write.c*. https://github.com/torvalds/linux/blob/v6.12/fs/read_write.c. rw_verify_area, Zeilen 422–448; vfs_read, Zeilen 517–545; vfs_write, Zeilen 624–651; Zugriff am 03.10.2026. 2024.

[Lin25a] The Linux Kernel Developers. *Linux BPF Helper Implementations*. https://github.com/torvalds/linux/blob/v6.16/kernel/bpf/helpers.c. Linux v6.16, Implementierung von bpf_get_current_uid_gid; Zugriff am 16.08.2026. 2025.

[Lin25b] The Linux Kernel Developers. *Linux Security Module Hook Definitions*. https://github.com/torvalds/linux/blob/v6.16/include/linux/lsm_hook_defs.h. Linux v6.16, Definition von file_open; Zugriff am 16.08.2026. 2025.

[Rus26b] The Rust Project Developers. *Build Scripts*. https://doc.rust-lang.org/cargo/reference/build-scripts.html. Zugriff am 23.07.2026. 2026.

[Wri+02] Chris Wright u. a. „Linux security modules: General security support for the Linux kernel“. In: *11th USENIX security symposium (USENIX Security 02)*. 2002. URL: https://www.usenix.org/legacy/events/sec02/full_papers/wright/wright.pdf (besucht am 12. 06. 2026).